A Tour of C++

- 문장별 근거 · 인쇄 점검

목차

검증 범위와 인쇄 점검 3
제1장. 기초 4
제2장. 사용자 정의 타입 19
제3장. 모듈화 27
제4장. 클래스 42
제5장. 필수적인 연산 58
제6장. 템플릿 73
제7장. 컨셉과 제네릭 프로그래밍 85
제8장. 라이브러리의 개요 95
제9장. 문자열과 정규 표현식 99
제10장. 입력과 출력 110
제11장. 컨테이너 124
제12장. 알고리즘 136
제13장. 유틸리티 149
제14장. 수치 167
제15장. 동시성 176
제16장. 역사와 호환성 190

검증 범위와 인쇄 점검

문장·목록·표의 항목·코드·수식 2202개에 근거 번호와 원문의 절·인쇄 쪽수·PDF 면 번호를 연결했습니다. 문장의 원문 위치는 기존 상세 원고의 절별 근거를 유지했으며, 추가한 정의·요약·치트·복습 답은 원문과 다시 대조했습니다. 같은 번호는 세 PDF에서 재사용합니다.

코드·수식·그림과 의심 표기는 원문 스캔을 우선했습니다. 의심스러운 내용을 외부 지식으로 교정하거나 미제공 부분을 채우지 않았습니다.

전체 페이지 렌더링 점검 후 결과를 반영합니다.

제1장. 기초

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-001	타입은 객체가 취할 수 있는 값과 연산의 집합을 정의한다.	(1.4, p.26 / PDF p.25)
U01-002	constexpr는 이 값을 컴파일 시간에 평가한다는 의미이며, constexpr의 값은 반드시 컴파일 시간에 계산된다.	(1.6, p.33 / PDF p.32)
U01-003	이 장은 C++의 표기법, 메모리 모델과 계산 모델, 코드를 조직해 프로그램을 만드는 기본 방법을 소개한다.	(1.1, p.20 / PDF p.19)
U01-004	C에서 흔히 볼 수 있는 스타일을 뒷받침하는 기능을 포함하며, 원문은 이 스타일을 절차적 프로그래밍이라고 부른다.	(1.1, p.20 / PDF p.19)
U01-005	C++는 컴파일되는 언어다.	(1.2, p.20-22 / PDF p.19-21)
U01-006	컴파일러가 소스 텍스트에서 목적 파일을 만들고, 링커가 목적 파일들을 병합해 실행 가능한 프로그램을 만든다.	(1.2, p.20-22 / PDF p.19-21)
U01-007	실행 파일은 특정 시스템·하드웨어 조합에 맞게 만들어진다.	(1.2, p.20-22 / PDF p.19-21)
U01-008	책에서 말하는 이식성은 주로 같은 소스를 여러 시스템에서 컴파일하고 실행할 수 있다는 뜻이다.	(1.2, p.20-22 / PDF p.19-21)
U01-009	ISO C++는 내장 타입·루프 등을 포함하는 핵심 언어 기능과 컨테이너·입출력 등을 포함하는 표준 라이브러리 구성 요소를 정의한다.	(1.2, p.20-22 / PDF p.19-21)
U01-010	표준 라이브러리는 보조적인 머신 코드를 제외하면 C++ 자체로 구현할 수 있다.	(1.2, p.20-22 / PDF p.19-21)
U01-011	C++는 정적 타입 언어이므로 컴파일러는 객체·값·이름·표현식을 사용하는 곳에서 그 타입을 알아야 한다.	(1.2, p.20-22 / PDF p.19-21)
U01-012	적용할 수 있는 연산은 객체의 타입으로 정해진다.	(1.2, p.20-22 / PDF p.19-21)
U01-013	모든 C++ 프로그램에는 전역 함수 main()이 하나 있어야 하며, main() 호출로 프로그램이 시작된다.	(1.2.1, p.21-23 / PDF p.20-22)
U01-014	중괄호는 함수 몸체를 묶고 //는 줄 끝까지 이어지는 주석이다.	(1.2.1, p.21-23 / PDF p.20-22)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-015	main()의 int 반환 값은 시스템에 전달되며, 반환 구문이 없으면 성공을 나타내는 값이 전달된다.	(1.2.1, p.21-23 / PDF p.20-22)
U01-016	원문은 0이 아닌 반환 값을 실패로 설명하면서, 운영체제·실행 환경마다 이 값의 활용 여부는 다르다고 덧붙인다.	(1.2.1, p.21-23 / PDF p.20-22)

U01-017	Hello, World! 출력 (원문 코드 발췌)
	#include <iostream>
	
	int main()
	{
	 std::cout << "Hello, World!\n";
	}
		(1.2.1, p.22 / PDF p.21)

U01-018	#include <iostream>은 표준 스트림 입출력 기능의 선언을 포함한다.	(1.2.1, p.21-23 / PDF p.20-22)
U01-019	std::cout은 표준 출력 스트림이며, std::는 표준 라이브러리 네임스페이스에서 이름을 찾도록 한정한다.	(1.2.1, p.21-23 / PDF p.20-22)
U01-020	문자열 리터럴은 큰따옴표로 둘러싼 문자들의 열이며 \n은 개행 문자다.	(1.2.1, p.21-23 / PDF p.20-22)
U01-021	실행 가능한 코드는 기본적으로 함수 안에 있고 main()에서 직접 또는 간접 호출된다.	(1.2.1, p.21-23 / PDF p.20-22)
U01-022	함수 정의는 어떤 작업을 어떻게 수행할지 명시한다.	(1.3, p.23-26 / PDF p.22-25)
U01-023	함수를 호출하기 전에 선언해야 하며, 선언에는 이름·반환 타입·인자의 개수와 타입이 들어간다.	(1.3, p.23-26 / PDF p.22-25)
U01-024	반환 타입은 이름 앞에, 인자 타입들은 이름 뒤의 괄호 안에 둔다.	(1.3, p.23-26 / PDF p.22-25)
U01-025	void는 반환 값이 없음을 뜻한다.	(1.3, p.23-26 / PDF p.22-25)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-026	인자의 타입을 검사하고 필요한 암묵적 변환을 수행하므로, 요구 타입과 맞지 않는 인자는 컴파일 시점에 거부될 수 있다.	(1.3, p.23-26 / PDF p.22-25)
U01-027	함수의 타입은 반환 타입과 인자 타입으로 이뤄진다.	(1.3, p.23-26 / PDF p.22-25)
U01-028	멤버 함수의 타입에는 클래스 이름도 포함된다.	(1.3, p.23-26 / PDF p.22-25)
U01-029	선언에 붙인 인자 이름은 독자를 돕지만, 정의가 아닌 선언에서는 컴파일러가 무시한다.	(1.3, p.23-26 / PDF p.22-25)
U01-030	원문은 의미 있는 작업을 짧은 함수로 나누고 이름을 붙이면 의존성을 드러내고 코드의 길이·복잡도에 따른 에러를 줄일 수 있다고 설명한다.	(1.3, p.23-26 / PDF p.22-25)

U01-031	인자 타입에 따른 함수 오버로딩 (원문 코드 발췌)
	void print(int); // 정수를 인자로 받음
	void print(double); // 부동소수점 수를 인자로 받음
	void print(string); // 문자열을 인자로 받음
	
	void user()
	{
	print(42); // print(int) 호출
	print(9.65); // print(double) 호출
	print("Barcelona"); // print(string) 호출
	}
		(1.3, p.25 / PDF p.24)

U01-032	같은 이름에 인자 타입이 다른 함수들을 정의하는 것을 함수 오버로딩이라고 한다.	(1.3, p.23-26 / PDF p.22-25)
U01-033	컴파일러는 호출마다 가장 적절한 함수를 선택한다.	(1.3, p.23-26 / PDF p.22-25)
U01-034	둘 이상을 호출할 수 있고 어느 쪽도 더 적합하지 않으면 모호한 호출 에러가 발생한다.	(1.3, p.23-26 / PDF p.22-25)
U01-035	오버로딩한 함수들은 모두 주어진 인자를 출력하는 print()처럼 같은 의미를 구현해야 한다.	(1.3, p.23-26 / PDF p.22-25)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-036	선언은 프로그램에 어떤 실체를 드러내고 그 타입을 지정한다.	(1.4, p.26-27 / PDF p.25-26)
U01-037	타입은 객체가 취할 수 있는 값과 연산의 집합이다.	(1.4, p.26-27 / PDF p.25-26)
U01-038	객체는 어떤 타입의 값을 갖는 메모리이며, 값은 타입에 따라 해석되는 비트의 집합이다.	(1.4, p.26-27 / PDF p.25-26)
U01-039	변수는 이름이 붙은 객체다.	(1.4, p.26-27 / PDF p.25-26)
U01-040	원문에 제시된 기본 타입은 bool, char, int, double, unsigned다.	(1.4, p.26-27 / PDF p.25-26)
U01-041	기본 타입은 하드웨어 기능에 직접 대응하고 고정된 크기를 갖지만, 실제 크기는 구현에 따라 다르다.	(1.4, p.26-27 / PDF p.25-26)
U01-042	char는 한 문자를 저장하는 크기이며 다른 타입의 크기는 char 크기의 배수다.	(1.4, p.26-27 / PDF p.25-26)
U01-043	sizeof로 크기를 확인한다.	(1.4, p.26-27 / PDF p.25-26)
U01-044	수 리터럴은 소수점이나 지수가 있으면 부동소수점 수로 인식한다.	(1.4, p.26-27 / PDF p.25-26)
U01-045	정수의 기본은 10진수이며 0b·0x·0 접두사는 각각 2진·16진·8진을 나타낸다.	(1.4, p.26-27 / PDF p.25-26)
U01-046	작은따옴표는 긴 리터럴의 자릿수 구분자로 사용할 수 있다.	(1.4, p.26-27 / PDF p.25-26)
U01-047	기본 타입의 적절한 조합에는 +, -, *, /, 정수 나머지 %와 단항 +··를 사용할 수 있다. ==·!=·<·>·<=·>=는 비교를 수행한다.	(1.4.1, p.27-29 / PDF p.26-28)
U01-048	비트별 논리 연산 &, |, ^, ~와 논리 연산 &&, ||, !를 구분한다.	(1.4.1, p.27-29 / PDF p.26-28)
U01-049	비트별 연산은 각 비트에 적용되며, &&와 ||는 연산 항의 값에 따라 true 또는 false를 반환한다.	(1.4.1, p.27-29 / PDF p.26-28)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-050	대입과 산술 연산에서는 기본 타입 간의 유효한 변환이 수행된다.	(1.4.1, p.27-29 / PDF p.26-28)
U01-051	일반 산술 변환은 주어진 연산 항에서 얻을 수 있는 최대 정밀도를 목표로 한다.	(1.4.1, p.27-29 / PDF p.26-28)
U01-052	double과 int의 덧셈은 배정도 부동소수점 연산으로 수행되지만, 그 결과를 int에 저장하면 정보가 버려질 수 있다. =는 대입이고 ==는 상등 비교다. +=, -=, *=, /=, %= 및 증가·감소 연산은 자주 쓰는 갱신을 간결하게 표현한다.	(1.4.1, p.27-29 / PDF p.26-28)
U01-053	원문은 표현식 평가와 대입 방향을 설명하고 함수 인자의 평가 순서는 정해져 있지 않다고 명시한다.	(1.4.1, p.27-29 / PDF p.26-28)
U01-054	객체는 사용 전에 값을 받아야 한다. = 형식과 중괄호 초깃값 목록 등을 사용할 수 있다.	(1.4.2, p.29-31 / PDF p.28-30)
U01-055	원문은 명명된 타입에 일반적인 {} 형식을 주로 사용하도록 권한다. {}는 정보 손실 위험이 있는 축소 변환을 막는다.	(1.4.2, p.29-31 / PDF p.28-30)
U01-056	반면 =를 이용한 기본 타입 변환에서는 double→int나 int→char와 같은 암묵적 축소 변환이 허용될 수 있다.	(1.4.2, p.29-31 / PDF p.28-30)

U01-057	초기화와 축소 변환 (원문 코드 발췌)
	int i1 = 7.8; // i1은 7이 됨(당황했는가?)
	int i2 {7.8}; // 에러: 부동소수점 수를 정수로 변환
		(1.4.2, p.30 / PDF p.29)

U01-058	상수는 초기화하지 않은 채 존재할 수 없다.	(1.4.2, p.29-31 / PDF p.28-30)
U01-059	변수도 적절한 초깃값을 얻기 전에는 정의하지 않는 것이 원문의 권고다.	(1.4.2, p.29-31 / PDF p.28-30)
U01-060	auto는 초깃값에서 타입을 유추해 반복되는 타입명을 줄인다.	(1.4.2, p.29-31 / PDF p.28-30)
U01-061	특히 정확한 타입명이 길거나 알기 어려운 제네릭 코드에 유용하다.	(1.4.2, p.29-31 / PDF p.28-30)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-062	스코프가 커서 독자에게 타입을 명확히 보이려 하거나 범위·정밀도를 지정하려는 경우처럼 이유가 있을 때는 구체적인 타입을 명시한다.	(1.4.2, p.29-31 / PDF p.28-30)
U01-063	선언은 이름을 스코프에 드러낸다.	(1.5, p.31-32 / PDF p.30-31)
U01-064	함수·람다 안의 지역 이름은 선언 지점부터 해당 블록 끝까지 유효하다.	(1.5, p.31-32 / PDF p.30-31)
U01-065	클래스 정의 안에서 함수·람다 밖에 둔 이름, 또는 enum class 안에서 정의한 이름은 멤버 이름이며 해당 선언의 스코프를 갖는다.	(1.5, p.31-32 / PDF p.30-31)
U01-066	네임스페이스 안에서 함수·람다·클래스·enum class 밖에 정의한 이름은 네임스페이스 멤버다.	(1.5, p.31-32 / PDF p.30-31)
U01-067	어떤 구성 요소에도 속하지 않는 이름은 전역 네임스페이스의 전역 이름으로 본다.	(1.5, p.31-32 / PDF p.30-31)
U01-068	이름의 유효 범위와 객체가 살아 있는 기간을 함께 구별한다.	(1.5, p.31-32 / PDF p.30-31)
U01-069	객체는 사용 전 생성·초기화되어야 하며 일반적으로 포함된 스코프 끝에서 소멸한다.	(1.5, p.31-32 / PDF p.30-31)
U01-070	네임스페이스 객체는 프로그램 종료 시 소멸하고, 멤버는 자신을 포함하는 객체의 소멸 시점에 따른다.	(1.5, p.31-32 / PDF p.30-31)
U01-071	new로 생성한 객체는 delete로 소멸할 때까지 유효하다.	(1.5, p.31-32 / PDF p.30-31)
U01-072	임시 객체나 new로 생성한 객체에는 이름이 없을 수도 있다.	(1.5, p.31-32 / PDF p.30-31)
U01-073	const는 값을 바꾸지 않겠다는 약속을 나타내며 컴파일러가 이를 강제한다.	(1.6, p.32-34 / PDF p.31-33)
U01-074	포인터나 참조로 전달된 데이터를 변경하지 않는다는 사실을 인터페이스에 표시할 수 있다.	(1.6, p.32-34 / PDF p.31-33)
U01-075	const 값은 실행 시간에 계산될 수 있다.	(1.6, p.32-34 / PDF p.31-33)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-076	constexpr는 값을 컴파일 시간에 평가한다는 뜻이며, constexpr 상수의 값은 반드시 컴파일 시간에 계산된다.	(1.6, p.32-34 / PDF p.31-33)

U01-077	constexpr 함수와 상수 표현식 (원문 코드 발췌)
	constexpr double square(double x) { return x*x; }
	
	constexpr double max1 = 1.4*square(17); // OK
	1.4*square(17)는 상수 표현식
	constexpr double max2 = 1.4*square(var); // 에러: var은 상수 표현식이 아님
	const double max3 = 1.4*square(var); // OK, 실행 시간에 평가할 수 있음
		(1.6, p.33 / PDF p.32)

U01-078	상수 표현식 안에서 호출할 함수도 constexpr로 정의해야 한다.	(1.6, p.32-34 / PDF p.31-33)
U01-079	constexpr 함수에 상수가 아닌 인자를 주는 것은 허용되지만 그 호출 결과는 상수 표현식이 아니다.	(1.6, p.32-34 / PDF p.31-33)
U01-080	따라서 같은 연산을 상수 버전·일반 버전으로 중복 정의할 필요가 없다.	(1.6, p.32-34 / PDF p.31-33)
U01-081	원문은 constexpr 함수가 단순하고 부수 효과가 없으며 인자로 전달된 정보만 사용해야 한다고 설명한다.	(1.6, p.32-34 / PDF p.31-33)
U01-082	지역 변수와 루프는 허용되지만 지역이 아닌 변수를 변경할 수 없다.	(1.6, p.32-34 / PDF p.31-33)

번호	문장·표 항목·원문 코드·수식	원문 위치

U01-083	지역 변수와 루프를 사용하는 constexpr (원문 코드 발췌)
	constexpr double nth(double x, int n) // 0<=n이라고 가정
	{
	double res = 1;
	int i = 0;
	while (i<n) { // while 루프: 조건이 true일 동안 반복(1.7.1절)
	res*=x;
	++i;
	}
	return res;
	}
		(1.6, p.34 / PDF p.33)

U01-084	원문에서 상수 표현식이 필요한 사례는 배열의 경계, case 라벨, 템플릿 값 인자, constexpr로 선언한 상수다.	(1.6, p.32-34 / PDF p.31-33)
U01-085	컴파일 시간 평가는 성능과 관계되며, 상태가 변하지 않는 객체라는 불변성 자체도 설계의 고려 사항이다.	(1.6, p.32-34 / PDF p.31-33)
U01-086	배열은 연속된 영역에 할당되는 동일 타입의 시퀀스다.	(1.7, p.34-37 / PDF p.33-36)
U01-087	배열의 아래쪽 경계는 0이고, 크기는 상수 표현식이어야 한다.	(1.7, p.34-37 / PDF p.33-36)
U01-088	포인터 변수는 적절한 타입 객체가 있는 주소를 저장한다.	(1.7, p.34-37 / PDF p.33-36)
U01-089	선언에서 T*는 T를 가리키는 포인터, 표현식의 단항 &는 주소, 접두사 *는 가리키는 객체의 내용을 뜻한다.	(1.7, p.34-37 / PDF p.33-36)
U01-090	일반 for는 초기화·조건 검사·증가를 명시한다.	(1.7, p.34-37 / PDF p.33-36)
U01-091	구간 for는 시퀀스의 요소들을 처음부터 끝까지 탐색한다.	(1.7, p.34-37 / PDF p.33-36)
U01-092	auto x는 요소를 복사하고, auto& x는 요소 자체를 참조한다.	(1.7, p.34-37 / PDF p.33-36)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-093	참조는 접근할 때 *를 쓰지 않으며 일단 초기화되면 다른 객체를 참조하도록 바꿀 수 없다.	(1.7, p.34-37 / PDF p.33-36)
U01-094	참조 인자는 원본을 복사하지 않고 변경할 때 사용하며, const 참조는 복사 없이 변경도 방지할 때 사용한다.	(1.7, p.34-37 / PDF p.33-36)

U01-095	구간 for와 요소 참조 (원문 코드 발췌)
	void increment()
	{
	int v[] = {0,1,2,3,4,5,6,7,8,9};
	
	for (auto& x : v) // v에 포함된 모든 x에 1을 더함
	++x;
	// ...
	}
		(1.7, p.36 / PDF p.35)

U01-096	유효하게 역참조하려면 포인터가 실제 객체를 가리켜야 한다.	(1.7.1, p.37-39 / PDF p.36-38)
U01-097	가리킬 객체가 없거나 가용한 객체가 없음을 나타낼 때 nullptr를 저장한다.	(1.7.1, p.37-39 / PDF p.36-38)
U01-098	모든 포인터 타입에 같은 nullptr 값을 사용하며, nullptr는 정수가 아니다.	(1.7.1, p.37-39 / PDF p.36-38)
U01-099	원문은 0·NULL 대신 nullptr를 사용해 정수와 포인터의 혼동을 줄이라고 권한다.	(1.7.1, p.37-39 / PDF p.36-38)

번호	문장·표 항목·원문 코드·수식	원문 위치

U01-100	널 검사 후 C 스타일 문자열 탐색 (원문 코드 발췌)
	int count_x(const char* p, char x)
	// p[]에 존재하는 x와 동일한 값의 수를 센다.
	// p는 0으로 끝나는 char의 배열을 가리킨다고 가정(그렇지 않으면 아무것도 가리키지 않음)
	{
	if (p==nullptr)
	return 0;
	int count = 0;
	for (; *p!=0; ++p)
	if (*p==x)
	++count;
	return count;
	}
		(1.7.1, p.38 / PDF p.37)

U01-101	수치 값의 조건 테스트는 0과의 비교, 포인터의 조건 테스트는 nullptr와의 비교에 해당한다.	(1.7.1, p.37-39 / PDF p.36-38)
U01-102	while는 조건이 false가 될 때까지 반복한다.	(1.7.1, p.37-39 / PDF p.36-38)
U01-103	참조에는 널 참조라는 개념이 없으며 반드시 유효한 객체를 참조해야 한다.	(1.7.1, p.37-39 / PDF p.36-38)
U01-104	if와 switch는 선택을, while와 for는 반복을 표현한다.	(1.8, p.39-42 / PDF p.38-41)
U01-105	입력 연산자 >>는 표준 입력 스트림 cin에서 값을 읽으며, 오른쪽 연산 항 타입이 받아들일 입력을 결정한다.	(1.8, p.39-42 / PDF p.38-41)
U01-106	변수는 구문을 둘 수 있는 곳에서 선언할 수 있으므로 원문은 실제로 필요한 위치에 정의한다.	(1.8, p.39-42 / PDF p.38-41)
U01-107	switch는 값을 서로 다른 상수 case 라벨들과 비교한다.	(1.8, p.39-42 / PDF p.38-41)
U01-108	어느 라벨과도 맞지 않으면 default가 선택되며, default도 없으면 아무 동작을 하지 않는다.	(1.8, p.39-42 / PDF p.38-41)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-109	함수를 반환하지 않고 switch 다음으로 진행하려면 break를 사용한다.	(1.8, p.39-42 / PDF p.38-41)
U01-110	여러 case를 같은 행동에 연결할 수도 있다.	(1.8, p.39-42 / PDF p.38-41)

U01-111	if 초기화 절과 조건 (원문 코드 발췌)
	void do_something(vector<int>& v)
	{
	if (auto n = v.size(); n!=0) {
	// ... n!=0 이면 여기를 실행...
	}
	// ...
	}
		(1.8, p.42 / PDF p.41)

U01-112	if의 초기화 절에서 선언한 이름은 다른 브랜치에서도 유효하다.	(1.8, p.39-42 / PDF p.38-41)
U01-113	for에서처럼 변수의 스코프를 제한해 가독성을 높이고 에러를 줄이려는 목적이다. 0 또는 nullptr와 비교할 때는 조건 값을 직접 테스트하는 간단한 형식도 사용할 수 있다.	(1.8, p.39-42 / PDF p.38-41)
U01-114	C++의 기본 연산은 하드웨어 기능에 직접 대응하며 대부분 하나의 기계어로 처리된다.	(1.9, p.42-43 / PDF p.41-42)
U01-115	구현은 메모리를 일련의 위치로 바라보고, 타입이 있는 객체를 그 위치에 저장하며, 포인터로 위치를 가리킨다.	(1.9, p.42-43 / PDF p.41-42)
U01-116	포인터는 머신 주소로 표현된다.	(1.9, p.42-43 / PDF p.41-42)
U01-117	배열은 메모리상의 연속된 객체들을 추상화한 것이다.	(1.9, p.42-43 / PDF p.41-42)
U01-118	원문은 이 기본 머신 모델이 수학이 아니라 컴퓨터 하드웨어에 기반한다고 설명한다.	(1.9, p.42-43 / PDF p.41-42)
U01-119	내장 타입의 대입은 머신의 복사 연산이다.	(1.9.1, p.43-45 / PDF p.42-44)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-120	x=y 뒤에 x와 y는 같은 값을 갖지만 서로 독립된 객체다.	(1.9.1, p.43-45 / PDF p.42-44)
U01-121	포인터 대입 p=q도 주소 값을 복사하므로 두 포인터 자체는 독립적이지만 같은 객체를 가리키게 된다.	(1.9.1, p.43-45 / PDF p.42-44)
U01-122	반면 참조에 대입하면 참조 대상을 바꾸는 대신 이미 참조하고 있는 객체에 값을 쓴다.	(1.9.1, p.43-45 / PDF p.42-44)
U01-123	원문은 모든 내장 타입에서 x=y의 결과로 x==y가 성립하며, 대입·상등 비교를 제공하는 잘 설계된 사용자 정의 타입에도 같은 기대를 적용한다.	(1.9.1, p.43-45 / PDF p.42-44)
U01-124	초기화는 초기화되지 않은 메모리를 유효한 객체로 만드는 일이고, 대입은 이미 값을 가진 객체에 새 값을 쓰는 일이다.	(1.9.2, p.45-46 / PDF p.44-45)
U01-125	초기화되지 않은 변수를 읽거나 쓰는 효과는 일반적으로 예측할 수 없다.	(1.9.2, p.45-46 / PDF p.44-45)
U01-126	참조는 초기화 없이 만들 수 없으며, int& r = x도 값 복사가 아니라 r을 x에 바인딩하는 초기화다.	(1.9.2, p.45-46 / PDF p.44-45)
U01-127	string·vector처럼 해제할 자원을 소유하는 타입에서도 두 동작의 구별이 중요하다.	(1.9.2, p.45-46 / PDF p.44-45)
U01-128	함수 인자 전달과 반환 값의 기본 동작은 초기화와 유사하다.	(1.9.2, p.45-46 / PDF p.44-45)
U01-129	의미 있는 연산을 짧은 함수로 묶고 한 함수는 하나의 논리적 연산을 수행하게 한다.	(1.10, p.46-48 / PDF p.45-47)
U01-130	비슷한 연산은 같은 의미로 오버로딩한다.	(1.10, p.46-48 / PDF p.45-47)
U01-131	변수는 반드시 초기화하고, 초깃값이 생길 때까지 선언을 미루며, 스코프를 작게 유지한다.	(1.10, p.46-48 / PDF p.45-47)
U01-132	축소 변환과 복잡한 표현식을 피한다.	(1.10, p.46-48 / PDF p.45-47)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-133	마법의 수 대신 기호 상수를 쓰고 변경 불가한 데이터를 선호한다.	(1.10, p.46-48 / PDF p.45-47)
U01-134	컴파일 시간 평가가 필요한 함수에는 constexpr를 쓴다.	(1.10, p.46-48 / PDF p.45-47)
U01-135	명명된 타입은 {} 초기화를 주로 사용하고 auto로 타입 이름 반복을 줄인다.	(1.10, p.46-48 / PDF p.45-47)
U01-136	선언 하나에는 이름 하나를 둔다.	(1.10, p.46-48 / PDF p.45-47)
U01-137	unsigned는 비트 조작에 사용하고 포인터는 단순하게 사용한다. 0·NULL 대신 nullptr를 쓴다.	(1.10, p.46-48 / PDF p.45-47)
U01-138	이름의 범위와 용도에 맞게 길이를 정하고 비슷한 이름·모두 대문자인 이름을 피한다.	(1.10, p.46-48 / PDF p.45-47)
U01-139	코드가 이미 보여 주는 사실보다 의도를 주석에 쓰고 들여쓰기를 일관되게 한다.	(1.10, p.46-48 / PDF p.45-47)
U01-140	의미	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-141	값을 바꾸지 않는다는 약속	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-142	컴파일 시간에 평가	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-143	계산 시점	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-144	실행 시간 계산 가능	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-145	상수의 값은 컴파일 시간 계산	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-146	함수 인자와의 관계	(const와 constexpr, p.32-34 / PDF p.31-33)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-147	const 참조로 변경 방지	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-148	비상수 인자를 주면 결과는 상수 표현식이 아님	(const와 constexpr, p.32-34 / PDF p.31-33)
U01-149	대입 대상	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-150	저장한 주소 값	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-151	이미 참조하는 객체	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-152	재지정	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-153	다른 객체 주소를 저장 가능	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-154	초기화 뒤 참조 대상을 바꾸지 못함	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-155	없음의 표현	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-156	nullptr 사용	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-157	유효한 객체를 반드시 참조	(포인터와 참조의 대입, p.44-46 / PDF p.43-45)
U01-158	객체는 어떤 타입의 값을 갖는 메모리다.	(1.4, p.26 / PDF p.25)
U01-159	값은 타입에 따라 해석되는 비트의 집합이다.	(1.4, p.26 / PDF p.25)
U01-160	변수는 명명된 객체다.	(1.4, p.26 / PDF p.25)
U01-161	지역 이름의 유효 스코프는 선언 지점부터 선언을 포함하는 블록의 끝까지다.	(1.5, p.31 / PDF p.30)

번호	문장·표 항목·원문 코드·수식	원문 위치
U01-162	new로 생성한 객체는 delete로 소멸할 때까지 유효하다.	(1.5, p.32 / PDF p.31)
U01-163	const는 이 값을 바꾸지 않는다고 약속하며, const의 값은 실행 시간에 계산할 수 있다.	(1.6, p.33 / PDF p.32)
U01-164	함수 하나는 논리적인 연산 하나를 수행하게 하고 짧게 유지한다.	(1.10, p.47 / PDF p.46)
U01-165	변수는 반드시 초기화하고 스코프는 작게 유지한다.	(1.10, p.47 / PDF p.46)
U01-166	명명된 타입에는 초깃값 목록 {}를 사용하고 auto로 타입 이름의 반복을 피한다.	(1.10, p.47 / PDF p.46)
U01-167	0이나 NULL 대신 nullptr를 사용한다.	(1.10, p.47 / PDF p.46)
U01-168	타입은 값과 연산의 집합을 정의하고, 객체는 타입의 값을 갖는 메모리이며, 값은 타입에 따라 해석되는 비트의 집합이고 변수는 명명된 객체다.	(1.4, p.26 / PDF p.25)
U01-169	const 값은 실행 시간에 계산할 수 있지만 constexpr 값은 반드시 컴파일 시간에 계산된다.	(1.6, p.33 / PDF p.32)
U01-170	const	(1.6, p.33 / PDF p.32)
U01-171	constexpr	(1.6, p.33 / PDF p.32)

제2장. 사용자 정의 타입

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-001	클래스는 데이터·함수·타입을 멤버로 가질 수 있으며, 인터페이스는 public 멤버로 정의된다.	(2.3, p.53 / PDF p.52)
U02-002	enum class는 독립된 타입이며 열거자의 스코프를 해당 enum class로 제한한다.	(2.5, p.58 / PDF p.57)
U02-003	기본 타입, const 제한자, 선언 연산에서 만들 수 있는 타입은 내장 타입이다.	(2.1, p.49-50 / PDF p.48-49)
U02-004	원문은 내장 타입이 하드웨어를 직접적·효율적으로 반영하도록 의도적으로 로우레벨에 가깝게 설계되었다고 설명한다.	(2.1, p.49-50 / PDF p.48-49)
U02-005	C++는 내장 타입과 연산에 추상화 메커니즘을 조합해 프로그래머가 필요한 하이레벨 기능을 만들도록 한다.	(2.1, p.49-50 / PDF p.48-49)
U02-006	이렇게 만든 사용자 정의 타입은 적절한 표현 방식과 연산을 갖추며, 클래스와 열거형을 예로 들 수 있다.	(2.1, p.49-50 / PDF p.48-49)
U02-007	내장 타입이나 다른 사용자 정의 타입을 기반으로 만들 수 있다.	(2.1, p.49-50 / PDF p.48-49)
U02-008	잘 설계한 사용자 정의 타입은 사용하기 쉽고 에러 위험이 적으며 효율적일 수 있다.	(2.1, p.49-50 / PDF p.48-49)
U02-009	표준 라이브러리도 주로 사용자 정의 타입으로 구성된다.	(2.1, p.49-50 / PDF p.48-49)
U02-010	새로운 타입을 만드는 첫 단계는 필요한 구성 요소를 데이터 구조로 조직화하는 것이다.	(2.2, p.50-52 / PDF p.49-51)
U02-011	struct Vector의 첫 버전은 요소의 개수 sz와 요소들을 가리키는 double* elem을 묶는다.	(2.2, p.50-52 / PDF p.49-51)
U02-012	Vector 변수를 선언하는 것만으로는 elem이 사용할 배열을 가리킨다는 보장이 없으므로 vector_init()이 별도로 필요하다.	(2.2, p.50-52 / PDF p.49-51)

번호	문장·표 항목·원문 코드·수식	원문 위치

U02-013	Vector의 데이터 표현 (원문 코드 발췌)
	struct Vector {
	int sz; // 요소의 개수
	double* elem; // 요소를 가리키는 포인터
	};
		(2.2, p.50 / PDF p.49)

U02-014	구조체를 초기화하는 별도 함수 (원문 코드 발췌)
	void vector_init(Vector& v, int s)
	{
	v.elem = new double[s]; // s개의 double로 이뤄진 배열 할당
	v.sz = s;
	}
		(2.2, p.51 / PDF p.50)

U02-015	new는 자유 저장 영역, 동적 메모리 또는 힙이라고 부르는 곳에 메모리를 할당한다.	(2.2, p.50-52 / PDF p.49-51)
U02-016	그 객체는 생성된 스코프와 무관하게 delete로 소멸할 때까지 유효하다.	(2.2, p.50-52 / PDF p.49-51)
U02-017	Vector&는 const가 아닌 참조이므로 vector_init()은 인자로 받은 원본 Vector를 변경할 수 있다.	(2.2, p.50-52 / PDF p.49-51)
U02-018	이름·참조를 통해 struct 멤버에 접근할 때는 .을 쓰고, 포인터를 통해서는 ->를 쓴다.	(2.2, p.50-52 / PDF p.49-51)
U02-019	원문의 read_and_sum()은 Vector를 초기화하고 입력을 요소에 저장한 뒤 합을 계산한다.	(2.2, p.50-52 / PDF p.49-51)
U02-020	그러나 사용자는 elem과 sz라는 표현의 세부 사항을 알아야 한다.	(2.2, p.50-52 / PDF p.49-51)
U02-021	원문은 이 Vector를 언어 기능·설계 기법을 설명하기 위한 점진적 예제로 쓰며, 표준 vector나 string을 재발명하기보다 표준 라이브러리 구성 요소를 활용하라고 권한다.	(2.2, p.50-52 / PDF p.49-51)
U02-022	사용자 정의 타입을 일관되게 사용하고 표현을 나중에 개선하려면, 모두가 쓰는 인터페이스와 외부에서 접근할 수 없는 데이터를 사용하는 구현을 분리한다.	(2.3, p.53-55 / PDF p.52-54)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-023	class가 이를 지원한다.	(2.3, p.53-55 / PDF p.52-54)
U02-024	클래스는 데이터·함수·타입을 멤버로 가질 수 있고 public 멤버가 인터페이스를 정의한다.	(2.3, p.53-55 / PDF p.52-54)
U02-025	private 멤버에 대한 접근은 인터페이스를 통해 이뤄진다.	(2.3, p.53-55 / PDF p.52-54)

U02-026	인터페이스와 표현을 분리한 Vector (원문 코드 발췌)
	class Vector {
	public:
	Vector(int s) :elem{new double[s]}, sz{s} { } // Vector 생성
	double& operator[](int i) { return elem[i]; } // 요소 접근: 첨자 연산
	int size() { return sz; }
	private:
	double* elem; // 요소를 가리키는 포인터
	int sz; // 요소의 개수
	};
		(2.3, p.53 / PDF p.52)

U02-027	Vector와 이름이 같은 멤버 함수는 생성자다.	(2.3, p.53-55 / PDF p.52-54)
U02-028	객체를 만들 때 생성자가 반드시 사용되므로 별도 초기화 함수 호출을 잊는 실수를 막는다.	(2.3, p.53-55 / PDF p.52-54)
U02-029	Vector(int)는 생성 시 요소 개수를 정수로 주어야 함을 드러낸다.	(2.3, p.53-55 / PDF p.52-54)
U02-030	멤버 초깃값 목록의 elem{new double[s]}는 할당된 배열의 주소로 elem을, sz{s}는 s로 sz를 초기화한다.	(2.3, p.53-55 / PDF p.52-54)
U02-031	operator[]는 적절한 요소의 double&를 반환해 읽기와 쓰기를 제공하고, size()는 요소 개수를 돌려준다.	(2.3, p.53-55 / PDF p.52-54)
U02-032	Vector 객체 자체는 elem과 sz를 갖는 고정 크기의 핸들이며, 실제 요소는 다른 곳에 저장된다.	(2.3, p.53-55 / PDF p.52-54)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-033	따라서 객체마다 요소 개수가 달라도 핸들 자체의 크기는 같다.	(2.3, p.53-55 / PDF p.52-54)
U02-034	원문은 이러한 핸들 기법이 개수가 변하는 데이터에 흔히 쓰인다고 설명한다.	(2.3, p.53-55 / PDF p.52-54)
U02-035	이 단계의 Vector에는 아직 에러 처리와 할당한 배열의 해제가 없다.	(2.3, p.53-55 / PDF p.52-54)
U02-036	원문은 이를 각각 3.5절과 4.2.2절에서 보완한다.	(2.3, p.53-55 / PDF p.52-54)
U02-037	struct와 class에는 근본적 차이가 없으며, struct는 멤버의 기본 접근이 public인 class다.	(2.3, p.53-55 / PDF p.52-54)
U02-038	따라서 struct에도 생성자를 포함한 멤버 함수를 둘 수 있다.	(2.3, p.53-55 / PDF p.52-54)
U02-039	union은 모든 멤버를 같은 메모리 주소에 할당하는 구조체이며, 원문은 공용체의 크기를 가장 큰 멤버의 크기로 설명한다.	(2.4, p.55-57 / PDF p.54-56)
U02-040	일반적으로 실제 값을 갖는 멤버는 하나뿐이다.	(2.4, p.55-57 / PDF p.54-56)
U02-041	원문의 Entry 예에서는 Node*와 int가 동시에 사용되지 않아 별도 멤버로 두면 공간이 낭비된다.	(2.4, p.55-57 / PDF p.54-56)
U02-042	둘을 union Value에 넣으면 메모리를 절약한다.	(2.4, p.55-57 / PDF p.54-56)

U02-043	Node*와 int의 저장 공간 공유 (원문 코드 발췌)
	union Value {
	 Node* p;
	 int i;
	};
		(2.4, p.56 / PDF p.55)

U02-044	언어는 union에 현재 어느 종류의 값이 있는지 확인하지 않으므로 프로그래머가 관리해야 한다.	(2.4, p.55-57 / PDF p.54-56)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-045	Entry의 Type t가 ptr이면 v.p, num이면 v.i를 쓰는 관계를 지켜야 한다.	(2.4, p.55-57 / PDF p.54-56)
U02-046	타입 필드와 실제 저장된 타입의 관계를 직접 관리하면 에러가 발생하기 쉽다.	(2.4, p.55-57 / PDF p.54-56)
U02-047	원문은 공용체와 타입 필드를 캡슐화하고 멤버 함수로만 접근하게 하는 태그된 공용체의 추상화를 권한다.	(2.4, p.55-57 / PDF p.54-56)

U02-048	variant로 표현하는 대안 값 (원문 코드 발췌)
	struct Entry {
	string name;
	variant<Node*,int> v;
	};
	
	void f(Entry* pe)
	{
	if (holds_alternative<int>(pe->v)) // *pe가 int를 포함하는가? (13.5.1절 참고)
	cout << get<int>(pe->v); // int 가져오기
	// ...
	}
		(2.4, p.57 / PDF p.56)

U02-049	표준 라이브러리 variant는 대안적인 여러 타입 중 하나의 값을 제공하며 대부분의 상황에서 union을 대체할 수 있다.	(2.4, p.55-57 / PDF p.54-56)
U02-050	variant<Node*,int>는 Node* 또는 int를 저장한다.	(2.4, p.55-57 / PDF p.54-56)
U02-051	holds_alternative<int>()로 int 포함 여부를 확인하고 get<int>()로 값을 가져오는 원문 예는 별도의 타입 필드를 수작업으로 관리하는 경우보다 간단하고 안전하다는 설명에 연결된다.	(2.4, p.55-57 / PDF p.54-56)
U02-052	열거형은 여러 정수 값의 집합을 명명된 상수들로 표현하는 사용자 정의 타입이다.	(2.5, p.57-59 / PDF p.56-58)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-053	enum class는 열거자의 스코프를 해당 열거형으로 제한하므로 Color::red와 Traffic_light::red를 혼동하지 않고 같은 red 이름을 사용할 수 있다.	(2.5, p.57-59 / PDF p.56-58)
U02-054	두 열거형은 독립된 타입이며 서로의 값이나 정수와 암묵적으로 혼용할 수 없다.	(2.5, p.57-59 / PDF p.56-58)
U02-055	서로 독립된 enum class (원문 코드 발췌) enum class Color { red, blue , green }; enum class Traffic_light { green, yellow, red }; Color col = Color::red; Traffic_light light = Traffic_light::red;	(2.5, p.57 / PDF p.56)
U02-056	원문은 기반 타입에서 열거형을 초기화할 필요가 있으면 Color{5}나 Color y {6}처럼 명시적 초기화를 할 수 있다고 설명한다.	(2.5, p.57-59 / PDF p.56-58)
U02-057	기본적으로 enum class는 대입·초기화·비교를 정의하며, 사용자 정의 타입이므로 연산을 추가할 수 있다.	(2.5, p.57-59 / PDF p.56-58)
U02-058	Traffic_light의 전치 증가 연산은 green→yellow→red→green 순서의 변화를 정의한다.	(2.5, p.57-59 / PDF p.56-58)

U02-059	열거형의 전치 증가 연산 정의 (원문 코드 발췌)
	Traffic_light& operator++(Traffic_light& t) // 전치 증가: ++
	{
	switch (t) {
	case Traffic_light::green: return t=Traffic_light::yellow;
	case Traffic_light::yellow: return t=Traffic_light::red;
	case Traffic_light::red: return t=Traffic_light::green;
	}
	}
	
	Traffic_light next = ++light; // 다음 값은 Traffic_light::green
		(2.5, p.59 / PDF p.58)

U02-060	class 없이 enum을 쓰면 열거자 이름은 enum 이름과 같은 스코프에 속하고, 열거자 값은 정수로 암묵 변환된다.	(2.5, p.57-59 / PDF p.56-58)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-061	원문의 enum Color { red, green, blue }; int col = green;에서 col은 1이다.	(2.5, p.57-59 / PDF p.56-58)
U02-062	기본 열거자 값은 0에서 시작해 하나씩 증가한다.	(2.5, p.57-59 / PDF p.56-58)
U02-063	원문은 예측하지 못한 에러를 줄이기 위해 enum class를 선호하라고 조언한다.	(2.5, p.57-59 / PDF p.56-58)
U02-064	내장 타입이 너무 로우레벨이면 잘 정의된 사용자 정의 타입을 쓰고, 관련 데이터는 struct나 class로 조직한다.	(2.6, p.59-60 / PDF p.58-59)
U02-065	class로 인터페이스와 구현을 분리하고 생성자로 초기화를 보장·단순화한다.	(2.6, p.59-60 / PDF p.58-59)
U02-066	벌거벗은 union을 피하고 공용체와 타입 필드를 클래스로 감싼다.	(2.6, p.59-60 / PDF p.58-59)
U02-067	명명된 상수 집합에는 열거형을 사용한다.	(2.6, p.59-60 / PDF p.58-59)
U02-068	enum보다 enum class를 선호하고 안전하고 쉽게 쓸 수 있도록 필요한 연산을 정의한다.	(2.6, p.59-60 / PDF p.58-59)
U02-069	struct / class	(원문에서 비교한 데이터 표현, p.53-59 / PDF p.52-58)
U02-070	struct는 기본 멤버 접근이 public인 class이며 근본적 차이는 없다.	(원문에서 비교한 데이터 표현, p.53-59 / PDF p.52-58)
U02-071	직접 union / variant	(원문에서 비교한 데이터 표현, p.53-59 / PDF p.52-58)
U02-072	union의 저장 타입은 프로그래머가 관리한다. variant는 대안 타입 중 하나를 제공한다.	(원문에서 비교한 데이터 표현, p.53-59 / PDF p.52-58)
U02-073	enum / enum class	(원문에서 비교한 데이터 표현, p.53-59 / PDF p.52-58)
U02-074	enum은 이름이 바깥 스코프에 있고 정수로 암묵 변환된다. enum class는 스코프와 타입을 제한한다.	(원문에서 비교한 데이터 표현, p.53-59 / PDF p.52-58)

번호	문장·표 항목·원문 코드·수식	원문 위치
U02-075	해당 클래스와 이름이 동일한 멤버 함수를 생성자라고 하며, 클래스를 객체화할 때 사용한다.	(2.3, p.54 / PDF p.53)
U02-076	원문의 Vector 객체는 요소 포인터 elem과 요소 개수 sz를 포함하는 핸들이며, Vector 객체 자체의 크기는 항상 같다.	(2.3, p.54 / PDF p.53)
U02-077	struct는 멤버가 기본적으로 public인 class이며, struct에도 생성자를 포함한 멤버 함수를 정의할 수 있다.	(2.3, p.55 / PDF p.54)
U02-078	union은 모든 멤버가 같은 메모리 주소에 할당되는 struct이며, 공용체의 크기는 가장 큰 멤버의 크기와 같다.	(2.4, p.55 / PDF p.54)
U02-079	variant는 대안적인 여러 타입의 값 중 하나를 제공하며, 대부분의 상황에서 공용체를 대체할 수 있다.	(2.4, p.57 / PDF p.56)
U02-080	관련된 데이터는 struct나 class로 구조화하고 class로 인터페이스와 구현을 분리한다.	(2.6, p.59 / PDF p.58)
U02-081	클래스의 초기화를 보장하고 단순화하도록 생성자를 정의한다.	(2.6, p.60 / PDF p.59)
U02-082	벌거벗은 union은 피하고 공용체와 타입 필드를 클래스로 감싼다.	(2.6, p.60 / PDF p.59)
U02-083	명명된 상수 집합은 열거형으로 표현하며, 예상하지 못한 에러를 줄이기 위해 enum class를 사용한다.	(2.6, p.60 / PDF p.59)
U02-084	struct	(2.3, p.55 / PDF p.54)
U02-085	variant	(2.4, p.57 / PDF p.56)

제3장. 모듈화

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-001	네임스페이스는 일부 선언을 함께 묶고 다른 이름과 충돌하지 않게 하는 메커니즘이다.	(3.4, p.68 / PDF p.67)
U03-002	어써션은 기대되는 조건을 나타내는 구문이며, static_assert는 상수 표현식으로 표현할 수 있는 속성을 확인한다.	(3.5.5, p.80 / PDF p.79)
U03-003	독립적으로 개발된 함수·타입·계층 구조·템플릿을 관리하려면 상호작용을 명확히 정의해야 한다.	(3.1, p.61-63 / PDF p.60-62)
U03-004	첫 단계는 인터페이스와 구현을 분리하는 것이다.	(3.1, p.61-63 / PDF p.60-62)
U03-005	C++는 선언으로 인터페이스를 표현하고 함수·타입을 사용하는 데 필요한 사항을 명시한다.	(3.1, p.61-63 / PDF p.60-62)
U03-006	함수 몸체인 정의는 다른 곳에 둘 수 있다.	(3.1, p.61-63 / PDF p.60-62)
U03-007	클래스 밖의 멤버 정의에서는 Vector::처럼 클래스 이름으로 한정한다.	(3.1, p.61-63 / PDF p.60-62)
U03-008	라이브러리도 같은 언어 기능으로 만든 다른 코드이므로 기본 원리는 같다.	(3.1, p.61-63 / PDF p.60-62)
U03-009	원문은 구성 요소를 여러 번 선언할 수 있지만 정의는 한 번만 가능하다고 설명한다.	(3.1, p.61-63 / PDF p.60-62)
U03-010	사용자 코드는 타입·함수의 선언만 있으면 컴파일할 수 있고, 정의는 다른 파일에서 따로 컴파일할 수 있다.	(3.2, p.63-66 / PDF p.62-65)
U03-011	분할 컴파일은 논리적으로 구별되는 부분을 강제로 나누고 컴파일 시간을 줄이는 데 도움이 된다.	(3.2, p.63-66 / PDF p.62-65)
U03-012	모듈의 인터페이스 선언은 보통 헤더에 두고, 사용자와 구현 파일이 모두 같은 헤더를 포함해 컴파일러가 일관성을 확인하게 한다.	(3.2, p.63-66 / PDF p.62-65)

번호	문장·표 항목·원문 코드·수식	원문 위치

U03-013	분할 컴파일을 위한 헤더 (원문 코드 발췌)
	// Vector.h:
	
	class Vector {
	public:
	Vector(int s);
	double& operator[](int i);
	int size();
	private:
	double* elem; // elem은 sz개의 double을 포함하는 배열을 가리킴
	int sz;
	};
		(3.2, p.64 / PDF p.63)

U03-014	헤더를 사용하는 독립 사용자 파일 (원문 코드 발췌)
	// user.cpp:
	
	#include "Vector.h" // Vector의 인터페이스 가져오기
	#include <cmath> // sqrt()를 포함하는 표준 라이브러리 수학 함수 인터페이스 가져오기
	
	double sqrt_sum(Vector& v)
	{
	double sum = 0;
	for (int i=0; i!=v.size(); ++i)
	sum+=std::sqrt(v[i]); // 제곱근의 합
	return sum;
	}
		(3.2, p.64 / PDF p.63)

U03-015	Vector.h는 인터페이스를 공유하는 지점이며 user.cpp와 Vector.cpp는 독립적으로 컴파일한다.	(3.2, p.63-66 / PDF p.62-65)
U03-016	원문은 프로그램을 의존성이 잘 정의된 모듈의 집합으로 생각하고 파일을 통한 물리적 모듈화를 활용하라고 권한다.	(3.2, p.63-66 / PDF p.62-65)
U03-017	독립적으로 컴파일하는 .cpp 파일과 그 파일이 #include하는 헤더를 포함한 단위를 변환 단위라고 부른다.	(3.2, p.63-66 / PDF p.62-65)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-018	한 프로그램은 수천 개의 변환 단위로 구성될 수 있다.	(3.2, p.63-66 / PDF p.62-65)
U03-019	#include는 사용하는 변환 단위마다 헤더 텍스트를 다시 처리한다.	(3.3, p.66-68 / PDF p.65-67)
U03-020	또한 먼저 포함한 헤더의 선언·매크로가 뒤 헤더의 의미에 영향을 줄 수 있어 포함 순서가 중요해지는 문제가 있다.	(3.3, p.66-68 / PDF p.65-67)
U03-021	원문 시점의 module은 아직 ISO C++가 아니라 ISO 기술 명세에 포함된 기능이며, 세부 사항이 바뀔 수 있고 제품 수준에서 사용하려면 더 시간이 필요할 수 있다고 설명된다.	(3.3, p.66-68 / PDF p.65-67)
U03-022	이 요약은 그 상태 설명을 그대로 유지한다.	(3.3, p.66-68 / PDF p.65-67)

U03-023	모듈 선언과 노출할 클래스 (원문 코드 발췌)
	// Vector.cpp 파일:
	module; // module을 정의
	
	// ... Vector 구현에 필요한 내용 작성 ...
	
	export module Vector; // "Vector" 모듈 정의
	
	export class Vector {
	public:
	Vector(int s);
	double& operator[](int i);
	int size();
	private:
	double* elem; // elem은 sz개의 double을 포함하는 배열을 가리킴
	int sz;
	};
		(3.3, p.66-67 / PDF p.65-66)

U03-024	export는 모듈 사용자에게 보여 줄 인터페이스를 노출하고, 필요한 곳에서 import로 가져온다.	(3.3, p.66-68 / PDF p.65-67)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-025	원문은 import와 #include를 함께 써 기존 코드에서 점진적으로 옮기는 방법을 보인다.	(3.3, p.66-68 / PDF p.65-67)
U03-026	모듈은 사용하는 모든 변환 단위마다 다시 컴파일하지 않고 한 번 컴파일한다.	(3.3, p.66-68 / PDF p.65-67)
U03-027	두 모듈의 import 순서는 의미에 영향을 주지 않으며 import는 전이되지 않는다.	(3.3, p.66-68 / PDF p.65-67)
U03-028	즉 모듈 내부에서 가져온 이름을 그 모듈의 사용자가 암묵적으로 접근·조작할 수 있는 것은 아니다.	(3.3, p.66-68 / PDF p.65-67)
U03-029	네임스페이스는 일부 선언을 묶고 독립적으로 개발한 이름들이 충돌하지 않도록 한다.	(3.4, p.68-70 / PDF p.67-69)
U03-030	다른 네임스페이스의 이름은 std::cout 또는 My_code::main처럼 이름을 한정해 접근한다.	(3.4, p.68-70 / PDF p.67-69)
U03-031	프로그램의 실제 main()은 전역 네임스페이스에 둔다.	(3.4, p.68-70 / PDF p.67-69)
U03-032	using std::swap과 같은 using 선언은 특정 이름을 현재 스코프에서 선언한 것처럼 사용하게 한다.	(3.4, p.68-70 / PDF p.67-69)
U03-033	using namespace std와 같은 using 지시자는 네임스페이스의 모든 이름을 사용할 수 있게 한다.	(3.4, p.68-70 / PDF p.67-69)
U03-034	후자는 이름을 선택적으로 가져오는 것이 아니므로 신중히 사용한다.	(3.4, p.68-70 / PDF p.67-69)
U03-035	원문은 애플리케이션 전반에서 쓰는 기반 라이브러리나 기존 프로그램을 개선하는 과정에서의 사용을 설명하고, 헤더에는 using 지시자를 넣지 말라고 조언한다.	(3.4, p.68-70 / PDF p.67-69)
U03-036	에러 처리의 가장 중요한 도구는 타입 시스템이다.	(3.5, p.71 / PDF p.70)
U03-037	내장 타입과 제어 구문만 사용하기보다 적절한 사용자 정의 타입과 알고리즘으로 높은 수준의 개념을 표현하면 실수 가능성을 낮추고 컴파일러가 에러를 찾을 가능성을 높인다.	(3.5, p.71 / PDF p.70)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-038	추상화는 런타임 에러를 탐지하는 지점과 처리하는 지점도 분리한다.	(3.5, p.71 / PDF p.70)
U03-039	라이브러리를 많이 쓰는 큰 프로그램일수록 일관된 전략이 중요하므로 설계 초기에 에러 처리 방식을 고려한다.	(3.5, p.71 / PDF p.70)
U03-040	Vector 작성자는 범위 밖 접근 시 사용자가 무엇을 해야 할지 알 수 없고, 사용자는 잘못된 접근을 항상 감지할 수 없다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-041	따라서 구현자가 문제를 탐지해 알리고 사용자가 조치를 취하게 한다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-042	예제의 operator[]는 인덱스가 범위 밖이면 out_of_range를 던진다.	(3.5.1, p.71-73 / PDF p.70-72)

U03-043	범위 검사와 예외 핸들러 (원문 코드 발췌)
	double& Vector::operator[](int i)
	{
	if (i<0 || size()<=i)
	throw out_of_range{"Vector::operator[]"};
	return elem[i];
	}
	
	void f(Vector& v)
	{
	// ...
	try { // 여기서 발생한 예외를 아래에 정의한 핸들러가 처리
	v[v.size()] = 7; // v의 끝을 넘어서 접근 시도
	}
	catch (out_of_range& err) { // out_of_range 예외
	// ... 스코프 밖 에러 처리...
	cerr << err.what() << '\n';
	}
	// ...
	}
		(3.5.1, p.72 / PDF p.71)

U03-044	throw는 해당 타입의 핸들러에 제어를 넘긴다.	(3.5.1, p.71-73 / PDF p.70-72)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-045	구현은 호출 스택을 거슬러 올라가며 필요한 스코프·함수를 종료하고 그 과정에서 소멸자를 호출한다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-046	try에 처리할 코드를 놓고 catch에서 예외를 잡는다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-047	예외는 복사를 피하려고 참조로 잡으며 what()은 던질 때 설정한 메시지를 제공한다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-048	모든 함수에서 try를 반복하기보다 생성자에서 자원을 획득하고 소멸자에서 해제하는 RAII를 이용하는 것이 원문의 권고다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-049	예외를 던지지 않는 함수는 noexcept로 선언할 수 있다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-050	그런 함수에서 예외가 빠져나오면 std::terminate()를 호출해 프로그램을 즉시 종료한다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-051	선언만으로 내부 호출이 성공한다고 보장되는 것은 아니라는 점을 원문의 user() 예제가 보여 준다.	(3.5.1, p.71-73 / PDF p.70-72)
U03-052	함수의 사전 조건은 작동의 기본 가정이다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-053	Vector 인덱싱은 인덱스가 [0:size()) 안에 있어야 한다는 전제를 검사한다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-054	[a:b)는 a를 포함하고 b는 포함하지 않는 반개구간이다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-055	클래스 불변 조건은 멤버 함수가 의지할 수 있도록 클래스 수준에서 보장되어야 하는 조건이다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-056	Vector에서는 elem이 sz개의 double을 갖는 배열을 가리킨다는 관계가 이에 해당한다.	(3.5.2, p.73-76 / PDF p.72-75)

번호	문장·표 항목·원문 코드·수식	원문 위치

U03-057	생성자에서 유효한 크기 확인 (원문 코드 발췌)
	Vector::Vector(int s)
	{
	if (s<0)
	throw length_error{"Vector 생성자: 크기가 음수임"};
	elem = new double[s];
	sz = s;
	}
		(3.5.2, p.74 / PDF p.73)

U03-058	생성자는 불변 조건을 성립시키고, 멤버 함수는 종료할 때 불변 조건을 보존해야 한다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-059	s<0이면 length_error를 던지고, new가 메모리를 할당하지 못하면 std::bad_alloc을 던진다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-060	사용자 정의 예외 타입으로 탐지 지점에서 처리 지점까지 필요한 정보를 전달할 수도 있다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-061	함수를 완수하지 못하면 최소한의 지역 정리를 하고 throw;로 다시 던지거나, 복구가 불가능하면 종료하는 예를 원문이 제시한다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-062	불변 조건은 무엇을 원하는지 명확하게 하고 더 구체적인 설계를 강제하며, 디버깅·테스팅을 거쳐 올바른 코드를 유지하게 돕는다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-063	생성자·소멸자 기반 자원 관리도 이 개념에 기초한다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-064	원문의 생성자 조건은 s<0인데 이어지는 설명은 “양수가 아님”이라고 표현한다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-065	[원문 확인 필요] 이 요약에서는 코드의 음수 검사 조건을 바꾸지 않았다.	(3.5.2, p.73-76 / PDF p.72-75)
U03-066	에러를 지역적으로 처리할 수 없다면 호출자에게 실패를 알려야 한다.	(3.5.3, p.76-78 / PDF p.75-77)
U03-067	원문은 예외, 실패를 나타내는 값, 프로그램 종료를 비교한다.	(3.5.3, p.76-78 / PDF p.75-77)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-068	C++의 예외는 정상적인 값을 대신 반환하는 기법이 아니라 요청받은 작업을 완수하지 못했음을 보고하도록 설계되었고 생성자·소멸자와 통합된다.	(3.5.3, p.76-78 / PDF p.75-77)
U03-069	에러 코드: 실패가 흔하고 예상 가능하며 직접 호출자가 처리할 것으로 기대할 수 있는 경우.	(3.5.3, p.76-78 / PDF p.75-77)
U03-070	파일 열기 실패가 예다.	(3.5.3, p.76-78 / PDF p.75-77)
U03-071	예외: 에러가 드물어 확인을 잊기 쉬운 경우, 직접 호출자가 처리하기 어려운 경우, 새 에러 종류가 늘 수 있는 경우, 생성자처럼 반환 경로가 없는 경우, 값과 에러를 함께 반환하면 경로가 복잡한 경우.	(3.5.3, p.76-78 / PDF p.75-77)
U03-072	예외가 적합한 다른 경우: 여러 호출을 거슬러 전달해야 하거나 복구가 여러 호출 결과에 의존하는 경우, 콜백에서 실패한 경우, 작업 취소가 필요한 경우.	(3.5.3, p.76-78 / PDF p.75-77)
U03-073	종료: 복구할 수 없거나, 심각한 에러를 스레드·프로세스·컴퓨터 재시작으로 처리하도록 설계한 시스템.	(3.5.3, p.76-78 / PDF p.75-77)
U03-074	원문은 에러 코드와 예외 중 어느 하나를 완전히 배제하지 말고, 판단이 어려우면 예외를 사용하라고 권한다.	(3.5.3, p.76-78 / PDF p.75-77)
U03-075	예외가 항상 느리다는 믿음도 부정하며, 복잡하거나 드문 실패를 처리하고 반복적인 에러 코드 확인을 피할 때 빠를 수도 있다고 설명한다.	(3.5.3, p.76-78 / PDF p.75-77)
U03-076	간결한 예외 처리에는 RAII가 필수적이다.	(3.5.3, p.76-78 / PDF p.75-77)
U03-077	원문 시점에는 사전 조건·불변 조건을 선택적으로 실행 시간에 검사하는 일반적인 표준 방법이 없다고 설명한다.	(3.5.4, p.79 / PDF p.78)
U03-078	C++20에 계약 메커니즘이 제안되어 테스트에서는 엄격히 검사하고 배포에서는 최소 검사하는 용도가 소개되지만, 원문은 당시 임시 방편을 사용해야 한다고 명시한다.	(3.5.4, p.79 / PDF p.78)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-079	예로 RANGE_CHECK 매크로에 따라 범위 검사를 수행하는 코드를 제시한다.	(3.5.4, p.79 / PDF p.78)
U03-080	표준 라이브러리의 assert()는 주어진 실행 시간 조건을 확인하는 디버그용 매크로다.	(3.5.4, p.79 / PDF p.78)
U03-081	디버그 모드에서 조건이 성립하지 않으면 종료하며, 디버그 모드가 아니면 검사하지 않는다.	(3.5.4, p.79 / PDF p.78)
U03-082	원문은 유연성은 적지만 충분한 경우가 많다고 설명한다.	(3.5.4, p.79 / PDF p.78)
U03-083	실행 시간 에러는 예외로 보고하되, 컴파일 시간에 찾을 수 있는 에러는 그때 찾는 것이 좋다.	(3.5.5, p.80-81 / PDF p.79-80)
U03-084	기대 조건을 표현하는 구문을 어써션이라고 하며 static_assert는 상수 표현식으로 나타낼 수 있는 속성을 검사한다.	(3.5.5, p.80-81 / PDF p.79-80)
U03-085	static_assert(A,S)에서 A가 true가 아니면 S를 컴파일러 에러 메시지로 출력한다.	(3.5.5, p.80-81 / PDF p.79-80)
U03-086	S를 생략하면 기본 메시지를 사용한다.	(3.5.5, p.80-81 / PDF p.79-80)
U03-087	컴파일 시간 속성 검사 (원문 코드 발췌) static_assert(4<=sizeof(int), "정수가 너무 작음"); // 정수 크기 확인	(3.5.5, p.80 / PDF p.79)
U03-088	원문의 속력 예제는 함수 인자 speed는 상수 표현식이 아니므로 static_assert에 쓸 수 없고 constexpr local_max는 사용할 수 있음을 보여 준다.	(3.5.5, p.80-81 / PDF p.79-80)
U03-089	제네릭 코드의 타입 파라미터에 대한 가정을 확인하는 용도로도 유용하다.	(3.5.5, p.80-81 / PDF p.79-80)
U03-090	함수 호출은 프로그램의 두 부분 사이에 정보를 전달하는 주된 방법이다.	(3.6, p.81-82 / PDF p.80-81)
U03-091	작업에 필요한 정보는 인자로, 결과는 반환 값으로 전달한다.	(3.6, p.81-82 / PDF p.80-81)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-092	전역 변수는 에러의 주요 원인이므로 피하고, 공유 상태는 클래스 멤버처럼 함께 잘 정의된 추상 개념을 구현하는 함수들로 제한한다.	(3.6, p.81-82 / PDF p.80-81)
U03-093	객체의 복사·공유 여부, 변경 허용 여부, 빈 객체를 남기는 이동 가능성이 경계 설계의 고려 사항이다.	(3.6, p.81-82 / PDF p.80-81)
U03-094	기본은 복사지만 암묵적 이동으로 최적화될 수 있다.	(3.6, p.81-82 / PDF p.80-81)
U03-095	값에 의한 전달은 복사본을 사용하고, 참조에 의한 전달은 호출자 환경의 원본 객체를 참조한다.	(3.6.1, p.82-83 / PDF p.81-82)
U03-096	원문의 test()는 값 인자 v를 수정해도 원본은 그대로이고, 참조 인자 rv를 수정하면 원본이 바뀜을 출력으로 보여 준다.	(3.6.1, p.82-83 / PDF p.81-82)
U03-097	작은 객체는 값으로, 복사 비용이 큰 객체는 참조로 전달한다. “작다”는 기준은 아키텍처에 따라 달라지며, 원문은 대략 포인터 두세 개 크기 이하를 제시한다.	(3.6.1, p.82-83 / PDF p.81-82)
U03-098	성능상 참조를 사용하되 변경이 필요 없으면 const 참조를 선호한다.	(3.6.1, p.82-83 / PDF p.81-82)
U03-099	기본 함수 인자는 자주 쓰는 기본값을 선언에 지정한다.	(3.6.1, p.82-83 / PDF p.81-82)
U03-100	원문의 print(int value, int base =10)는 base를 생략하면 10을 사용하며, 기본값을 넘기는 별도 오버로드보다 문법이 간단하다.	(3.6.1, p.82-83 / PDF p.81-82)
U03-101	반환의 기본 동작도 복사이며 작은 값에는 적합하다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-102	참조 반환은 함수 호출과 독립적으로 존재하는 객체에 호출자가 접근할 때 사용한다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-103	Vector 요소는 인덱싱 호출 뒤에도 존재하므로 참조로 반환할 수 있다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-104	반면 지역 변수는 함수가 끝나면 사라지므로 그 포인터·참조를 반환하면 안 된다.	(3.6.2, p.84-86 / PDF p.83-85)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-105	큰 Matrix 결과도 값을 반환하는 인터페이스를 유지하면서 이동 생성자로 효율적으로 전달할 수 있다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-106	new로 만든 Matrix*를 반환하고 사용자가 delete하게 하는 예전 스타일은 직접 메모리 관리와 해제 누락 위험을 키운다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-107	작업을 완료하지 못하면 예외를 던진다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-108	auto 반환 타입 추론은 함수 템플릿·람다에서 유용하지만 구현 변경이 반환 타입 변경으로 이어질 수 있어 안정적인 인터페이스를 위해 주의한다.	(3.6.2, p.84-86 / PDF p.83-85)
U03-109	함수는 하나의 값만 반환하지만 그 값이 여러 멤버를 가진 객체일 수 있다.	(3.6.3, p.86-88 / PDF p.85-87)
U03-110	Entry 읽기 예제는 문자열·정수를 하나의 객체로 반환한다.	(3.6.3, p.86-88 / PDF p.85-87)
U03-111	구조화된 바인딩은 그 객체의 멤버에 지역 이름을 부여한다.	(3.6.3, p.86-88 / PDF p.85-87)
U03-112	auto [n,v]의 타입은 반환 객체에서 유추하며, 일반 auto처럼 const와 &를 붙일 수 있다.	(3.6.3, p.86-88 / PDF p.85-87)

번호	문장·표 항목·원문 코드·수식	원문 위치

U03-113	여러 멤버를 가진 반환 객체 (원문 코드 발췌)
	struct Entry {
	string name;
	int value;
	};
	
	Entry read_entry(istream& is) // 원시적인 읽기 함수(10.5절에서 더 나은 버전을 볼 수 있음)
	{
	string s;
	int i;
	is >> s >> i;
	return {s,i};
	}
	
	auto e = read_entry(cin);
	cout << "{ " << e.name << " , " << e.value << " }\n";
		(3.6.3, p.86 / PDF p.85)

U03-114	구조화된 바인딩으로 멤버 사용 (원문 코드 발췌) auto [n,v] = read_entry(is); cout << "{ " << n << " , " << v << " }\n";	(3.6.3, p.87 / PDF p.86)
U03-115	private 데이터가 없는 클래스의 단순한 경우 바인딩 이름의 수는 비정적 데이터 멤버 수와 같아야 하고 이름들은 각 멤버에 대응한다.	(3.6.3, p.86-88 / PDF p.85-87)
U03-116	원문은 멤버를 직접 지칭할 때와 객체 코드 품질 차이는 없지만 아이디어 표현에 유용하다고 설명한다.	(3.6.3, p.86-88 / PDF p.85-87)
U03-117	접근 함수를 가진 complex의 구조화된 바인딩 예도 제시하지만 그 구현 기법은 책의 범위 밖이라고 명시한다.	(3.6.3, p.86-88 / PDF p.85-87)
U03-118	선언과 정의를 구별하고 인터페이스를 헤더로 표현한다.	(3.7, p.88-89 / PDF p.87-88)
U03-119	구현 파일도 해당 헤더를 포함한다.	(3.7, p.88-89 / PDF p.87-88)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-120	지원되는 경우 모듈을 선호한다.	(3.7, p.88-89 / PDF p.87-88)
U03-121	네임스페이스로 논리적 구조를 표현하고 헤더에 using 지시자를 넣지 않는다.	(3.7, p.88-89 / PDF p.87-88)
U03-122	설계 초기부터 에러 전략을 정한다.	(3.7, p.88-89 / PDF p.87-88)
U03-123	생성자는 불변 조건을 지키거나 예외를 던지며, 에러는 적절한 지점에서 처리한다.	(3.7, p.88-89 / PDF p.87-88)
U03-124	모든 함수에서 모든 예외를 잡으려 하지 말고 명시적 try 반복보다 RAII를 이용한다.	(3.7, p.88-89 / PDF p.87-88)
U03-125	던지지 않는 함수는 noexcept로 선언한다.	(3.7, p.88-89 / PDF p.87-88)
U03-126	컴파일 시간에 확인할 수 있는 것은 그때 확인한다.	(3.7, p.88-89 / PDF p.87-88)
U03-127	작은 것은 값으로, 큰 것은 참조로 전달하고 변경이 필요 없으면 const 참조를 선호한다.	(3.7, p.88-89 / PDF p.87-88)
U03-128	결과는 함수 반환 값으로 전달하며 반환 타입 추론·구조화된 바인딩을 과용하지 않는다.	(3.7, p.88-89 / PDF p.87-88)
U03-129	명명된 반환 타입은 문서화에 유리할 수 있다.	(3.7, p.88-89 / PDF p.87-88)
U03-130	처리	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-131	변환 단위마다 헤더 텍스트 처리	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-132	모듈 한 번 컴파일	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-133	순서	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-134	선언·매크로가 포함 순서에 영향	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-135	import 순서가 의미에 영향 없음	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-136	가시성	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-137	포함한 선언·매크로가 뒤 코드에 영향	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-138	import는 전이되지 않음	(헤더와 원문 시점의 모듈, p.66-68 / PDF p.65-67)
U03-139	값 인자	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-140	작고 복사 비용이 낮은 객체	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-141	참조 인자	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-142	원본 접근 또는 큰 객체 복사 회피	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-143	const 참조	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-144	복사를 피하고 원본을 변경하지 않음	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-145	값 반환	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-146	결과 전달의 기본 방법, 큰 값은 이동 가능	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-147	참조 반환	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)

번호	문장·표 항목·원문 코드·수식	원문 위치
U03-148	호출과 독립적으로 존재하는 객체 접근	(함수 경계의 전달 방법, p.82-86 / PDF p.81-85)
U03-149	throw는 해당 타입의 예외 핸들러에 제어권을 넘기며, 필요한 만큼 스코프와 함수를 종료하는 과정에서 소멸자를 호출한다.	(3.5.1, p.72 / PDF p.71)
U03-150	절대로 예외를 던지지 않는 함수는 noexcept로 선언할 수 있으며, 그래도 예외를 던지면 std::terminate()가 호출된다.	(3.5.1, p.73 / PDF p.72)
U03-151	사전 조건은 함수 작동의 기본적인 가정이다.	(3.5.2, p.73 / PDF p.72)
U03-152	클래스 수준에서 보장돼야 할 조건을 클래스 불변 조건이라고 한다.	(3.5.2, p.74 / PDF p.73)
U03-153	선언과 정의를 구별하고 인터페이스를 표현하기 위해 헤더 파일을 사용한다.	(3.7, p.88 / PDF p.87)
U03-154	설계 초기부터 에러 처리 전략을 세우고, 생성자는 불변 조건을 지키거나 예외를 던진다.	(3.7, p.88-89 / PDF p.87-88)
U03-155	컴파일 시간에 확인할 수 있는 것은 그때 확인하며, 작은 것은 값으로 큰 것은 참조로 전달한다.	(3.7, p.89 / PDF p.88)
U03-156	값을 반환할 때는 함수 반환 값을 사용하고 출력 파라미터는 피한다.	(3.7, p.89 / PDF p.88)
U03-157	사전 조건은 함수 작동의 기본 가정이고 클래스 불변 조건은 클래스 수준에서 보장돼야 할 조건이다.	(3.5.2, p.73-74 / PDF p.72-73)
U03-158	static_assert(A,S)는 A가 true가 아니면 S를 컴파일러 에러 메시지로 출력한다.	(3.5.5, p.80 / PDF p.79)
U03-159	예외	(3.5.5, p.80 / PDF p.79)
U03-160	예외는 실행 시간에 발생하는 에러를 보고하는 용도로 사용한다.	(3.5.5, p.80 / PDF p.79)
U03-161	static_assert	(3.5.5, p.80 / PDF p.79)

제4장. 클래스

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-001	구체 타입은 타입 정의의 일부로 메모리 표현을 포함하며, 내장 타입처럼 작동하는 것이 기본 아이디어다.	(4.2, p.93 / PDF p.91)
U04-002	구현 상속은 기반 클래스의 함수와 데이터 멤버로 파생 클래스의 구현을 단순하게 한다.	(4.5.1, p.112 / PDF p.110)
U04-003	클래스는 프로그램에서 어떤 개념을 표현하는 사용자 정의 타입이다.	(4.1, p.91-93 / PDF p.89-91)
U04-004	원문은 유용한 아이디어를 클래스라는 코드로 직접 표현하면 내장 타입만으로 구현한 경우보다 이해하고 디버깅하기 쉽다고 설명한다.	(4.1, p.91-93 / PDF p.89-91)
U04-005	이 장은 구체 클래스, 추상 클래스, 계층 구조에 속하는 클래스를 다룬다.	(4.1, p.91-93 / PDF p.89-91)
U04-006	뒤의 장에서는 생애 주기 연산, 템플릿, 제네릭 프로그래밍으로 이어진다.	(4.1, p.91-93 / PDF p.89-91)
U04-007	구체 타입의 기본 생각은 내장 타입처럼 작동하도록 하는 것이다.	(4.2, p.93-94 / PDF p.91-92)
U04-008	complex는 자신만의 산술 연산을 가진다는 점을 제외하면 int처럼 쓰고, vector·string은 개선된 배열처럼 쓴다.	(4.2, p.93-94 / PDF p.91-92)
U04-009	타입 정의의 일부로 메모리 표현이 포함되므로, 객체를 지역 메모리나 다른 객체 안에 둘 수 있고, 포인터·참조 없이 직접 접근하고, 완전히 초기화하며, 복사·이동할 수 있다.	(4.2, p.93-94 / PDF p.91-92)
U04-010	표현이 private이더라도 각 객체 안에 실제로 존재하므로 표현을 바꾸면 다시 컴파일해야 한다.	(4.2, p.93-94 / PDF p.91-92)
U04-011	이는 명확성과 효율성을 얻는 대가다.	(4.2, p.93-94 / PDF p.91-92)
U04-012	중요한 데이터를 자유 저장 영역에 두고 객체에는 접근용 포인터만 두면 유연성을 높일 수 있다.	(4.2, p.93-94 / PDF p.91-92)
U04-013	원문은 vector·string을 잘 설계된 인터페이스를 가진 자원 핸들로 설명한다.	(4.2, p.93-94 / PDF p.91-92)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-014	원문의 complex는 실수부 re와 허수부 im이라는 두 double을 표현으로 사용한다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-015	두 스칼라, 한 스칼라, 인자 없음의 세 생성자가 각각 일반 복소수·실수에서의 복소수·{0,0} 기본값을 만든다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-016	인자 없이 호출할 수 있는 생성자를 기본 생성자라고 한다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-017	이를 두면 초기화되지 않는 경우를 막을 수 있다.	(4.2.1, p.94-97 / PDF p.92-95)

번호	문장·표 항목·원문 코드·수식	원문 위치

U04-018	구체 산술 타입 complex (원문 코드 발췌)
	class complex {
	double re, im; // 메모리 표현: double 두 개
	public:
	complex(double r, double i) :re{r}, im{i} {} // 두 스칼라를 이용해 complex 생성
	complex(double r) :re{r}, im{0} {} // 스칼라 하나를 이용해 complex 생성
	complex() :re{0}, im{0} {} // 기본 complex: {0,0}
	
	double real() const { return re; }
	void real(double d) { re=d; }
	double imag() const { return im; }
	void imag(double d) { im=d; }
	
	complex& operator+=(complex z)
	{
	re+=z.re; // re와 im에 더하기
	im+=z.im;
	return *this; // 결과 반환
	}
	complex& operator-=(complex z)
	{
	re-=z.re;
	im-=z.im;
	return *this;
	}
	
	complex& operator*=(complex); // 클래스 밖의 어딘가에서 정의
	complex& operator/=(complex); // 클래스 밖의 어딘가에서 정의
	};
		(4.2.1, p.94-95 / PDF p.92-93)

U04-019	클래스 안에 정의한 함수는 기본적으로 인라인이며, 선언 앞의 inline으로 그 의도를 명시할 수도 있다.	(4.2.1, p.94-97 / PDF p.92-95)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-020	원문은 단순한 연산이 함수 호출 없이 구현될 수 있도록 하여 효율성을 얻는다고 설명한다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-021	const 멤버 함수는 호출한 객체를 변경하지 않는다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-022	const·비const 객체 모두에서 호출할 수 있지만, 비const 멤버 함수는 비const 객체에서만 호출한다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-023	표현에 직접 접근할 필요가 없는 산술 연산은 클래스 밖에 정의할 수 있다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-024	원문의 비멤버 +·-·*·/는 값으로 받은 인자 복사본을 변경해 반환하므로 호출자의 인자를 바꾸지 않는다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-025	상등 비교는 실수부·허수부를 모두 비교하고 !=는 !(a==b)로 정의한다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-026	연산자는 관례대로 신중하게 사용한다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-027	언어가 정한 문법을 바꿀 수 없으므로 단항 /를 새로 만들거나 int의 +를 다른 뜻으로 재정의할 수 없다.	(4.2.1, p.94-97 / PDF p.92-95)
U04-028	컨테이너는 요소의 컬렉션을 저장하는 객체다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-029	앞의 Vector도 컨테이너지만 new로 할당한 요소를 해제하지 않았다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-030	생성자에서 얻은 자원을 해제하는 소멸자가 이를 보완한다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-031	소멸자의 이름은 클래스 이름 앞에 ~를 붙인다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-032	delete는 개별 객체를, delete[]는 배열을 해제한다.	(4.2.2, p.97-99 / PDF p.95-97)

번호	문장·표 항목·원문 코드·수식	원문 위치

U04-033	생성자·소멸자로 배열을 관리하는 Vector (원문 코드 발췌)
	class Vector {
	public:
	Vector(int s) :elem{new double[s]}, sz{s} // 생성자: 자원 획득
	{
	for (int i=0; i!=s; ++i) // 요소 초기화
	elem[i]=0;
	}
	
	~Vector() { delete[] elem; } // 소멸자 : 자원 해제
	
	double& operator[](int i);
	int size() const;
	private:
	double* elem; // elem은 double요소 sz개를 가리킴
	int sz;
	};
		(4.2.2, p.98 / PDF p.96)

U04-034	Vector 사용자는 해제 코드를 직접 쓰지 않아도 된다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-035	지역 Vector는 내장 타입처럼 명명·스코프·할당·생애 주기 규칙에 따르고 스코프를 벗어나면 소멸된다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-036	원문은 예제를 단순화하려고 이 위치에서 에러 처리 코드는 생략했다고 밝힌다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-037	따라서 앞 절의 검사가 이 코드에 그대로 들어 있다고 가정하지 않는다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-038	생성자에서 자원을 획득하고 소멸자에서 해제하는 기법은 RAII, 즉 자원 획득이 곧 초기화다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-039	객체가 실제 데이터를 소유하는 핸들이 되어 생성·소멸과 자원 수명을 결합한다.	(4.2.2, p.97-99 / PDF p.95-97)
U04-040	원문은 애플리케이션에 드러난 벌거벗은 new·delete를 피하면 에러와 메모리 누수에 덜 취약한 코드를 만들 수 있다고 설명한다.	(4.2.2, p.97-99 / PDF p.95-97)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-041	컨테이너는 요소의 저장을 위해 존재하므로 편리한 입력 방법이 필요하다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-042	초깃값 목록 생성자는 요소 목록으로 바로 초기화하며, push_back()은 끝에 하나를 추가하고 크기를 1 늘린다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-043	요소 개수를 미리 모를 때 push_back()이 유용하다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-044	원문의 read()는 입력이 파일 끝이나 형식 에러로 끝날 때까지 읽은 double을 계속 추가한다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-045	for를 쓴 이유는 임시 d의 스코프를 루프 안으로 제한하기 위해서다.	(4.2.3, p.100-102 / PDF p.98-100)

U04-046	초깃값 목록 생성자 (원문 코드 발췌)
	Vector::Vector(std::initializer_list<double> lst) // 초깃값 목록 초기화
	:elem{new double[lst.size()]}, sz{static_cast<int>(lst.size())}
	{
	copy(lst.begin(),lst.end(),elem); // lst를 elem에 복사(12.6절)
	}
		(4.2.3, p.101 / PDF p.99)

U04-047	std::initializer_list는 {} 목록을 받아 초깃값 목록 생성자에 전달할 때 사용하는 표준 라이브러리 타입이다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-048	생성자는 목록 크기만큼 배열을 할당한 후 copy()로 요소를 옮긴다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-049	표준 라이브러리의 크기 타입은 unsigned 정수여서 이 예제는 static_cast<int>로 변환한다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-050	static_cast는 실제 값을 검사하지 않으므로 올바르게 쓰는 책임은 프로그래머에게 있다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-051	원문은 명시적 타입 변환을 가능하면 피하고, 검사 없는 캐스트는 시스템의 낮은 수준에 제한하라고 권한다.	(4.2.3, p.100-102 / PDF p.98-100)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-052	객체를 바이트 시퀀스로 취급하는 reinterpret_cast와 const를 제거하는 const_cast도 언급한다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-053	높은 수준에서는 타입 시스템과 잘 설계된 라이브러리로 검사 없는 변환을 피한다.	(4.2.3, p.100-102 / PDF p.98-100)
U04-054	구체 타입이 정의 안에 표현을 포함하는 것과 달리, 추상 타입은 사용자에게 상세 구현을 완전히 감춘다.	(4.3, p.102-106 / PDF p.100-104)
U04-055	인터페이스와 메모리 표현을 분리하고 포인터·참조로 접근한다.	(4.3, p.102-106 / PDF p.100-104)
U04-056	순수 가상 함수를 가진 추상 클래스 자체의 객체는 만들 수 없다.	(4.3, p.102-106 / PDF p.100-104)

U04-057	순수 가상 인터페이스 Container (원문 코드 발췌)
	class Container {
	public:
	virtual double& operator[](int) = 0; // 순수 가상 함수
	virtual int size() const = 0; // const 멤버 함수(4.2.1절)
	virtual ~Container() {} // 소멸자(4.2.2절)
	};
		(4.3, p.102-103 / PDF p.100-101)

U04-058	구체 구현을 모르는 사용자 함수 (원문 코드 발췌)
	void use(Container& c)
	{
	const int sz = c.size();
	for (int i=0; i!=sz; ++i)
	cout << c[i] << '\n';
	}
		(4.3, p.103 / PDF p.101)

U04-059	virtual은 파생 클래스에서 재정의할 수 있는 함수를 표시한다. =0은 순수 가상 함수로서 실제 구현 클래스가 그 연산을 구현해야 함을 나타낸다.	(4.3, p.102-106 / PDF p.100-104)
U04-060	Container는 operator[]와 size()라는 인터페이스만 제공하며, use(Container&)는 구체 구현을 모른 채 그 인터페이스를 호출한다.	(4.3, p.102-106 / PDF p.100-104)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-061	여러 클래스에 대한 인터페이스를 제공하는 타입을 원문은 다형 타입이라고 부른다.	(4.3, p.102-106 / PDF p.100-104)

U04-062	Vector를 이용한 추상 인터페이스 구현 (원문 코드 발췌)
	class Vector_container :public Container { // Vector_container가 Container를 구현
	public:
	Vector_container(int s) : v(s) { } // 요소들의 벡터
	~Vector_container() {}
	
	double& operator[](int i) override { return v[i]; }
	int size() const override { return v.size(); }
	private:
	Vector v;
	};
		(4.3, p.104 / PDF p.102)

U04-063	:public Container는 Container로부터 파생된 서브타입임을 뜻한다.	(4.3, p.102-106 / PDF p.100-104)
U04-064	Container는 기반·슈퍼클래스, Vector_container는 파생·서브클래스다.	(4.3, p.102-106 / PDF p.100-104)
U04-065	파생 클래스는 기반 클래스 멤버를 상속한다.	(4.3, p.102-106 / PDF p.100-104)
U04-066	override는 어떤 가상 함수를 재정의한다는 의도를 명시해 이름이나 타입을 잘못 쓴 실수를 컴파일러가 찾게 한다.	(4.3, p.102-106 / PDF p.100-104)
U04-067	Container에는 초기화할 데이터가 없어 생성자를 따로 두지 않지만 가상 소멸자가 있다.	(4.3, p.102-106 / PDF p.100-104)
U04-068	인터페이스 포인터로 소멸할 때 구체 구현의 자원을 알 수 없으므로 올바른 파생 소멸자를 부르기 위해서다.	(4.3, p.102-106 / PDF p.100-104)
U04-069	Vector_container 소멸 시 그 멤버 Vector의 소멸자도 암묵 호출된다.	(4.3, p.102-106 / PDF p.100-104)
U04-070	원문은 list<double>을 표현으로 쓰는 List_container도 구현해 같은 use()를 적용한다.	(4.3, p.102-106 / PDF p.100-104)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-071	list의 인덱싱은 vector보다 크게 느려 일반적인 인덱스 컨테이너 구현에는 쓰지 않지만 대안 구현을 보이는 예다.	(4.3, p.102-106 / PDF p.100-104)
U04-072	구현 변경이나 새 파생 클래스 추가에도 use(Container&)를 다시 컴파일할 필요가 없다는 유연성을 얻으며, 그 대가로 객체에는 포인터·참조로 접근한다.	(4.3, p.102-106 / PDF p.100-104)
U04-073	c[i]가 어느 구현을 호출할지는 실행 시간의 실제 객체에 달려 있다.	(4.4, p.107-108 / PDF p.105-106)
U04-074	따라서 객체에는 적절한 함수를 찾을 정보가 필요하다.	(4.4, p.107-108 / PDF p.105-106)
U04-075	원문은 일반적인 구현 기법으로 가상 함수 테이블(vtbl)을 설명한다.	(4.4, p.107-108 / PDF p.105-106)
U04-076	컴파일러는 가상 함수 이름을 함수 포인터 테이블의 인덱스로 변환하고, 가상 함수가 있는 각 클래스는 자기 함수들의 테이블을 갖는다.	(4.4, p.107-108 / PDF p.105-106)
U04-077	호출자는 구체 객체의 크기나 데이터 모양을 몰라도 테이블 포인터 위치와 함수 인덱스로 접근할 수 있다.	(4.4, p.107-108 / PDF p.105-106)
U04-078	원문은 가상 호출을 일반 호출에 가까운 효율로 구현할 수 있다고 설명하고, 추가 저장 공간은 가상 함수가 있는 객체당 포인터 하나와 클래스당 테이블 하나라고 제시한다.	(4.4, p.107-108 / PDF p.105-106)
U04-079	계층 구조는 파생 관계로 클래스 집합을 조직하며 “원은 도형의 일종” 같은 관계를 표현한다.	(4.5, p.108-111 / PDF p.106-109)
U04-080	원문은 Shape 아래 Circle·Triangle, Circle 아래 Smiley를 둔다.	(4.5, p.108-111 / PDF p.106-109)
U04-081	Shape는 중심·이동·그리기·회전과 가상 소멸자의 추상 인터페이스를 제공한다.	(4.5, p.108-111 / PDF p.106-109)
U04-082	rotate_all()은 실제 도형 종류를 몰라도 Shape*를 통해 같은 rotate() 호출을 적용한다.	(4.5, p.108-111 / PDF p.106-109)
U04-083	Circle은 중심 Point와 반지름으로 구체 속성을 구현하고, Smiley는 Circle을 기반으로 눈·입을 추가한다.	(4.5, p.108-111 / PDF p.106-109)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-084	Smiley::draw()는 Circle::draw()로 얼굴을 그리고 eyes와 mouth의 draw()를 호출한다.	(4.5, p.108-111 / PDF p.106-109)
U04-085	새로운 파생 클래스는 데이터·연산을 추가할 수 있지만 유연성만큼 잘못된 설계의 가능성도 있다.	(4.5, p.108-111 / PDF p.106-109)
U04-086	추상 기반 클래스 포인터로 파생 객체를 소멸시키려면 기반 소멸자는 가상이어야 한다.	(4.5, p.108-111 / PDF p.106-109)
U04-087	가상 호출이 올바른 파생 소멸자를 선택하고, 그 소멸자는 멤버·기반 소멸자를 암묵 호출한다.	(4.5, p.108-111 / PDF p.106-109)
U04-088	원문의 Smiley 소멸자는 소유한 눈·입을 해제한 후 기반 소멸로 이어진다.	(4.5, p.108-111 / PDF p.106-109)
U04-089	인터페이스 상속은 기반 객체를 사용하는 곳에서 파생 객체를 대신 쓸 수 있게 한다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-090	Container·Shape처럼 기반이 파생의 인터페이스가 되며 종종 추상 클래스다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-091	구현 상속은 기반의 데이터·함수로 파생 구현을 단순화한다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-092	Smiley가 Circle 생성자와 draw()를 이용하는 것이 예다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-093	원문의 read_shape()는 입력으로 도형 종류를 읽고 알맞은 Circle·Triangle·Smiley를 생성해 Shape*로 돌려준다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-094	user()는 vector<Shape*>에 넣어 종류를 모른 채 그리기·회전을 수행한다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-095	이 사용자 코드는 새 도형이 추가되어도 한 번 컴파일해 사용할 수 있다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-096	예제에서는 user()만 포인터를 보관하므로 모든 도형을 해제할 책임도 user()에 있다.	(4.5.1, p.112-114 / PDF p.110-112)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-097	delete와 Shape의 가상 소멸자가 최하위 파생 클래스부터 소멸자를 호출해 자원을 반납한다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-098	생성은 기반에서 파생으로, 소멸은 파생에서 기반으로 진행된다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-099	원문은 해당 예제가 특히 에러 처리 면에서 단순화되어 있음을 명시한다.	(4.5.1, p.112-114 / PDF p.110-112)
U04-100	공통 Shape 인터페이스 밖에 있는 Smiley::wink() 같은 기능을 사용하려면 실제 타입을 확인할 수 있다.	(4.5.2, p.114-115 / PDF p.112-113)
U04-101	포인터에 대한 dynamic_cast는 기대 타입 또는 그 파생 타입을 가리키지 않으면 nullptr를 반환하므로 다른 타입이 합법적인 대안일 때 사용한다.	(4.5.2, p.114-115 / PDF p.112-113)

U04-102	포인터 dynamic_cast와 대안 처리 (원문 코드 발췌)
	Shape* ps {read_shape(cin)};
	
	if (Smiley* p = dynamic_cast<Smiley*>(ps)) { // ... ps 가 Smiley를 가리키는가? ...
	// Smiley가 맞음. 적절히 사용.
	}
	else {
	// ... Smiley가 아님. 다른 작업 시도 ...
	}
		(4.5.2, p.114 / PDF p.112)

U04-103	기대한 타입이 아닌 경우를 실패로 취급해야 하면 참조에 dynamic_cast를 사용할 수 있다.	(4.5.2, p.114-115 / PDF p.112-113)
U04-104	이 경우 잘못된 타입이면 bad_cast 예외를 던진다.	(4.5.2, p.114-115 / PDF p.112-113)
U04-105	원문은 타입 검사를 직접 쓰지 않아도 되는 설계가 더 간단·효율적이라고 권하면서, 기반 인터페이스만 받는 시스템에 객체를 보낸 뒤 돌려받을 때처럼 숨겨진 원래 타입을 복원해야 할 수도 있다고 설명한다.	(4.5.2, p.114-115 / PDF p.112-113)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-106	원문의 벌거벗은 포인터 예제에는 Smiley가 mouth 삭제를 잊거나, read_shape() 사용자가 반환 포인터 삭제를 잊거나, 포인터 컨테이너 소유자가 각 객체 삭제를 잊을 위험이 있다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-107	new 다음에 예외를 던지거나 조기 return하면 마지막 delete까지 도달하지 못해 누수가 생긴다.	(4.5.3, p.115-117 / PDF p.113-115)

U04-108	unique_ptr로 소유권을 표현한 Smiley (원문 코드 발췌)
	class Smiley : public Circle {
	// ...
	private:
	vector<unique_ptr<Shape>> eyes; // 눈 두 개
	unique_ptr<Shape> mouth;
	};
		(4.5.3, p.116 / PDF p.114)

U04-109	삭제 책임이 있다면 표준 unique_ptr로 소유권을 표현한다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-110	Smiley의 eyes를 vector<unique_ptr<Shape>>, mouth를 unique_ptr<Shape>로 바꾸면 멤버들의 소멸이 자원을 반납해 별도 Smiley 소멸자를 정의할 필요가 없다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-111	원문은 이 방법의 효율이 올바른 벌거벗은 포인터 코드와 동등하다고 설명한다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-112	read_shape()도 unique_ptr<Shape>를 반환하고 user()는 그 포인터들을 vector에 보관한다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-113	unique_ptr가 스코프를 벗어날 때 소유 객체를 삭제한다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-114	이 버전을 사용하려면 draw_all()·rotate_all()도 vector<unique_ptr<Shape>>&를 받도록 바꿔야 한다는 원문의 조건을 함께 유지한다.	(4.5.3, p.115-117 / PDF p.113-115)
U04-115	아이디어를 코드로 직접 표현하고, 간단한 개념에는 구체 클래스를 사용한다.	(4.6, p.117-118 / PDF p.115-116)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-116	성능이 중요한 경우에도 구체 타입을 고려한다.	(4.6, p.117-118 / PDF p.115-116)
U04-117	생성자가 초기화를 맡고 상태를 바꾸지 않는 멤버 함수는 const로 선언한다.	(4.6, p.117-118 / PDF p.115-116)
U04-118	표현에 직접 접근해야 하는 함수는 멤버로, 대칭적 연산은 비멤버로 만든다.	(4.6, p.117-118 / PDF p.115-116)
U04-119	자원 획득에는 해제가 짝을 이루게 하고 RAII 자원 핸들을 쓴다.	(4.6, p.117-118 / PDF p.115-116)
U04-120	컨테이너에는 초깃값 목록 생성자를 제공한다.	(4.6, p.117-118 / PDF p.115-116)
U04-121	표현과 인터페이스를 완전히 분리하려면 추상 클래스를 사용하고 다형 객체는 포인터·참조로 접근한다.	(4.6, p.117-118 / PDF p.115-116)
U04-122	계층 관계를 표현할 때 인터페이스 상속과 구현 상속을 구별한다.	(4.6, p.117-118 / PDF p.115-116)
U04-123	가상 함수가 있는 클래스에는 가상 소멸자를 두고 override로 의도를 드러낸다.	(4.6, p.117-118 / PDF p.115-116)
U04-124	불가피한 계층 탐색에는 dynamic_cast를 사용한다.	(4.6, p.117-118 / PDF p.115-116)
U04-125	찾지 못한 타입이 대안이면 포인터, 실패이면 참조를 사용한다.	(4.6, p.117-118 / PDF p.115-116)
U04-126	소유권은 unique_ptr·shared_ptr로 표현해 해제 누락을 피한다.	(4.6, p.117-118 / PDF p.115-116)
U04-127	표현	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-128	타입 정의에 존재	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-129	사용자에게 구현을 완전히 감춤	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-130	사용	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-131	직접 객체·지역 변수·복사·이동	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-132	인터페이스의 포인터·참조	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-133	변경 영향	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-134	표현 변경 시 다시 컴파일	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-135	인터페이스가 유지되면 사용자 재컴파일 없이 구현 변경	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-136	예	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-137	complex, Vector	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-138	Container, Shape	(구체 타입과 추상 타입, p.93-106 / PDF p.91-104)
U04-139	인터페이스 상속	(상속의 두 역할, p.112 / PDF p.110)
U04-140	기반 타입을 쓰는 곳에 파생 객체를 사용	(상속의 두 역할, p.112 / PDF p.110)
U04-141	Container, Shape	(상속의 두 역할, p.112 / PDF p.110)
U04-142	구현 상속	(상속의 두 역할, p.112 / PDF p.110)
U04-143	기반의 데이터·함수로 파생 구현 단순화	(상속의 두 역할, p.112 / PDF p.110)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-144	Smiley의 Circle 생성자·draw 활용	(상속의 두 역할, p.112 / PDF p.110)
U04-145	포인터	(dynamic_cast의 두 실패 표현, p.114-115 / PDF p.112-113)
U04-146	nullptr 반환	(dynamic_cast의 두 실패 표현, p.114-115 / PDF p.112-113)
U04-147	다른 타입이 합법적인 대안	(dynamic_cast의 두 실패 표현, p.114-115 / PDF p.112-113)
U04-148	참조	(dynamic_cast의 두 실패 표현, p.114-115 / PDF p.112-113)
U04-149	bad_cast 예외	(dynamic_cast의 두 실패 표현, p.114-115 / PDF p.112-113)
U04-150	기대 타입이 아니면 실패	(dynamic_cast의 두 실패 표현, p.114-115 / PDF p.112-113)
U04-151	컨테이너는 요소의 컬렉션을 저장하는 객체다.	(4.2.2, p.97 / PDF p.95)
U04-152	소멸자는 클래스 이름 앞에 ~를 붙인 이름을 가지며, Vector의 소멸자는 할당한 배열을 delete[]로 해제한다.	(4.2.2, p.98 / PDF p.96)
U04-153	추상 타입은 사용자에게 상세 구현을 완전히 감추며, 인터페이스와 메모리 표현을 분리한다.	(4.3, p.102 / PDF p.100)
U04-154	가상 함수 테이블(vtbl)은 일반적인 구현 기법에서 가상 함수 이름을 인덱스로 바꿔 접근하는 함수 포인터 테이블이다.	(4.4, p.107 / PDF p.105)
U04-155	인터페이스 상속은 기반 클래스 객체를 사용하는 곳에서 파생 클래스 객체를 대신 사용할 수 있게 한다.	(4.5.1, p.112 / PDF p.110)

번호	문장·표 항목·원문 코드·수식	원문 위치
U04-156	구체 클래스로 간단한 개념을 표현하고, 객체 초기화는 생성자가 맡는다.	(4.6, p.117 / PDF p.115)
U04-157	생성자에서 자원을 획득하는 클래스라면 그 자원을 해제하는 소멸자가 필요하다.	(4.6, p.118 / PDF p.116)
U04-158	인터페이스와 구현을 완전히 분리하려면 추상 클래스를 사용하고, 다형 객체에는 포인터와 참조로 접근한다.	(4.6, p.118 / PDF p.116)
U04-159	클래스 계층 구조를 설계할 때 구현 상속과 인터페이스 상속을 구별한다.	(4.6, p.118 / PDF p.116)
U04-160	구체 타입은 타입 정의 안에 메모리 표현을 포함하고, 추상 타입은 상세 구현을 감추며 인터페이스와 메모리 표현을 분리한다.	(4.2, p.93,102 / PDF p.91,100)
U04-161	인터페이스 상속은 기반 객체 대신 파생 객체를 사용하게 하고, 구현 상속은 기반의 함수·데이터 멤버로 파생 구현을 단순하게 한다.	(4.5.1, p.112 / PDF p.110)
U04-162	인터페이스 상속	(4.5.1, p.112 / PDF p.110)
U04-163	구현 상속	(4.5.1, p.112 / PDF p.110)

제5장. 필수적인 연산

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-001	생성자와 소멸자, 복사와 이동은 논리적으로 서로 구분되지 않는 관련 연산이다.	(5.1.1, p.120 / PDF p.118)
U05-002	move()는 아무것도 이동시키지 않고 앞으로 이동시킬 인자의 rvalue 참조를 반환하는 일종의 타입 변환이다.	(5.2.2, p.130 / PDF p.128)
U05-003	초기화·대입·복사·이동은 프로그램의 기초 동작이다.	(5.1, p.119 / PDF p.117)
U05-004	원문은 이러한 연산에 관례적인 의미가 있으며 이를 임의로 무시해서는 안 된다고 설명한다.	(5.1, p.119 / PDF p.117)
U05-005	타입을 정의할 때는 객체 생성에서 사용·전달·정리까지의 동작이 일관되게 연결되도록 설계한다.	(5.1, p.119 / PDF p.117)
U05-006	생성자·소멸자·복사·이동은 서로 관련된 연산이다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-007	소멸자가 동적 저장 공간이나 락을 해제하는 등 특별한 일을 한다면 복사·이동에도 별도 정의가 필요할 가능성이 크다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-008	원문은 일반 생성자, 기본 생성자, 복사·이동 생성자, 복사·이동 대입, 소멸자를 한 집합으로 제시한다.	(5.1.1, p.120-122 / PDF p.118-120)

U05-009	함께 설계할 생애 주기 연산 (원문 코드 발췌)
	class X {
	public:
	X(Sometype); // "일반적인 생성자": 객체 생성
	X(); // 기본 생성자
	X(const X&); // 복사 생성자
	X(X&&); // 이동 생성자
	X& operator=(const X&); // 복사 대입: 대상을 정리하고 복사
	X& operator=(X&&); // 이동 대입: 대상을 정리하고 이동
	~X(); // 소멸자: 정리 작업
	// ...
	};
		(5.1.1, p.120 / PDF p.118)

U05-010	객체는 대입 원본, 초깃값, 함수 인자, 함수 반환 값, 예외로서 복사·이동될 수 있다.	(5.1.1, p.120-122 / PDF p.118-120)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-011	대입에는 대입 연산자를 사용하고 다른 경우에는 생성자가 관련되지만, 대상의 저장 공간에 직접 초기화해 생성자 호출을 생략할 수 있다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-012	이 복사 생략은 객체 전달의 최적화다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-013	컴파일러는 일반 생성자를 제외한 특수 멤버를 필요할 때 생성하며 =default로 기본 구현을 명시할 수 있다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-014	원문은 일부 기본 함수를 명시하면 나머지 기본 함수가 생성되지 않는다고 설명하므로, 연산 집합 전체를 함께 고려한다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-015	포인터가 delete할 자원을 가리킨다면 단순한 멤버별 복사는 적합하지 않다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-016	반대로 비소유 포인터라면 그 사실도 독자가 알아야 한다.	(5.1.1, p.120-122 / PDF p.118-120)

U05-017	기반 클래스 복사의 사용 금지 (원문 코드 발췌)
	class Shape {
	public:
	Shape(const Shape&) =delete; // 복사 연산 없음
	Shape& operator=(const Shape&) =delete;
	// ...
	};
	
	void copy(Shape& s1, const Shape& s2)
	{
	s1 = s2; // 에러: Shape의 복사 연산이 제거됨
	}
		(5.1.1, p.122 / PDF p.120)

U05-018	기본 의미가 충분한 클래스는 컴파일러가 멤버의 기본 생성·복사·이동·소멸을 조합하도록 두고, 특별한 의미가 필요하다면 필수 연산을 함께 정의한다. =delete는 어떤 연산의 사용을 금지하며 사용하면 컴파일 시간 에러가 난다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-019	원문은 멤버별 복사가 적합하지 않은 Shape 기반 클래스를 예로 든다.	(5.1.1, p.120-122 / PDF p.118-120)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-020	특수 멤버 외의 함수에도 =delete를 사용할 수 있다.	(5.1.1, p.120-122 / PDF p.118-120)
U05-021	인자 하나를 받는 생성자는 그 인자 타입에서 클래스 타입으로의 변환을 정의한다.	(5.1.2, p.122-123 / PDF p.120-121)
U05-022	complex에 double을 주어 {값,0}을 만드는 암묵 변환은 유용할 수 있다.	(5.1.2, p.122-123 / PDF p.120-121)
U05-023	반면 int 7을 Vector로 암묵 변환해 요소 7개짜리 객체를 만드는 것은 원하지 않는 결과일 수 있다.	(5.1.2, p.122-123 / PDF p.120-121)

U05-024	explicit 생성자로 암묵 변환 제한 (원문 코드 발췌)
	class Vector {
	public:
	explicit Vector(int s); // int에서 Vector로 암묵적인 변환 불가
	// ...
	};
	
	Vector v1(7); // OK: v1은 7개 요소를 포함
	Vector v2 = 7; // 에러: int에서 Vector로 암묵적인 변환 불가
		(5.1.2, p.123 / PDF p.121)

U05-025	explicit은 명시적 변환만 허용하게 한다.	(5.1.2, p.122-123 / PDF p.120-121)
U05-026	이때 Vector v1(7)은 가능하지만 Vector v2 = 7은 금지된다.	(5.1.2, p.122-123 / PDF p.120-121)
U05-027	원문은 특별한 이유가 없다면 한 인자를 받는 생성자를 explicit으로 선언해 Vector 쪽의 관례를 따르라고 권한다.	(5.1.2, p.122-123 / PDF p.120-121)
U05-028	데이터 멤버 선언에서 기본 멤버 초깃값을 지정할 수 있다.	(5.1.3, p.124 / PDF p.122)
U05-029	원문의 complex는 re와 im을 각각 0으로 지정하고, 두 인자 생성자는 둘 다 대체하며 한 인자 생성자는 re만 지정한다.	(5.1.3, p.124 / PDF p.122)
U05-030	생성자가 별도 값을 주지 않은 멤버에는 기본값을 사용한다.	(5.1.3, p.124 / PDF p.122)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-031	이를 통해 생성자 코드를 줄이고 실수로 멤버를 초기화하지 않는 일을 막는다.	(5.1.3, p.124 / PDF p.122)
U05-032	원문은 사용자 정의 타입과 내장 타입의 기본 복사 의미를 멤버별 복사로 설명한다.	(5.2, p.124-125 / PDF p.122-123)
U05-033	두 double로 표현되는 complex에서는 초기화와 대입 시 두 성분을 복사하면 독립된 같은 값이 되어 적합하다.	(5.2, p.124-125 / PDF p.122-123)
U05-034	그러나 자원 핸들이거나 추상 타입이면 이런 의미가 적절하지 않은 경우가 많다.	(5.2, p.124-125 / PDF p.122-123)
U05-035	클래스 설계 시 복사가 무엇을 뜻하는지 반드시 고려한다.	(5.2, p.124-125 / PDF p.122-123)
U05-036	Vector의 elem 포인터와 sz만 복사하면 두 Vector가 같은 요소를 가리키게 된다.	(5.2.1, p.125-127 / PDF p.123-125)
U05-037	한 벡터의 요소 변경이 다른 벡터에도 보이고, 자원을 각 객체가 책임진다는 의미와 충돌한다.	(5.2.1, p.125-127 / PDF p.123-125)
U05-038	원문의 bad_copy()는 이 문제를 도식으로 보여 준다.	(5.2.1, p.125-127 / PDF p.123-125)
U05-039	소멸자가 있는 자원 소유 클래스라면 기본 복사가 적절한지 특히 확인해야 한다.	(5.2.1, p.125-127 / PDF p.123-125)

U05-040	자기 저장 공간을 갖는 Vector 복사 생성자 (원문 코드 발췌)
	Vector::Vector(const Vector& a) // 복사 생성자
	:elem{new double[a.sz]}, // 요소를 저장할 공간 할당
	sz{a.sz}
	{
	for (int i=0; i!=sz; ++i) // 요소를 복사
	elem[i] = a.elem[i];
	}
		(5.2.1, p.126 / PDF p.124)

번호	문장·표 항목·원문 코드·수식	원문 위치

U05-041	기존 자원을 교체하는 복사 대입 (원문 코드 발췌)
	Vector& Vector::operator=(const Vector& a) // 복사 대입
	{
	double* p = new double[a.sz];
	for (int i=0; i!=a.sz; ++i)
	p[i] = a.elem[i];
	delete[] elem; // 예전 요소 삭제
	elem = p;
	sz = a.sz;
	return *this;
	}
		(5.2.1, p.127 / PDF p.125)

U05-042	복사 생성자는 원본 크기의 공간을 새로 할당하고 각 요소를 복사해, 두 Vector가 각각 자기 요소의 복사본을 갖게 한다.	(5.2.1, p.125-127 / PDF p.123-125)
U05-043	복사 대입은 대상에 이미 있던 자원을 정리해야 하므로 새 배열 할당·요소 복사 후 이전 배열을 해제하고 포인터·크기를 교체한다.	(5.2.1, p.125-127 / PDF p.123-125)
U05-044	this는 멤버 함수를 호출한 객체를 가리키며 *this는 결과로 대상 객체 자체를 돌려준다.	(5.2.1, p.125-127 / PDF p.123-125)
U05-045	큰 객체의 복사 비용은 클 수 있다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-046	참조 인자로 입력 복사는 피할 수 있지만 사라질 지역 객체를 참조로 반환할 수는 없다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-047	원문의 벡터 덧셈은 지역 res를 만들어 반환한다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-048	덧셈을 연속하면 더 이상 사용하지 않을 중간 결과를 복사하는 비용이 문제다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-049	원하는 것은 원소의 또 다른 복사본이 아니라 계산 결과를 함수 밖으로 가져오는 것이다.	(5.2.2, p.127-131 / PDF p.125-129)

번호	문장·표 항목·원문 코드·수식	원문 위치

U05-050	지역 결과를 값으로 반환하는 벡터 덧셈 (원문 코드 발췌)
	Vector operator+(const Vector& a, const Vector& b)
	{
	if (a.size()!=b.size())
	throw Vector_size_mismatch{};
	Vector res(a.size());
	
	for (int i=0; i!=a.size(); ++i)
	res[i]=a[i]+b[i];
	return res;
	}
		(5.2.2, p.127-128 / PDF p.125-126)

U05-051	배열 소유권을 가져오는 이동 생성자 (원문 코드 발췌)
	Vector::Vector(Vector&& a)
	:elem{a.elem}, // 인자로부터 요소를 가져옴
	sz{a.sz}
	{
	a.elem = nullptr; // 이제 요소를 포함하지 않음
	a.sz = 0;
	}
		(5.2.2, p.129 / PDF p.127)

U05-052	이동 생성자는 원본 elem과 sz를 가져오고 원본을 nullptr·0 상태로 만든다. &&는 rvalue 참조를 나타내며, 원문은 함수 반환 결과처럼 이동 가능한 값을 표현하는 데 사용한다고 설명한다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-053	이동은 원본에서 값을 제거하므로 이동 생성자는 const 인자를 받지 않는다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-054	이동 대입도 비슷한 목적을 갖는다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-055	이동 후 원본은 소멸자를 문제없이 호출할 수 있어야 하고, 일반적으로 다시 대입할 수도 있어야 한다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-056	프로그래머가 앞으로 값을 사용하지 않을 것을 알면 std::move로 그 의도를 나타낼 수 있다.	(5.2.2, p.127-131 / PDF p.125-129)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-057	move() 자체는 아무것도 이동시키지 않고 이동 가능하게 만드는 rvalue 참조를 반환하는 일종의 타입 변환이다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-058	원문은 이동 뒤의 값을 사용하지 않는 것이 좋다고 주석으로 명시한다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-059	객체 초기화에서는 복사 생략으로 실제 이동 생성 호출도 줄어들 수 있다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-060	반면 기존 객체에 대한 대입의 복사·이동은 암묵적으로 없애기 어려우므로 이동 대입은 성능 면에서 중요한 역할을 한다.	(5.2.2, p.127-131 / PDF p.125-129)
U05-061	생성자·복사·이동·소멸을 정의하면 객체가 소유하는 자원의 생애를 제어할 수 있다.	(5.3, p.131-133 / PDF p.129-131)
U05-062	이동은 복사할 수 없거나 복사하기 싫은 객체를 낮은 비용으로 다른 스코프로 옮긴다.	(5.3, p.131-133 / PDF p.129-131)
U05-063	원문의 예에서 thread는 복사할 수 없고, double 100만 개의 Vector는 복사를 피하고 싶은 대상이다.	(5.3, p.131-133 / PDF p.129-131)

번호	문장·표 항목·원문 코드·수식	원문 위치

U05-064	스레드와 큰 Vector의 자원 이전 (원문 코드 발췌)
	std::vector<thread> my_threads;
	
	Vector init(int n)
	{
	thread t {heartbeat}; // heartbeat를 동시적으로(각자 다른 스레드로) 실행
	my_threads.push_back(std::move(t)); // t를 my_threads로 이동(13.2.2절)
	// ... 다른 초기화 수행...
	
	Vector vec(n);
	for (int i=0; i!=vec.size(); ++i)
	vec[i] = 777;
	return vec; // vec를 init() 밖으로 이동
	}
	
	auto v = init(1'000'000); // heartbeat 을 시작하고 v를 초기화
		(5.3, p.131 / PDF p.129)

U05-065	자원 핸들에는 포인터 자체도 감춰지게 할 수 있으며 추가 부담 없이 단순하고 유지 보수하기 쉬운 코드를 얻는다.	(5.3, p.131-133 / PDF p.129-131)
U05-066	강력한 자원 안전성은 일반적인 의미의 자원 모두에 대해 누수를 막는 것이다.	(5.3, p.131-133 / PDF p.129-131)
U05-067	메모리뿐 아니라 락·소켓·파일·스레드 핸들도 자원이다.	(5.3, p.131-133 / PDF p.129-131)
U05-068	오래 실행되는 시스템에서는 누수뿐 아니라 필요 이상 오래 보유하는 것도 문제다.	(5.3, p.131-133 / PDF p.129-131)
U05-069	원문은 가비지 컬렉션을 전역적 메모리 관리로 설명하고, 더 명확하고 일반적이며 지역적인 관리 방법이 없을 때 최후 수단으로 고려한다는 저자의 입장을 제시한다.	(5.3, p.131-133 / PDF p.129-131)
U05-070	먼저 각 자원의 소유자가 특정 스코프에 존재하고 종료 시 해제하도록 RAII를 체계적으로 사용한다.	(5.3, p.131-133 / PDF p.129-131)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-071	이동·스마트 포인터는 스코프 사이 자원 이전을, 공유 포인터는 공유 소유권을 표현한다.	(5.3, p.131-133 / PDF p.129-131)
U05-072	표준 라이브러리는 string·vector·map의 메모리, ifstream·ofstream의 파일, thread의 스레드, lock_guard·unique_lock의 락, unique_ptr·shared_ptr의 객체에 RAII를 적용한다.	(5.3, p.131-133 / PDF p.129-131)
U05-073	그 결과 보통의 코드에서 관리가 두드러지지 않으면서 소유 시간을 줄이는 암묵적 자원 관리가 가능하다.	(5.3, p.131-133 / PDF p.129-131)
U05-074	많은 프로그래머와 표준 라이브러리는 특정 연산의 관례적인 뜻을 가정한다.	(5.4, p.133 / PDF p.131)
U05-075	새로운 타입에서도 비교, 크기·범위 접근, 입출력, 리터럴, swap, hash의 의미가 관례를 따르도록 설계한다.	(5.4, p.133 / PDF p.131)
U05-076	상등 비교 ==·!=는 복사 의미와 밀접하다.	(5.4.1, p.133-134 / PDF p.131-132)
U05-077	복사한 뒤 원본과 복사본은 같아야 한다. ==를 정의하면 !=도 정의하고 a!=b가 !(a==b)를 뜻하게 한다.	(5.4.1, p.133-134 / PDF p.131-132)
U05-078	순서 비교 연산도 서로 일관된 의미를 가져야 한다.	(5.4.1, p.133-134 / PDF p.131-132)
U05-079	원문은 두 피연산자를 동등하게 취급하도록 이항 연산자를 해당 클래스와 같은 네임스페이스의 독립 함수로 정의하는 방식을 권한다.	(5.4.1, p.133-134 / PDF p.131-132)
U05-080	[원문 확인 필요] 인쇄 134쪽 비교 규칙 목록은 첫 항목과 셋째 항목에 모두 !(b<a)를 인쇄한다.	(5.4.1, p.133-134 / PDF p.131-132)
U05-081	이 요약에서는 외부 지식으로 그 식을 수정하거나 새로운 식으로 대체하지 않고, 비교 연산 간의 일관성이라는 원문 설명을 보존했다.	(5.4.1, p.133-134 / PDF p.131-132)
U05-082	특별한 이유가 없다면 표준 라이브러리 컨테이너 설계를 따른다.	(5.4.2, p.134-135 / PDF p.132-133)
U05-083	컨테이너를 필수 연산을 갖춘 자원 핸들로 구현하고 size()로 요소 개수를 알게 한다.	(5.4.2, p.134-135 / PDF p.132-133)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-084	알고리즘은 인덱스 대신 반복자 쌍으로 정의된 시퀀스를 사용한다.	(5.4.2, p.134-135 / PDF p.132-133)
U05-085	begin()은 첫 요소, end()는 마지막 요소의 다음을 가리킨다.	(5.4.2, p.134-135 / PDF p.132-133)
U05-086	반복자는 포인터처럼 ++로 다음으로 이동하고 *로 값을 읽고 쓴다.	(5.4.2, p.134-135 / PDF p.132-133)
U05-087	sort(v.begin(),v.end())처럼 반복자 쌍을 표준 알고리즘에 전달한다.	(5.4.2, p.134-135 / PDF p.132-133)
U05-088	구간 for도 같은 begin()·end() 범위를 사용하며 명시적 반복자 루프와 거의 같은 방식이다.	(5.4.2, p.134-135 / PDF p.132-133)
U05-089	정수 쌍에 대한 <<·>>는 각각 왼쪽·오른쪽 시프트다.	(5.4.3, p.135-136 / PDF p.133-134)
U05-090	iostream에 대해서는 출력·입력 연산으로 사용한다.	(5.4.3, p.135-136 / PDF p.133-134)
U05-091	원문은 자세한 입출력 설명을 10장으로 연결하며, 이 절에서는 같은 기호의 관례적인 사용 맥락을 구별한다.	(5.4.3, p.135-136 / PDF p.133-134)
U05-092	사용자 정의 타입에도 내장 타입처럼 편리한 리터럴을 제공할 수 있다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-093	적절한 접미사의 의미를 리터럴 연산자로 정의하면 접미사 앞 리터럴을 필요한 타입으로 변환한다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-094	원문은 문자열 s, 시간 단위 s, 허수 i의 예를 들고, 사용하려면 적절한 표준 헤더와 네임스페이스를 가져와야 한다고 설명한다.	(5.4.4, p.136-137 / PDF p.134-135)

번호	문장·표 항목·원문 코드·수식	원문 위치

U05-095	허수 접미사를 정의하는 리터럴 연산자 (원문 코드 발췌)
	constexpr complex<double> operator""i(long double arg) // 허수
	{
	 return {0,arg};
	}
	
	complex<double> z = 2.7182818+6.283185i;
		(5.4.4, p.137 / PDF p.135)

U05-096	operator""는 리터럴 연산자를, 그 뒤 i는 접미사를 나타낸다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-097	long double 인자는 이 접미사가 부동소수점 리터럴에 적용됨을, complex<double> 반환 타입은 만들어질 리터럴의 타입을 나타낸다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-098	원문의 chrono 접미사는 h·min·s·ms·us·ns이고 complex에는 i·il·if가 열거된다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-099	원문 표의 string_view 행은 string_literals 및 s로 인쇄되어 있다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-100	[원문 확인 필요] 이를 임의의 다른 이름으로 고치지 않았다.	(5.4.4, p.136-137 / PDF p.134-135)
U05-101	sort()를 비롯한 많은 알고리즘은 두 객체 값을 교환하는 swap()을 사용하며 빠르고 예외를 던지지 않는다고 가정한다.	(5.4.5, p.137-138 / PDF p.135-136)
U05-102	std::swap(a,b)는 세 번의 이동으로 정의된다.	(5.4.5, p.137-138 / PDF p.135-136)
U05-103	복사 비용이 큰 타입을 정렬·교환해야 한다면 이동 연산 또는 swap(), 혹은 둘 다 제공하는 것이 좋다.	(5.4.5, p.137-138 / PDF p.135-136)
U05-104	표준 컨테이너와 string은 빠른 이동을 제공한다.	(5.4.5, p.137-138 / PDF p.135-136)
U05-105	unordered_map<K,V>는 키 타입 K와 값 타입 V를 갖는 해시 테이블이다.	(5.4.6, p.138 / PDF p.136)
U05-106	사용자 타입 X를 키로 사용하려면 hash<X>를 정의해야 한다.	(5.4.6, p.138 / PDF p.136)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-107	표준 라이브러리는 std::string을 포함한 일반적인 타입의 해시를 제공한다.	(5.4.6, p.138 / PDF p.136)
U05-108	이 절은 해시 구현을 새로 전개하지 않고 11.5절로 연결한다.	(5.4.6, p.138 / PDF p.136)
U05-109	생성자·대입·소멸자를 관련 집합으로 설계하고 필수 연산은 모두 정의하거나 모두 기본에 맡긴다.	(5.5, p.138-139 / PDF p.136-137)
U05-110	기본 구현이 충분하면 직접 다시 쓰지 않는다.	(5.5, p.138-139 / PDF p.136-137)
U05-111	포인터를 가진 클래스나 소멸자가 있는 클래스는 사용자 정의 또는 제거된 복사·이동이 필요할 수 있다.	(5.5, p.138-139 / PDF p.136-137)
U05-112	부적절한 기본 복사는 재정의하거나 막는다.	(5.5, p.138-139 / PDF p.136-137)
U05-113	한 인자 생성자는 기본적으로 explicit으로 두고 합리적인 기본 멤버 값은 선언에 지정한다.	(5.5, p.138-139 / PDF p.136-137)
U05-114	컨테이너는 값을 반환하되 이동으로 효율을 확보하고 큰 피연산자는 const 참조로 받는다.	(5.5, p.138-139 / PDF p.136-137)
U05-115	어떤 자원도 누수되지 않도록 강력한 자원 안전성을 제공한다.	(5.5, p.138-139 / PDF p.136-137)
U05-116	연산자는 관례에 맞게 오버로드하며 표준 컨테이너 설계를 따른다.	(5.5, p.138-139 / PDF p.136-137)
U05-117	저장 공간	(복사와 이동, p.125-131 / PDF p.123-129)
U05-118	새 배열을 할당하고 요소 복사	(복사와 이동, p.125-131 / PDF p.123-129)
U05-119	기존 배열 포인터를 가져옴	(복사와 이동, p.125-131 / PDF p.123-129)
U05-120	원본	(복사와 이동, p.125-131 / PDF p.123-129)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-121	자기 요소를 계속 가짐	(복사와 이동, p.125-131 / PDF p.123-129)
U05-122	예제에서는 nullptr·크기 0	(복사와 이동, p.125-131 / PDF p.123-129)
U05-123	입력	(복사와 이동, p.125-131 / PDF p.123-129)
U05-124	const Vector&	(복사와 이동, p.125-131 / PDF p.123-129)
U05-125	Vector&&	(복사와 이동, p.125-131 / PDF p.123-129)
U05-126	결과	(복사와 이동, p.125-131 / PDF p.123-129)
U05-127	독립된 같은 값	(복사와 이동, p.125-131 / PDF p.123-129)
U05-128	자원의 소유권 이전	(복사와 이동, p.125-131 / PDF p.123-129)
U05-129	=default	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-130	컴파일러 기본 구현을 명시	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-131	=delete	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-132	함수 사용을 컴파일 시간에 금지	(생성자 제어 표기, p.121-124 / PDF p.119-122)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-133	explicit	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-134	한 인자 생성자의 암묵 변환 제한	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-135	멤버 기본 초깃값	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-136	생성자가 값을 지정하지 않은 멤버 초기화	(생성자 제어 표기, p.121-124 / PDF p.119-122)
U05-137	일반적인 생성자를 제외한 특수 멤버 함수의 기본 구현을 명시적으로 만들고 싶을 때 =default를 사용한다.	(5.1.1, p.121 / PDF p.119)
U05-138	=delete로 제거한 함수를 사용하면 컴파일 시간 에러가 발생한다.	(5.1.1, p.122 / PDF p.120)
U05-139	원문의 Vector 복사 생성자는 새 저장 공간을 할당하고 요소를 복사하여 각 Vector가 자기 요소의 복사본을 갖게 한다.	(5.2.1, p.126 / PDF p.124)
U05-140	클래스 객체의 복사는 복사 생성자와 복사 대입이라는 두 멤버로 정의할 수 있다.	(5.2.1, p.126 / PDF p.124)
U05-141	원문의 Vector 이동 생성자는 원본의 elem과 sz를 가져오고 원본을 nullptr와 크기 0으로 만든다.	(5.2.2, p.129 / PDF p.127)
U05-142	&&는 rvalue 참조를 나타내며, 누구도 다시는 대입할 수 없는 것의 참조로 값을 이동시킬 수 있다.	(5.2.2, p.129 / PDF p.127)
U05-143	생성자와 대입 연산자, 소멸자를 상호 연관된 연산자의 집합으로 설계한다.	(5.5, p.138 / PDF p.136)
U05-144	기본 복사가 적합하지 않은 타입에는 복사를 재정의하거나 막는다.	(5.5, p.139 / PDF p.137)
U05-145	자원으로 취급할 수 있는 어떤 것도 누수되지 않게 강력한 자원 안전성을 제공한다.	(5.5, p.139 / PDF p.137)

번호	문장·표 항목·원문 코드·수식	원문 위치
U05-146	연산자를 오버로드할 때 관례적인 사용법을 흉내 내고 표준 라이브러리 컨테이너의 설계를 따른다.	(5.5, p.139 / PDF p.137)
U05-147	=default는 기본 구현을 명시적으로 만들게 하고, =delete는 해당 함수의 사용을 컴파일 시간에 금지한다.	(5.1.1, p.121-122 / PDF p.119-120)
U05-148	Vector 복사	(5.2.1, p.126 / PDF p.124)
U05-149	Vector 이동	(5.2.2, p.129 / PDF p.127)

제6장. 템플릿

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-001	템플릿은 타입이나 값의 집합을 파라미터화한 클래스나 함수다.	(6.1, p.141 / PDF p.138)
U06-002	if constexpr에서는 선택된 경우만 인스턴스화되므로 지역적인 최적화가 가능하다.	(6.4.3, p.160 / PDF p.157)
U06-003	템플릿은 타입이나 값의 집합을 파라미터화한 클래스 또는 함수다.	(6.1, p.141 / PDF p.138)
U06-004	특정 타입에 종속되지 않는 벡터와 같은 일반적인 개념을 표현하고, 인자를 지정하여 특정 타입이나 함수를 만들어 낸다.	(6.1, p.141 / PDF p.138)
U06-005	요소를 저장한다는 벡터의 개념과 요소가 double이라는 사실을 분리하는 것이 출발점이다.	(6.1, p.141 / PDF p.138)
U06-006	template<typename T>는 앞으로 선언할 클래스의 타입 파라미터를 T로 지정한다.	(6.2, p.142-145 / PDF p.139-142)
U06-007	이는 모든 타입 T에 대해라는 뜻이다.	(6.2, p.142-145 / PDF p.139-142)
U06-008	타입 파라미터 자리에 class를 쓰는 오래된 표기도 같은 의미다.	(6.2, p.142-145 / PDF p.139-142)
U06-009	클래스의 멤버를 밖에서 정의할 때도 template 접두사와 Vector<T>를 이용한다.	(6.2, p.142-145 / PDF p.139-142)

번호	문장·표 항목·원문 코드·수식	원문 위치

U06-010	타입 파라미터로 일반화한 Vector (1/2) (원문 코드 발췌)
	template<typename T>
	class Vector {
	private:
	T* elem; // elem은 T 타입 요소 sz개를 포함하는 배열을 가리킴
	int sz;
	public:
	explicit Vector(int s); // 생성자: 불변 조건을 조성하고, 자원을 획득
	~Vector() { delete[] elem; } // 소멸자: 자원 해제
	
	// ... 복사와 이동 연산 ...
	
	T& operator[](int i); // const가 아닌 Vector를 위한 연산자
	const T& operator[](int i) const; // const Vector를 위한 연산자 (4.2.1절)
	int size() const { return sz; }
	};
	
	template<typename T>
	Vector<T>::Vector(int s)
	{
	if (s<0)
	throw Negative_size{};
	elem = new T[s];
	sz = s;
	}
	
	template<typename T>
	const T& Vector<T>::operator[](int i) const
	{
	if (i<0 || size()<=i)
		(6.2, p.142-143 / PDF p.139-140)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-011	타입 파라미터로 일반화한 Vector (2/2) (원문 코드 발췌) throw out_of_range{"Vector::operator[]"}; return elem[i]; }	(6.2, p.142-143 / PDF p.139-140)
U06-012	Vector<char>, Vector<string>, Vector<list<int>>는 서로 다른 요소 타입의 벡터다.	(6.2, p.143-144 / PDF p.140-141)
U06-013	중첩된 템플릿 인자 끝의 >>는 입력 연산자가 아니다.	(6.2, p.143-144 / PDF p.140-141)
U06-014	사용자 정의 Vector를 구간 for로 순회하려면 첫 요소를 나타내는 begin()과 마지막 요소 다음을 나타내는 end()를 알맞게 제공해야 한다.	(6.2, p.143-144 / PDF p.140-141)
U06-015	템플릿은 컴파일 시간 메커니즘이므로 같은 일을 손으로 작성한 코드에 비해 실행 시간의 추가 부담이 없다.	(6.2, p.144-145 / PDF p.141-142)
U06-016	템플릿과 템플릿 인자의 집합을 인스턴스화 또는 특수화라고 부른다.	(6.2, p.144-145 / PDF p.141-142)
U06-017	사용하는 각 인스턴스의 코드는 컴파일 후반의 인스턴스화 시간에 생성되며 타입 검사를 받는다.	(6.2, p.144-145 / PDF p.141-142)
U06-018	따라서 타입 안전성은 보장되지만 검사 시점이 늦다.	(6.2, p.144-145 / PDF p.141-142)
U06-019	템플릿에 의미가 있으려면 인자가 특정 성질을 만족해야 한다.	(6.2.1, p.145 / PDF p.142)
U06-020	Vector가 복사 연산을 제공하려면 요소도 복사할 수 있어야 한다.	(6.2.1, p.145 / PDF p.142)
U06-021	Element와 같은 컨셉은 이 요구 사항을 확인하는 술어다.	(6.2.1, p.145 / PDF p.142)
U06-022	컨셉을 지정한 인자는 제한된 인자, 이러한 인자를 사용하는 템플릿은 제한된 템플릿이다.	(6.2.1, p.145 / PDF p.142)
U06-023	C++20 이전 코드에서는 필요한 조건을 문서화한다고 설명한다.	(6.2.1, p.145 / PDF p.142)
U06-024	[원문 확인 필요] 원문 예는 Vector<int>를 OK, Vector<thread>를 표준 스레드 복사 불가에 의한 에러로 표시한다.	(6.2.1, p.145 / PDF p.142)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-025	다만 int 예의 주석은 “int 는 복사할 수 없음”으로 인쇄되어 있어 본문의 복사 가능 조건 및 OK 표시와 함께 원문 확인이 필요하다.	(6.2.1, p.145 / PDF p.142)
U06-026	이 주석을 반대로 고쳐 옮기지 않았다.	(6.2.1, p.145 / PDF p.142)
U06-027	템플릿은 타입뿐 아니라 값도 인자로 받는다.	(6.2.2, p.146 / PDF p.143)
U06-028	Buffer의 T는 요소 타입, N은 크기를 나타낸다.	(6.2.2, p.146 / PDF p.143)
U06-029	이 방식으로 자유 저장 영역을 사용하지 않는 전역 버퍼와 스택의 지역 버퍼를 만들 수 있다.	(6.2.2, p.146 / PDF p.143)
U06-030	값 템플릿 인자는 반드시 상수 표현식이어야 한다.	(6.2.2, p.146 / PDF p.143)

U06-031	값 템플릿 인자를 받는 Buffer (원문 코드 발췌)
	template<typename T, int N>
	struct Buffer {
	 using value_type = T;
	 constexpr int size() { return N; }
	 T[N];
	 // ...
	};
		(6.2.2, p.146 / PDF p.143)

U06-032	make_pair()는 전달받은 함수 인자로 반환할 pair의 템플릿 인자를 추론한다.	(6.2.3, p.146-149 / PDF p.143-146)
U06-033	C++17에서는 생성자 인자를 통해 클래스 템플릿 인자도 추론할 수 있다.	(6.2.3, p.146-149 / PDF p.143-146)
U06-034	원문 Vector에서는 초깃값 목록의 요소 타입이나 복사할 Vector의 타입에서 요소 타입을 알아내지만, 크기만 지정하는 생성에서는 요소 타입을 명시해야 한다.	(6.2.3, p.146-149 / PDF p.143-146)
U06-035	추론은 중복을 줄이지만 항상 의도한 타입을 만들지는 않는다.	(6.2.3, p.148 / PDF p.145)
U06-036	C 스타일 문자열 리터럴을 요소로 전달하면 string이 아닌 const char*가 추론될 수 있다.	(6.2.3, p.148 / PDF p.145)
U06-037	string 리터럴을 원하면 접미사 s를 사용한다.	(6.2.3, p.148 / PDF p.145)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-038	초깃값 목록의 요소 타입이 서로 일치하지 않으면 유일한 요소 타입을 추론하지 못한다.	(6.2.3, p.148 / PDF p.145)
U06-039	생성자 인자 한 쌍이 반복자라는 사실과 반복자의 값 타입을 컴파일러가 추론하지 못하는 경우에는 추론 가이드를 지정할 수 있다.	(6.2.3, p.148-149 / PDF p.145-146)
U06-040	원문 Vector2는 반복자의 value_type에서 요소 타입을 알아내게 한다.	(6.2.3, p.148-149 / PDF p.145-146)
U06-041	다만 추론 가이드의 효과에는 미묘한 부분이 있어, 가능하다면 가이드가 필요하지 않게 클래스 템플릿을 설계하라고 조언한다.	(6.2.3, p.148-149 / PDF p.145-146)
U06-042	타입이나 값에 따라 달라지는 연산은 함수 템플릿, 함수 객체, 람다 표현식으로 나타낼 수 있다.	(6.3, p.149 / PDF p.146)
U06-043	함수 객체는 데이터도 보유하면서 함수처럼 호출되는 객체이고, 람다는 이러한 함수 객체를 간결하게 표기하는 방법이다.	(6.3, p.149 / PDF p.146)
U06-044	표준 라이브러리는 타입과 알고리즘 모두를 파라미터화하는 데 이 수단들을 사용한다.	(6.3, p.149 / PDF p.146)
U06-045	sum()의 시퀀스 타입과 누산기 타입은 서로 독립적이다.	(6.3.1, p.150 / PDF p.147)
U06-046	호출자가 초기 누산기 값을 0으로 주는지 0.0으로 주는지에 따라 누산기의 타입이 달라지므로, 같은 int 벡터를 int 또는 double 덧셈으로 처리할 수 있다.	(6.3.1, p.150 / PDF p.147)
U06-047	원문은 double 누산기로 int의 최댓값보다 큰 합을 처리할 수 있다고 설명한다.	(6.3.1, p.150 / PDF p.147)
U06-048	이 sum()은 표준 accumulate()의 단순화다.	(6.3.1, p.150 / PDF p.147)

번호	문장·표 항목·원문 코드·수식	원문 위치

U06-049	시퀀스와 누산기 타입을 분리한 sum (원문 코드 발췌)
	template<typename Sequence, typename Value>
	Value sum(const Sequence& s, Value v)
	{
	for (auto x : s)
	v+=x;
	return v;
	}
		(6.3.1, p.150 / PDF p.147)

U06-050	함수 템플릿은 클래스의 멤버가 될 수 있지만 virtual 멤버는 될 수 없다.	(6.3.1, p.150 / PDF p.147)
U06-051	컴파일러가 프로그램에 존재하는 모든 인스턴스화를 알 수 없어 가상 함수 테이블을 만들 수 없기 때문이다.	(6.3.1, p.150 / PDF p.147)
U06-052	operator()를 정의한 객체는 함수처럼 호출할 수 있다.	(6.3.2, p.151-152 / PDF p.148-149)
U06-053	Less_than은 비교 기준을 객체 안에 저장하고 그 값보다 작은지를 판단한다.	(6.3.2, p.151-152 / PDF p.148-149)
U06-054	술어는 호출할 수 있고 결과로 true 또는 false를 반환하는 무엇이다.	(6.3.2, p.151-152 / PDF p.148-149)
U06-055	count()는 컨테이너의 각 요소에 술어를 적용해 참인 요소의 개수를 센다.	(6.3.2, p.151-152 / PDF p.148-149)

U06-056	데이터를 보유하는 함수 객체 (원문 코드 발췌)
	template<typename T>
	class Less_than {
	const T val; // 비교 대상
	public:
	Less_than(const T& v) :val{v} { }
	bool operator()(const T& x) const { return x<val; } // 연산자 호출
	};
		(6.3.2, p.151 / PDF p.148)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-057	함수 객체는 서로 다른 비교 기준마다 별도 함수를 정의하거나 전역 변수에 기준을 저장하지 않아도 된다.	(6.3.2, p.152 / PDF p.149)
U06-058	간단한 함수 객체는 인라인으로 처리하기 쉬워 간접 함수 호출보다 효율적이라고 설명한다.	(6.3.2, p.152 / PDF p.149)
U06-059	일반 알고리즘에서 핵심 동작의 의미를 지정하는 이런 객체를 정책 객체라고도 부른다.	(6.3.2, p.152 / PDF p.149)
U06-060	람다는 사용되는 위치에서 함수 객체를 암묵적으로 만든다.	(6.3.3, p.153 / PDF p.150)
U06-061	캡처 리스트 [&]는 몸체에서 사용하는 지역 이름을 참조로 접근하고, [&x]는 x만 참조로 캡처한다.	(6.3.3, p.153 / PDF p.150)
U06-062	원문의 복사 캡처 표기 설명에는 [=x]가 인쇄되어 있어 [원문 확인 필요]로 남긴다.	(6.3.3, p.153 / PDF p.150)
U06-063	캡처할 지역 이름이 없을 때 빈 []를 사용할 수 있다.	(6.3.3, p.153 / PDF p.150)

U06-064	같은 위치에서 정의하는 람다 (원문 코드 발췌)
	void f(const Vector<int>& vec, const list<string>& lst, int x, const string& s)
	{
	cout << x
	<< "보다 작은 값의 개수: " << count(vec, [&](int a){ return a<x; })
	<< '\n';
	cout << s
	<< "보다 작은 값의 개수: " << count(lst, [&](const string& a){ return a<s; })
	<< '\n';
	}
		(6.3.3, p.153 / PDF p.150)

U06-065	람다는 간단한 동작을 한 곳에서 쓸 때 편리하다.	(6.3.3, p.153-154 / PDF p.150-151)
U06-066	단순하지 않은 동작은 이름을 부여하여 목적을 명확히 하고 여러 곳에서 사용할 수 있게 한다.	(6.3.3, p.153-154 / PDF p.150-151)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-067	for_all()과 람다를 결합하면 컨테이너를 순회하는 작업과 각 요소에서 수행할 작업을 분리할 수 있다.	(6.3.3, p.153-154 / PDF p.150-151)
U06-068	원문은 unique_ptr<Shape>를 참조로 받으므로 호출이 Shape의 수명에 영향을 주지 않는다고 설명한다.	(6.3.3, p.153-154 / PDF p.150-151)
U06-069	람다의 파라미터를 auto로 선언하면 다양한 타입을 받는 제네릭 람다가 된다.	(6.3.3, p.154-155 / PDF p.151-152)
U06-070	원문 rotate_and_draw()는 포인터 또는 unique_ptr로 저장한 Shape를 같은 방식으로 회전하고 그린다.	(6.3.3, p.154-155 / PDF p.151-152)
U06-071	일반 함수 인자에 auto를 사용할 수 없다는 원문의 당시 설명은 현재 언어 지식으로 보충하거나 수정하지 않는다.	(6.3.3, p.154-155 / PDF p.151-152)
U06-072	람다는 복잡한 초기화에도 쓰인다.	(6.3.3, p.155-157 / PDF p.152-154)
U06-073	먼저 벡터를 선언하고 switch의 각 가지에서 대입하면, 의도한 값 전에 사용될 위험이 있고 초기화 코드가 다른 코드와 섞이거나 경우 하나를 놓치기 쉽다.	(6.3.3, p.155-157 / PDF p.152-154)
U06-074	값을 반환하는 람다를 초깃값으로 사용하면 이 선택을 초기화 위치에 모을 수 있다.	(6.3.3, p.155-157 / PDF p.152-154)
U06-075	원문은 개선 예에도 case 하나가 빠졌지만 이를 더 쉽게 알아볼 수 있다고 지적한다.	(6.3.3, p.155-157 / PDF p.152-154)
U06-076	좋은 템플릿을 만드는 도구에는 타입에 의존하는 값, 타입·템플릿의 별칭, if constexpr에 의한 컴파일 시간 선택, requires 표현식에 의한 타입·표현식 속성 확인이 있다.	(6.4, p.157 / PDF p.154)
U06-077	constexpr 함수와 static_assert도 함께 사용한다.	(6.4, p.157 / PDF p.154)
U06-078	이 기능들은 일반적인 추상화를 구성하는 기본 수단이다.	(6.4, p.157 / PDF p.154)
U06-079	이 소절은 타입 T에 따라 달라지는 상수나 변수를 정의하는 예를 다룬다.	(6.4.1, p.157-158 / PDF p.154-155)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-080	원문은 점성 계수 viscosity<T>, 3차원 벡터 external_acceleration<T>와 대입 가능성을 나타내는 Assignable<T,T2>를 제시한다.	(6.4.1, p.157-158 / PDF p.154-155)
U06-081	값은 constexpr 표현식으로 정의할 수 있고 static_assert로 해당 타입의 조건을 검사할 수 있다.	(6.4.1, p.157-158 / PDF p.154-155)
U06-082	이 아이디어는 7장의 컨셉 정의로 이어진다.	(6.4.1, p.157-158 / PDF p.154-155)
U06-083	원문 소절 제목을 유지하며 7.4의 가변 개수 인자 템플릿과 구별해 읽는다.	(6.4.1, p.157-158 / PDF p.154-155)
U06-084	using은 타입이나 템플릿의 다른 이름을 정의한다.	(6.4.2, p.158-160 / PDF p.155-157)
U06-085	size_t의 실제 타입은 구현마다 다르지만 같은 이름으로 접근하면 이식성을 유지할 수 있다.	(6.4.2, p.158-160 / PDF p.155-157)
U06-086	컨테이너가 요소 타입을 value_type으로 제공하는 관례를 따르면, 다른 코드가 특정 컨테이너 이름 없이 요소 타입을 얻을 수 있다.	(6.4.2, p.158-160 / PDF p.155-157)
U06-087	템플릿 인자의 일부를 고정하여 남은 인자만 받는 별칭 템플릿도 정의할 수 있다.	(6.4.2, p.158-160 / PDF p.155-157)
U06-088	일부 인자를 지정한 템플릿 별칭 (원문 코드 발췌) template<typename Value> using String_map = Map<string,Value>; String_map<int> m; // m의 타입은 Map<string,int>	(6.4.2, p.160 / PDF p.157)
U06-089	C++17의 if constexpr은 타입의 성질에 따라 필요한 구현을 선택한다.	(6.4.3, p.160-161 / PDF p.157-158)
U06-090	원문은 is_pod<T>를 바탕으로 단순하고 빠른 연산과 느리지만 안전한 연산을 구분한다.	(6.4.3, p.160-161 / PDF p.157-158)
U06-091	선택된 경우만 인스턴스화되므로 특정 타입에 대한 지역 최적화를 할 수 있다.	(6.4.3, p.160-161 / PDF p.157-158)

번호	문장·표 항목·원문 코드·수식	원문 위치

U06-092	컴파일 시간 if로 구현 선택 (원문 코드 발췌)
	template<typename T>
	void update(T& target)
	{
	// ...
	if constexpr(is_pod<T>::value)
	simple_and_fast(target); // 단순한 데이터용
	else
	slow_and_safe(target);
	// ...
	}
		(6.4.3, p.160 / PDF p.157)

U06-093	if constexpr은 단순한 텍스트 조작이 아니다.	(6.4.3, p.160-161 / PDF p.157-158)
U06-094	일반적인 문법·타입·스코프 규칙을 지켜야 하며, 조건부로 try의 시작과 끝을 서로 분리하는 원문 bad() 예는 문법 에러다.	(6.4.3, p.160-161 / PDF p.157-158)
U06-095	여러 인자 타입에 적용할 알고리즘과 컨테이너를 템플릿으로 표현한다.	(6.5, p.161-162 / PDF p.158-159)
U06-096	간단한 함수 객체를 한 곳에서만 사용한다면 람다를 이용한다.	(6.5, p.161-162 / PDF p.158-159)
U06-097	표기를 단순하게 하고 구현을 숨길 때 템플릿 별칭을 사용한다.	(6.5, p.161-162 / PDF p.158-159)
U06-098	템플릿의 타입 검사는 컴파일 시간에 수행되지만 인스턴스화까지 늦어질 수 있다.	(6.5, p.161-162 / PDF p.158-159)
U06-099	사용되는 스코프에는 템플릿의 선언뿐 아니라 정의도 필요하다.	(6.5, p.161-162 / PDF p.158-159)
U06-100	원문은 템플릿 정의를 사용하는 모든 변환 단위에서 포함하도록 조언한다.	(6.5, p.161-162 / PDF p.158-159)
U06-101	함수 템플릿	(파라미터화된 연산의 표현, p.149-153 / PDF p.146-150)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-102	타입·값이 달라져도 공통 연산 수행	(파라미터화된 연산의 표현, p.149-153 / PDF p.146-150)
U06-103	함수 객체	(파라미터화된 연산의 표현, p.149-153 / PDF p.146-150)
U06-104	호출 연산과 데이터를 함께 전달	(파라미터화된 연산의 표현, p.149-153 / PDF p.146-150)
U06-105	람다	(파라미터화된 연산의 표현, p.149-153 / PDF p.146-150)
U06-106	사용 위치에서 함수 객체를 간단히 생성	(파라미터화된 연산의 표현, p.149-153 / PDF p.146-150)
U06-107	컨셉이 적용된 템플릿 인자를 제한된 인자라고 한다.	(6.2.1, p.145 / PDF p.142)
U06-108	템플릿의 값 인자는 반드시 상수 표현식이어야 한다.	(6.2.2, p.146 / PDF p.143)
U06-109	함수 객체는 함수처럼 호출할 수 있는 객체다.	(6.3.2, p.151 / PDF p.148)
U06-110	술어는 호출 가능하며 그 결과로 true나 false를 반환하는 무언가를 말한다.	(6.3.2, p.152 / PDF p.149)
U06-111	람다 표현식은 함수 객체를 암묵적으로 만들어내는 표기법이다.	(6.3.3, p.153 / PDF p.150)
U06-112	별칭은 타입이나 템플릿의 동의어다.	(6.4.2, p.158 / PDF p.155)
U06-113	다양한 인자 타입에 적용할 알고리즘과 컨테이너를 템플릿으로 표현한다.	(6.5, p.161 / PDF p.158)
U06-114	간단한 함수 객체를 한 곳에서만 사용한다면 람다를 이용한다.	(6.5, p.161 / PDF p.158)
U06-115	표기를 간단히 하고 상세 구현을 숨기기 위해 템플릿 별칭을 사용한다.	(6.5, p.162 / PDF p.159)

번호	문장·표 항목·원문 코드·수식	원문 위치
U06-116	템플릿을 사용할 때는 선언뿐 아니라 정의가 스코프에 있는지 확인한다.	(6.5, p.162 / PDF p.159)
U06-117	함수 객체는 함수처럼 호출할 수 있는 객체이며, 술어는 호출 가능하고 true나 false를 반환하는 무언가다.	(6.3.2, p.151-152 / PDF p.148-149)
U06-118	함수 객체	(6.3.2, p.151 / PDF p.148)
U06-119	람다 표현식	(6.3.3, p.153 / PDF p.150)

제7장. 컨셉과 제네릭 프로그래밍

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-001	컨셉은 하나 이상의 타입을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 술어다.	(7.2.4, p.170 / PDF p.167)
U07-002	접힘 표현식은 파라미터 팩의 요소를 연산으로 결합하며, 원문은 오른쪽 접힘과 왼쪽 접힘을 구분한다.	(7.4.1, p.178 / PDF p.175)
U07-003	템플릿은 타입·값·템플릿 자체를 정보 손실 없이 인자로 전달하고, 인스턴스화 때 알게 되는 정보로 최적화하며, 상수 인자로 컴파일 시간 계산도 수행하게 한다.	(7.1, p.163-164 / PDF p.160-161)
U07-004	클래스는 코드와 값을 함께 담을 수 있으므로 템플릿의 파라미터화 대상은 단순한 요소 타입에 그치지 않는다.	(7.1, p.163-164 / PDF p.160-161)
U07-005	제네릭 프로그래밍의 중심은 요구 사항을 만족하는 여러 타입에 공통으로 적용할 알고리즘을 설계·구현·사용하는 것이다.	(7.1, p.163-164 / PDF p.160-161)
U07-006	템플릿은 컴파일 시간에 파라미터화할 수 있는 다형성을 제공하고, 컨셉은 인자에 요구되는 기본 개념을 표현한다.	(7.1, p.163-164 / PDF p.160-161)
U07-007	sum()은 서로 다른 시퀀스의 자료 구조와 서로 다른 요소 타입을 모두 받는다.	(7.2, p.164-165 / PDF p.161-162)
U07-008	다만 시퀀스는 구간 for를 사용할 수 있는 begin()/end()를 제공해야 하고 요소는 누산할 값에 더할 수 있어야 한다.	(7.2, p.164-165 / PDF p.161-162)
U07-009	이처럼 템플릿 인자가 만족해야 할 요구 사항을 컨셉이라고 한다.	(7.2, p.164-165 / PDF p.161-162)
U07-010	원문은 컨셉의 언어 지원이 아직 ISO C++에 포함되지 않았고 ISO 기술 명세에 있으며, 구현체는 있지만 세부 사항의 변경이나 상용 사용까지의 시간이 위험 요소라고 설명한다.	(7.2, p.164-165 / PDF p.161-162)
U07-011	소절 제목의 C++20 표기와 함께 이 당시 상태를 그대로 유지한다.	(7.2, p.164-165 / PDF p.161-162)
U07-012	typename은 인자가 타입이어야 한다는 최소 요구만 지정한다.	(7.2.1, p.165-167 / PDF p.162-164)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-013	Sequence와 Number라는 의미 있는 제약을 쓰면 의도가 드러나고, 구현을 모두 보지 않아도 인터페이스에서 잘못된 호출을 막을 수 있어 에러 보고가 좋아진다.	(7.2.1, p.165-167 / PDF p.162-164)
U07-014	그러나 시퀀스와 수라는 두 개별 조건만으로 요소와 누산기의 연산 관계까지 완전히 지정한 것은 아니다.	(7.2.1, p.165-167 / PDF p.162-164)
U07-015	시퀀스와 수의 관계를 지정하는 요구 절 (원문 코드 발췌) template<Sequence Seq, Number Num> requires Arithmetic<Value_type<Seq>,Num> Num sum(Seq s, Num n);	(7.2.1, p.166 / PDF p.163)
U07-016	Value_type은 시퀀스의 요소 타입이다.	(7.2.1, p.166 / PDF p.163)
U07-017	Arithmetic<X,Y>는 두 수치 타입 사이의 산술 연산 가능성을 표현한다.	(7.2.1, p.166 / PDF p.163)
U07-018	원문은 실제 구현에 +=만 등장해도 이를 지나치게 좁은 요구로 삼으면 유연성이 줄어든다고 설명한다.	(7.2.1, p.166 / PDF p.163)
U07-019	나중에 +와 =를 이용하는 구현으로 바꿀 수도 있으므로 일반적인 의미를 가진 컨셉이 더 낫다.	(7.2.1, p.166 / PDF p.163)
U07-020	부분적인 명세도 의도를 알리고 점진적으로 개발하는 데 유용하지만, 명시하지 못한 요구에 관한 에러는 인스턴스화 시간에 나타날 수 있다.	(7.2.1, p.166-167 / PDF p.163-164)
U07-021	requires 절과 template<Sequence Seq>와 같은 축약 표기를 사용할 수 있다.	(7.2.1, p.166-167 / PDF p.163-164)
U07-022	컨셉을 쓸 수 없는 환경에서는 명명 관례와 주석으로 요구를 나타낸다고 설명한다.	(7.2.1, p.166-167 / PDF p.163-164)
U07-023	템플릿도 인자가 만족하는 속성에 따라 오버로딩할 수 있다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-024	원문 advance()는 순방향 반복자에는 반복해서 ++를 수행하고, 임의 접근 반복자에는 +=를 수행하는 두 버전을 비교한다.	(7.2.2, p.167-168 / PDF p.164-165)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-025	컴파일러는 인자가 만족하는 가장 강한 요구 사항에 맞춰 선택한다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-026	이 선택은 컴파일 시간에 이루어져 실행 시간의 부담을 추가하지 않는다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-027	컨셉을 만족하지 못하는 인자가 있는 함수는 후보에서 제외한다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-028	맞는 함수가 하나뿐이면 선택하고, 똑같이 맞는 두 함수를 구분하지 못하면 모호성 에러를 낸다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-029	한 함수의 요구가 다른 함수의 모든 요구와 추가 요구를 포함한다면 더 엄격하게 맞는 함수를 선택한다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-030	선택되는 함수는 모든 인자가 맞아야 하고, 다른 후보보다 각 인자가 적어도 같은 정도로 맞으며, 적어도 하나는 더 잘 맞아야 한다.	(7.2.2, p.167-168 / PDF p.164-165)
U07-031	요구 사항의 바탕에는 어떤 표현식이 유효한가라는 문제가 있다.	(7.2.3, p.169-170 / PDF p.166-167)
U07-032	requires 표현식은 주어진 표현식 집합이 유효한 코드인지 판단하는 술어다.	(7.2.3, p.169-170 / PDF p.166-167)
U07-033	requires requires에서 첫 단어는 요구 절을, 두 번째 단어는 이 표현식을 시작하므로 중복 자체는 오타가 아니다.	(7.2.3, p.169-170 / PDF p.166-167)
U07-034	표현식의 유효성 확인 (원문 코드 발췌) requires(Iter p, int i) { p[i]; p+i; }	(7.2.3, p.169 / PDF p.166)
U07-035	원문은 표현식을 직접 나열한 advance()에서도 +=와 반환 타입 요구를 빠뜨렸다고 지적한다.	(7.2.3, p.169-170 / PDF p.166-167)
U07-036	이런 저수준 표현은 매우 유연하지만 의미를 전달하기 어렵다.	(7.2.3, p.169-170 / PDF p.166-167)
U07-037	보통 코드에는 적절한 이름과 명시된 의미가 있는 컨셉을 사용하고, requires 표현식은 컨셉 정의에 사용하도록 조언한다.	(7.2.3, p.169-170 / PDF p.166-167)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-038	컨셉은 하나 이상의 타입을 어떻게 사용할 수 있는지 지정하는 컴파일 시간 술어이며 값은 항상 bool이다.	(7.2.4, p.170-172 / PDF p.167-169)
U07-039	원문 Equality_comparable은 같은 타입의 두 값을 ==와 !=로 비교할 수 있고 그 결과가 bool로 변환될 수 있어야 한다는 요구다.	(7.2.4, p.170-172 / PDF p.167-169)
U07-040	int에 대한 검사는 성공하지만, 구조체에 이러한 연산이 자동으로 정의되지 않으면 검사가 실패한다.	(7.2.4, p.170-172 / PDF p.167-169)
U07-041	두 타입을 비교하도록 확장할 때는 양방향의 ==와 !=를 요구한다.	(7.2.4, p.171 / PDF p.168)
U07-042	두 번째 템플릿 인자를 지정하지 않을 때 첫 타입을 쓰도록 기본 인자를 줄 수 있다.	(7.2.4, p.171 / PDF p.168)
U07-043	원문은 int와 double, 같은 int의 검사는 성공하고 int와 string의 검사는 실패하는 예를 제시한다.	(7.2.4, p.171 / PDF p.168)
U07-044	Sequence 정의는 값 타입과 반복자 타입을 제공하고, 컨테이너 관례에 맞는 begin()/end()를 제공하며, 반복자가 요소 타입에 맞는 입력 반복자여야 한다는 조건을 결합한다.	(7.2.4, p.171-172 / PDF p.168-169)
U07-045	기본 언어 개념을 정확히 표현하는 컨셉을 정의하는 일이 가장 어려우므로, 잘 정돈된 라이브러리의 컨셉을 이용하라고 설명한다.	(7.2.4, p.171-172 / PDF p.168-169)
U07-046	구체적이고 효율적인 알고리즘에서 자료 표현에 묶인 부분을 추상화하여 서로 다른 표현에도 적용하는 것이 제네릭 프로그래밍의 중심이다.	(7.3, p.172 / PDF p.169)
U07-047	컨셉은 기본 연산과 자료 구조에 대한 추상화이자 템플릿 파라미터가 만족해야 하는 요구 사항이다.	(7.3, p.172 / PDF p.169)
U07-048	유용한 컨셉은 정수·부동소수점 수·시퀀스와 같은 해당 분야의 기본 개념을 표현한다.	(7.3.1, p.172-173 / PDF p.169-170)
U07-049	Regular 타입은 기본 생성이 가능하고 생성·대입으로 독립적이면서 같은 값인 복사본을 만들 수 있으며 ==와 !=로 비교할 수 있다.	(7.3.1, p.172-173 / PDF p.169-170)
U07-050	원문은 과도하게 영리한 프로그래밍 기법으로 기술적 문제를 겪지 않는다는 조건도 덧붙인다.	(7.3.1, p.172-173 / PDF p.169-170)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-051	int, vector, string이 예이며, int와 string은 <·<=·>·>=로 비교 가능한 StrictTotallyOrdered의 예로도 설명한다.	(7.3.1, p.172-173 / PDF p.169-170)
U07-052	컨셉은 단순한 구문보다 의미에 관한 개념이다.	(7.3.1, p.173 / PDF p.170)
U07-053	예를 들어 +가 존재하기만 한다고 충분하지 않으며, +를 나눗셈으로 정의해서는 합리적인 수의 요구를 만족하지 않는다.	(7.3.1, p.173 / PDF p.170)
U07-054	언어로 의미론 전체를 표현할 수 없으므로 분야의 지식과 상식이 필요하다.	(7.3.1, p.173 / PDF p.170)
U07-055	원문은 의미 없는 Addable·Subtractable 같은 개념을 나열하기보다 분야의 기본 개념에 대응하도록 설계하라고 조언한다.	(7.3.1, p.173 / PDF p.170)
U07-056	좋은 추상화는 실제 구체적 용례에서 시작한다.	(7.3.2, p.173-175 / PDF p.170-172)
U07-057	모든 미래 필요를 예상하여 추상화하기보다는 하나 이상의 사례에서 불필요한 세부를 제거한다.	(7.3.2, p.173-175 / PDF p.170-172)
U07-058	원문의 합산은 요소가 int인지, 저장소가 vector인지, 누산기가 double인지, 초깃값이 0인지에 대한 고정을 템플릿 인자와 함수 인자로 풀어낸다.	(7.3.2, p.173-175 / PDF p.170-172)

U07-059	구체적인 합산을 일반화한 accumulate (원문 코드 발췌)
	template<typename Iter, typename Val>
	Val accumulate(Iter first, Iter last, Val res)
	{
	for (auto p = first; p!=last; ++p)
	res += *p;
	return res;
	}
		(7.3.2, p.174 / PDF p.171)

U07-060	반복자 한 쌍은 자료 구조를 추상화하고, Val은 누산기 타입을, res는 초깃값을 전달한다.	(7.3.2, p.174-175 / PDF p.171-172)
U07-061	동일한 성능을 유지하며 구체적인 코드를 일반화하는 과정을 리프팅이라고 한다.	(7.3.2, p.174-175 / PDF p.171-172)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-062	권장되는 개발 순서는 구체 버전을 만들고 디버그·테스트·측정한 후 구체 타입을 템플릿 인자로 교체하는 것이다.	(7.3.2, p.174-175 / PDF p.171-172)
U07-063	begin()/end() 반복을 줄이려면 Range를 받는 인터페이스로 단순화할 수 있고, 더 일반화하려면 +=도 대체할 수 있다.	(7.3.2, p.174-175 / PDF p.171-172)
U07-064	가변 템플릿은 다양한 타입의 인자를 가변 개수로 받는다. ...으로 선언하는 파라미터를 파라미터 팩이라고 하며, Tail은 타입 시퀀스, tail은 그 타입들에 해당하는 값의 시퀀스다.	(7.4, p.176-177 / PDF p.173-174)
U07-065	전통적인 구현은 첫 인자를 처리한 뒤 나머지 팩을 재귀 호출한다.	(7.4, p.176-177 / PDF p.173-174)

U07-066	파라미터 팩을 재귀적으로 처리 (원문 코드 발췌)
	void print()
	{
	// 인자가 없으면 아무것도 하지 않음
	}
	
	template<typename T, typename ... Tail>
	void print(T head, Tail... tail)
	{
	// 각 인자에 대해
	cout << head << ' ';
	print(tail...);
	}
		(7.4, p.176 / PDF p.173)

U07-067	마지막에 팩이 비면 인자 없는 print()가 호출되어 종료한다.	(7.4, p.176-177 / PDF p.173-174)
U07-068	빈 인자를 허용하고 싶지 않으면 if constexpr으로 실제 마지막 호출을 생성하지 않는 대안을 쓸 수 있다.	(7.4, p.176-177 / PDF p.173-174)

번호	문장·표 항목·원문 코드·수식	원문 위치

U07-069	빈 파라미터 팩 호출을 생성하지 않기 (원문 코드 발췌)
	template<typename T, typename ... Tail>
	void print(T head, Tail... tail)
	{
	cout << head << ' ';
	if constexpr(sizeof...(tail)> 0)
	print(tail...);
	}
		(7.4, p.177 / PDF p.174)

U07-070	가변 템플릿의 장점은 원하는 여러 타입을 받을 수 있다는 유연성이다.	(7.4, p.177 / PDF p.174)
U07-071	반면 재귀 구현을 고치기 어렵고 컴파일 비용이 커질 수 있으며 인터페이스 타입 검사에 복잡한 템플릿 프로그램이 필요할 수 있다.	(7.4, p.177 / PDF p.174)
U07-072	표준 라이브러리의 중요한 수단이지만 남용할 수 있다고 경고한다.	(7.4, p.177 / PDF p.174)
U07-073	C++17의 접힘 표현식은 파라미터 팩의 요소에 연산을 적용하는 일부 가변 템플릿 구현을 간단히 만든다. (v + ... + 0)은 오른쪽의 0에서 시작하는 괄호 묶음으로, (0 + ... + v)는 왼쪽의 0에서 시작하는 괄호 묶음으로 설명된다.	(7.4.1, p.177-180 / PDF p.174-177)

U07-074	오른쪽 접힘과 원문 결과 (원문 코드 발췌)
	template<Number... T>
	int sum(T... v)
	{
	return (v + ... + 0); // v의 모든 요소를 0부터 더함
	}
	
	int x = sum(1, 2, 3, 4, 5); // x는 15가 됨
	int y = sum('a', 2.4, x); // y는 114가 됨(2.4는 소수점 아래는 버려지고 'a'의 값은 97)
		(7.4.1, p.178 / PDF p.175)

번호	문장·표 항목·원문 코드·수식	원문 위치

U07-075	왼쪽 접힘 (원문 코드 발췌)
	template<typename ... T>
	int sum2(T... v)
	{
	return (0 + ... + v); // v의 모든 요소를 0에 더함
	}
		(7.4.1, p.178 / PDF p.175)

U07-076	접힘은 합산뿐 아니라 <<를 통한 모든 인자의 출력에도 쓸 수 있다.	(7.4.1, p.179-180 / PDF p.176-177)
U07-077	원문 to_vector는 여러 인자를 공통 요소 타입의 벡터에 담아 이후 다루기 쉽게 하는 예다.	(7.4.1, p.179-180 / PDF p.176-177)
U07-078	접힘의 일반적인 아이디어는 accumulate에서도 사용되지만, 원문은 당시 C++의 접힘 표현식은 가변 템플릿 구현을 간단히 하는 용도로 제한된다고 설명한다.	(7.4.1, p.179-180 / PDF p.176-177)
U07-079	가변 템플릿의 중요한 용도는 인터페이스에서 받은 인자를 다른 인터페이스로 그대로 전달하는 것이다.	(7.4.2, p.180 / PDF p.177)
U07-080	원문 InputChannel은 실제 전송 방식을 Transport로 파라미터화하고 생성자 인자를 forward()로 Transport 생성자에 전달한다.	(7.4.2, p.180 / PDF p.177)
U07-081	작성자가 특정 Transport에 필요한 인자의 종류를 몰라도 객체를 생성할 수 있다.	(7.4.2, p.180 / PDF p.177)
U07-082	일반성과 낮은 실행 부담이 중요한 기본 라이브러리에서 널리 쓴다고 설명한다.	(7.4.2, p.180 / PDF p.177)
U07-083	컨셉을 사용하면 먼저 템플릿 인자가 컨셉을 만족하는지 확인한다.	(7.5, p.181 / PDF p.178)
U07-084	이때 확인할 수 없는 항목은 실제 코드를 생성하는 인스턴스화 시간까지 미룬다.	(7.5, p.181 / PDF p.178)
U07-085	컨셉이 없는 템플릿은 타입 검사를 인스턴스화 시점에 수행하므로 오류가 늦게 발견되고 여러 곳의 정보가 합쳐진 진단이 쓸모없어질 수 있다.	(7.5, p.181 / PDF p.178)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-086	원문은 이처럼 템플릿 인자에 필요한 연산이 있는지 뒤늦게 확인하는 특성을 컴파일 시간 덕 타이핑과 연결한다.	(7.5, p.181 / PDF p.178)
U07-087	일반적인 객체가 타입에 따라 연산을 갖는 것과 달리 템플릿 계산의 많은 부분은 객체보다 값에 관한 것이며, constexpr 함수의 지역 변수는 예외로 든다.	(7.5, p.181 / PDF p.178)
U07-088	제한되지 않은 템플릿은 사용하는 스코프에 선언뿐 아니라 정의가 있어야 한다.	(7.5, p.181 / PDF p.178)
U07-089	그래서 보통 정의는 .cpp보다 헤더에 놓인다.	(7.5, p.181 / PDF p.178)
U07-090	원문은 모듈을 사용하면 일반 함수와 비슷하게 코드를 정리할 수 있고 텍스트 포함 문제로부터 보호할 수 있다고 설명한다.	(7.5, p.181 / PDF p.178)
U07-091	템플릿 인자의 요구 개념을 세심하게 고려하고 컨셉을 설계 도구로 사용한다.	(7.6, p.182 / PDF p.179)
U07-092	의미 있는 동작을 내포하는 컨셉을 사용하고 요구 연산을 완전하게 명시한다.	(7.6, p.182 / PDF p.179)
U07-093	구체 구현을 디버그하고 측정한 뒤 일반화한다.	(7.6, p.182 / PDF p.179)
U07-094	같은 타입의 인자 목록에는 초깃값 목록을 사용하고 가변 템플릿을 남용하지 않는다.	(7.6, p.182 / PDF p.179)
U07-095	템플릿의 정의가 사용되는 스코프에 있어야 한다.	(7.6, p.182 / PDF p.179)
U07-096	컨셉 사용	(원문이 대비한 검사 시점, p.181 / PDF p.178)
U07-097	템플릿 인자의 컨셉 만족 여부	(원문이 대비한 검사 시점, p.181 / PDF p.178)
U07-098	그 시점에 확인할 수 없는 항목은 인스턴스화 때 검사	(원문이 대비한 검사 시점, p.181 / PDF p.178)
U07-099	컨셉 미사용	(원문이 대비한 검사 시점, p.181 / PDF p.178)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-100	컨셉에 의한 인터페이스 검사 없음	(원문이 대비한 검사 시점, p.181 / PDF p.178)
U07-101	타입 검사가 인스턴스화 시간까지 지연	(원문이 대비한 검사 시점, p.181 / PDF p.178)
U07-102	requires 표현식은 주어진 문장이 유효한 코드면 true, 그렇지 않으면 false로 평가되는 술어다.	(7.2.3, p.169 / PDF p.166)
U07-103	제네릭 프로그래밍은 서로 다른 데이터 표현에 적용할 제네릭 알고리즘을 만들기 위해 구체적이고 효율적인 알고리즘을 추상화하는 아이디어에 주안점을 둔다.	(7.3, p.172 / PDF p.169)
U07-104	Regular 타입은 기본 생성자로 생성하고 생성자·대입으로 독립적이고 같은 값인 복사본을 만들며 ==와 !=로 비교할 수 있고, 과도하게 영리한 프로그래밍 트릭으로 기술적 문제를 겪지 않는다.	(7.3.1, p.173 / PDF p.170)
U07-105	리프팅은 동일한 성능을 유지하면서 구체적인 코드를 일반화하는 과정이다.	(7.3.2, p.175 / PDF p.172)
U07-106	...으로 선언된 파라미터를 파라미터 팩이라고 한다.	(7.4, p.176 / PDF p.173)
U07-107	템플릿 인자에 가정하는 개념을 세심히 고려하고 컨셉을 설계 도구로 사용한다.	(7.6, p.182 / PDF p.179)
U07-108	템플릿을 설계할 때 우선 구체 타입을 바탕으로 구현하고 디버깅과 측정을 거친다.	(7.6, p.182 / PDF p.179)
U07-109	의미 있는 동작을 내포하지 않는 컨셉은 피하고, 컨셉에 요구되는 연산의 집합을 완벽히 명시한다.	(7.6, p.182 / PDF p.179)
U07-110	동일한 타입의 인자 목록에는 가변 템플릿 대신 초깃값 목록을 사용하고, 사용하는 곳에는 템플릿 정의가 존재해야 한다.	(7.6, p.182 / PDF p.179)
U07-111	컨셉을 사용하면 먼저 템플릿 인자의 컨셉 만족 여부를 검사하고, 그 시점에 확인할 수 없는 것은 인스턴스화 시간까지 미룬다.	(7.5, p.181 / PDF p.178)
U07-112	컨셉 사용	(7.5, p.181 / PDF p.178)
U07-113	컨셉 미사용	(7.5, p.181 / PDF p.178)

번호	문장·표 항목·원문 코드·수식	원문 위치
U07-114	컨셉을 사용하기 이전 코드에서는 모든 타입 검사를 인스턴스화 시간에 수행한다.	(7.5, p.181 / PDF p.178)

제8장. 라이브러리의 개요

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-001	C++ 표준 라이브러리는 가장 널리 사용하는 기본 자료 구조와 그 자료 구조에서 동작하는 알고리즘을 제공한다.	(8.2, p.186 / PDF p.183)
U08-002	표준 라이브러리는 std 네임스페이스에 정의되므로, 기능을 사용할 때 std:: 접두사를 사용할 수 있다.	(8.3, p.186 / PDF p.183)
U08-003	실제 프로그램은 기본 언어 기능만으로 작성하기보다 라이브러리를 토대로 구성된다.	(8.1, p.183-184 / PDF p.180-181)
U08-004	좋은 라이브러리를 이용하면 기본 언어만으로 작성하기 어려운 작업을 쉽게 만들 수 있다.	(8.1, p.183-184 / PDF p.180-181)
U08-005	이 책은 모든 세부를 완전히 설명하기보다 표준 라이브러리의 유용한 기능과 일반적인 사용법을 익히는 데 초점을 둔다.	(8.1, p.183-184 / PDF p.180-181)
U08-006	표준 라이브러리에는 설계·구현·유지 보수·확장을 위한 많은 노력이 들어간다.	(8.1, p.183-184 / PDF p.180-181)
U08-007	원문은 직접 코드를 다시 만드는 대신 이 기능을 사용하라고 권한다.	(8.1, p.183-184 / PDF p.180-181)
U08-008	한편 GUI·웹·데이터베이스 인터페이스나 산업별 기반 라이브러리도 있지만, 이 책은 표준에서 정의한 C++와 이식성 있는 예제에 집중하므로 이러한 외부 시스템은 다루지 않는다.	(8.1, p.183-184 / PDF p.180-181)
U08-009	표준 라이브러리는 널리 쓰는 기본 자료 구조와 그 위에서 동작하는 알고리즘을 일반적인 형태로 제공한다.	(8.2, p.184-186 / PDF p.181-183)
U08-010	기본적인 사용 예는 작업 자체의 복잡성에 비해 쉽게 배울 수 있고, 단순한 대안에 비해 큰 추가 부담이 없으며, 초보자와 전문가 모두에게 유용해야 한다는 선정 기준을 설명한다.	(8.2, p.184-186 / PDF p.181-183)
U08-011	언어 실행 지원과 C 표준 라이브러리: 메모리 할당, 실행 시간 타입 정보, 타입 시스템 위배를 줄이기 위한 일부 수정.	(8.2, p.184-186 / PDF p.181-183)
U08-012	문자열·문자열 뷰·정규 표현식: 여러 문자 세트, 지역화, 읽기 전용 부분 문자열, 패턴 매칭.	(8.2, p.184-186 / PDF p.181-183)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-013	입출력·파일 시스템: 타입·스트림·버퍼링·로케일을 확장할 수 있는 입출력과 이식성 있는 파일 조작.	(8.2, p.184-186 / PDF p.181-183)
U08-014	컨테이너·알고리즘: vector·map과 find·sort·merge 등의 확장 가능한 STL 프레임워크.	(8.2, p.184-186 / PDF p.181-183)
U08-015	수치·동시성·병렬 계산: 수학 함수, 복소수, 수치 벡터, 난수와 thread·mutex, 일부 병렬 알고리즘.	(8.2, p.184-186 / PDF p.181-183)
U08-016	일반적 유틸리티: 타입 특성, pair·variant·optional, 시간, 스마트 포인터와 자원 관리, array·bitset·tuple, 단위를 나타내는 접미사.	(8.2, p.184-186 / PDF p.181-183)
U08-017	표준 기능은 표준 헤더가 제공하며 std 네임스페이스에 정의된다.	(8.3, p.186-188 / PDF p.183-185)
U08-018	사용하려는 기능의 헤더를 포함하고 std:: 등의 방법으로 이름에 접근해야 한다.	(8.3, p.186-188 / PDF p.183-185)
U08-019	원문의 많은 예제는 단순하게 보이도록 헤더와 std::를 생략하므로, 그러한 코드 조각이 독립적인 완전한 프로그램이라는 뜻은 아니다.	(8.3, p.186-188 / PDF p.183-185)
U08-020	기능을 제공하는 표준 헤더 (원문 코드 발췌) #include<string> #include<list>	(8.3, p.186 / PDF p.183)
U08-021	std 이름을 직접 한정하기 (원문 코드 발췌) std::string sheep {"Four legs Good; two legs Baaad!"};	(8.3, p.186 / PDF p.183)
U08-022	using namespace std를 쓰면 std:: 없이 이름을 사용할 수 있다.	(8.3, p.187 / PDF p.184)
U08-023	하지만 한 네임스페이스의 모든 이름을 전역으로 가져오는 것은 일반적으로 좋지 않다고 명시한다.	(8.3, p.187 / PDF p.184)
U08-024	책에서는 표준 라이브러리의 사용법 학습을 위해 이 방식을 쓰기도 한다.	(8.3, p.187 / PDF p.184)
U08-025	알고리즘·컨테이너	(8.3, p.186-188 / PDF p.183-185)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-026	<algorithm>, <array>, <forward_list>, <list>, <map>, <set>, <unordered_map>, <vector>	(8.3, p.186-188 / PDF p.183-185)
U08-027	문자·입출력	(8.3, p.186-188 / PDF p.183-185)
U08-028	<string>, <regex>, <ios>, <iostream>, <fstream>, <sstream>	(8.3, p.186-188 / PDF p.183-185)
U08-029	수치·시간·동시성	(8.3, p.186-188 / PDF p.183-185)
U08-030	<cmath>, <complex>, <random>, <chrono>, <thread>, <future>	(8.3, p.186-188 / PDF p.183-185)
U08-031	자원·지원	(8.3, p.186-188 / PDF p.183-185)
U08-032	<memory>, <stdexcept>, <utility>, <variant>, <filesystem>	(8.3, p.186-188 / PDF p.183-185)
U08-033	C 라이브러리의 <stdlib.h> 같은 헤더도 제공한다.	(8.3, p.188 / PDF p.185)
U08-034	이름 앞에 c를 붙이고 .h를 제거한 <cstdlib> 같은 버전은 이름을 std 네임스페이스에 선언한다.	(8.3, p.188 / PDF p.185)
U08-035	헤더 목록은 선별된 일부이며 표준 라이브러리 전체 목록이 아니다.	(8.3, p.188 / PDF p.185)
U08-036	라이브러리를 활용하고, 선택할 수 있다면 표준 라이브러리를 사용한다.	(8.4, p.188 / PDF p.185)
U08-037	표준 라이브러리가 모든 문제를 해결하는 것은 아니다.	(8.4, p.188 / PDF p.185)
U08-038	필요한 기능을 제공하는 헤더를 포함한다.	(8.4, p.188 / PDF p.185)
U08-039	표준 라이브러리 기능은 std 네임스페이스에 정의된다.	(8.4, p.188 / PDF p.185)
U08-040	컨테이너와 find(), sort(), merge() 등을 비롯한 알고리즘 프레임워크를 관례적으로 STL이라고 한다.	(8.2, p.185 / PDF p.182)
U08-041	표준 라이브러리의 모든 기능은 표준 헤더에서 제공된다.	(8.3, p.186 / PDF p.183)

번호	문장·표 항목·원문 코드·수식	원문 위치
U08-042	라이브러리를 활용하고 선택권이 있다면 표준 라이브러리를 사용한다.	(8.4, p.188 / PDF p.185)
U08-043	표준 라이브러리가 만병통치약은 아니다.	(8.4, p.188 / PDF p.185)
U08-044	원하는 기능을 제공하는 헤더를 #include해야 하며, 표준 라이브러리 기능은 std 네임스페이스에 정의된다.	(8.4, p.188 / PDF p.185)
U08-045	〈string〉	(8.3, p.186 / PDF p.183)
U08-046	〈string〉 헤더를 이용하면 표준 string을 사용할 수 있다.	(8.3, p.186 / PDF p.183)
U08-047	〈list〉	(8.3, p.186 / PDF p.183)
U08-048	〈list〉 헤더를 이용하면 표준 list를 사용할 수 있다.	(8.3, p.186 / PDF p.183)

제9장. 문자열과 정규 표현식

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-001	string은 문자 시퀀스를 저장하고 조작하는 Regular 타입이다.	(9.2, p.190,192 / PDF p.187,189)
U09-002	짧은 문자열 최적화는 짧은 값을 객체 자체에 두고 긴 값에만 자유 저장소를 이용한다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-003	텍스트 조작을 위해 표준 라이브러리는 문자 시퀀스를 저장·조작하는 string, 저장 방식에 독립적으로 문자를 읽는 string_view, 패턴을 기술하고 찾는 regex를 제공한다.	(9.1, p.189-190 / PDF p.186-187)
U09-004	string과 regex는 여러 문자 타입에 적용할 수 있으며 원문은 유니코드를 그 예로 든다.	(9.1, p.189-190 / PDF p.186-187)
U09-005	string은 문자 시퀀스를 저장하고 조작하는 Regular 타입이다. +는 문자열 연결을 뜻하며 string·문자열 리터럴·C 스타일 문자열·문자 하나를 함께 연결할 수 있다.	(9.2, p.190-192 / PDF p.187-189)
U09-006	끝에 추가하는 경우 +=는 의도를 명료하게 나타내고 간결하며 때로는 더 효율적이어서 원문이 권장한다.	(9.2, p.190-192 / PDF p.187-189)
U09-007	이동 생성자가 있으므로 긴 string도 값으로 효율적으로 반환할 수 있다.	(9.2, p.190-192 / PDF p.187-189)
U09-008	string은 변경할 수 있고 대입·인덱싱·부분 문자열 연산을 제공한다.	(9.2, p.191 / PDF p.188)
U09-009	substr()은 지정한 위치부터 길이만큼 별도 string을 복사한다.	(9.2, p.191 / PDF p.188)
U09-010	replace()는 지정한 부분을 교체하며, 원래 부분과 새 문자열의 길이가 같을 필요는 없다.	(9.2, p.191 / PDF p.188)

번호	문장·표 항목·원문 코드·수식	원문 위치

U09-011	부분 문자열 복사와 교체 (원문 코드 발췌)
	string name = "Niels Stroustrup";
	
	void m3()
	{
	string s = name.substr(6,10); // s = "Stroustrup"
	name.replace(0,5,"nicholas"); // name은 "nicholas Stroustrup"
	name[0] = toupper(name[0]); // name은 "Nicholas Stroustrup"
	}
		(9.2, p.191 / PDF p.188)

U09-012	비교에는 ==와 !=, 사전 순서 비교에는 <·<=·>·>=를 사용한다.	(9.2, p.191-192 / PDF p.188-189)
U09-013	[]와 at()을 이용한 접근, 반복자 순회, 입력과 스트리밍도 제공한다.	(9.2, p.191-192 / PDF p.188-189)
U09-014	C 스타일의 0으로 끝나는 문자 배열이 필요할 때는 c_str()로 읽기 전용 문자 포인터를 얻는다.	(9.2, p.191-192 / PDF p.188-189)
U09-015	원문은 필요할 때에만 이 표현을 사용하라고 조언한다.	(9.2, p.191-192 / PDF p.188-189)
U09-016	문자열 리터럴을 std::string으로 사용하려면 접미사 s를 쓴다.	(9.2, p.192 / PDF p.189)
U09-017	접미사가 없는 리터럴과의 타입 차이를 주의한다.	(9.2, p.192 / PDF p.189)
U09-018	원문은 std::literals::string_literals 네임스페이스가 필요하다고 설명한다.	(9.2, p.192 / PDF p.189)
U09-019	일반적으로 사용하는 string은 직접 처음 만든 구현보다 표준 구현을 선택하라고 설명한다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-020	흔히 사용하는 짧은 문자열 최적화는 짧은 값을 객체 자체에 두고 긴 값에만 자유 저장소를 이용한다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-021	문자열 길이가 변하면 표현도 적절히 바뀌며, 짧다고 보는 길이는 구현에 따라 달라진다.	(9.2.1, p.193-194 / PDF p.190-191)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-022	실제 성능은 실행 환경의 영향을 받는다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-023	특히 다중 스레드 환경의 메모리 할당 비용과 서로 다른 길이 문자열에 의한 메모리 단편화는 짧은 문자열 최적화를 사용하는 이유다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-024	문자 타입을 일반화한 basic_string<Char>가 바탕이며, string은 basic_string<char>의 별칭이다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-025	원문 Jchar 예처럼 다른 문자 타입에도 일반적인 문자열 연산을 적용할 수 있다.	(9.2.1, p.193-194 / PDF p.190-191)
U09-026	string_view는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있다.	(9.3, p.194-196 / PDF p.191-193)
U09-027	문자는 string·C 스타일 문자열 등 서로 다른 방식으로 저장될 수 있다.	(9.3, p.194-196 / PDF p.191-193)
U09-028	뷰 자체는 문자를 소유하지 않으므로 포인터·참조 및 STL 반복자와 유사하다.	(9.3, p.194-196 / PDF p.191-193)

U09-029	서로 다른 저장 형태를 받는 문자열 뷰 (원문 코드 발췌)
	string cat(string_view sv1, string_view sv2)
	{
	string res(sv1.length()+sv2.length());
	char* p = &res[0];
	for (char c : sv1) // 복사를 수행하는 한 가지 방법
	*p++ = c;
	copy(sv2.begin(),sv2.end(),p); // 다른 방법
	return res;
	}
		(9.3, p.195 / PDF p.192)

U09-030	const string&를 받는 compose()와 비교해 원문 cat()은 다양한 저장 방식을 받을 수 있고, C 스타일 문자열에서 임시 string을 만들지 않으며, 부분 문자열을 쉽게 전달할 수 있다.	(9.3, p.195-196 / PDF p.192-193)
U09-031	접미사 sv를 쓰는 문자열 뷰 리터럴은 길이를 컴파일 시간에 계산한다.	(9.3, p.195-196 / PDF p.192-193)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-032	원문은 std::literals::string_view_literals를 지정한다.	(9.3, p.195-196 / PDF p.192-193)

U09-033	지역 문자열을 가리키는 뷰 반환의 위험 (원문 코드 발췌)
	string_view bad()
	{
	string s = "Once upon a time";
	return {&s[5],4}; // 위험: 지역 변수를 가리키는 포인터 반환
	}
		(9.3, p.196 / PDF p.193)

U09-034	뷰를 반환할 때는 실제로 무엇을 가리키는지 수명을 확인해야 한다.	(9.3, p.196 / PDF p.193)
U09-035	string_view는 읽기 전용이므로 문자를 소문자로 바꾸는 함수의 쓰기 인터페이스로 사용할 수 없다.	(9.3, p.196 / PDF p.193)
U09-036	원문은 이런 용도에 gsl::span 또는 gsl::string_span을 연결한다.	(9.3, p.196 / PDF p.193)
U09-037	구간 밖 접근 결과를 예측해서는 안 되며, 검사가 필요하면 out_of_range를 던지는 at()이나 gsl::string_span을 사용하도록 설명한다.	(9.3, p.196 / PDF p.193)
U09-038	정규 표현식은 텍스트 안의 패턴을 간단히 기술하고 찾는 도구다.	(9.4, p.197-198 / PDF p.194-195)
U09-039	<regex>는 regex 클래스와 검색·매칭·치환·반복자를 제공한다.	(9.4, p.197-198 / PDF p.194-195)
U09-040	로우 문자열 리터럴은 백슬래시와 큰따옴표를 직접 표현할 수 있어 정규 표현식을 적을 때 편리하다.	(9.4, p.197-198 / PDF p.194-195)
U09-041	일반 문자열에서는 백슬래시를 두 번 적어야 한다.	(9.4, p.197-198 / PDF p.194-195)
U09-042	로우 문자열로 기술한 원문 우편번호 패턴 (원문 코드 발췌) regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 패턴: XXddddd-dddd와 그 변종	(9.4, p.197 / PDF p.194)
U09-043	regex_match()	(9.4, p.197-198 / PDF p.194-195)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-044	입력 전체를 정규 표현식과 매칭	(9.4, p.197-198 / PDF p.194-195)
U09-045	regex_search()	(9.4, p.197-198 / PDF p.194-195)
U09-046	입력 안에서 매칭되는 부분 검색	(9.4, p.197-198 / PDF p.194-195)
U09-047	regex_replace()	(9.4, p.197-198 / PDF p.194-195)
U09-048	매칭되는 부분을 찾아 교체	(9.4, p.197-198 / PDF p.194-195)
U09-049	regex_iterator	(9.4, p.197-198 / PDF p.194-195)
U09-050	매치와 부분 매치를 찾으며 순회	(9.4, p.197-198 / PDF p.194-195)
U09-051	regex_token_iterator	(9.4, p.197-198 / PDF p.194-195)
U09-052	매치되지 않는 부분 순회	(9.4, p.197-198 / PDF p.194-195)
U09-053	regex_search(line,matches,pat)은 line에서 pat에 맞는 부분을 찾아 matches에 저장하고, 없으면 false를 반환한다.	(9.4.1, p.198-199 / PDF p.195-196)
U09-054	smatch는 string의 매치 결과들을 저장하는 컨테이너다.	(9.4.1, p.198-199 / PDF p.195-196)
U09-055	matches[0]은 전체 매치이고 이후 항목은 괄호로 지정한 부분 패턴에 대응한다.	(9.4.1, p.198-199 / PDF p.195-196)
U09-056	선택적인 부분 패턴을 이용할 때는 항목의 존재와 matched 상태를 확인한다.	(9.4.1, p.198-199 / PDF p.195-196)

번호	문장·표 항목·원문 코드·수식	원문 위치

U09-057	파일에서 전체 매치와 부분 매치 찾기 (원문 코드 발췌)
	void use()
	{
	ifstream in("file.txt"); // 입력 파일
	if (!in) // 파일이 열렸는지 확인
	cerr << "파일 없음\n";
	
	regex pat {R"(\w{2}\s*\d{5}(-\d{4})*)"}; // 미국 우편번호 코드
	
	int lineno = 0;
	for (string line; getline(in,line);) {
	++lineno;
	smatch matches; // 매치된 문자열 저장
	if (regex_search(line , matches, pat)) {
	cout << lineno << ": " << matches[0] << '\n'; // 전체 매치
	if (1<matches.size() && matches[1].matched) // 부분 패턴이 존재하고
	// 그 부분 패턴이 매치됐으면
	cout << "\t: " << matches[1] << '\n'; // 부분 매치
	}
	}
	}
		(9.4.1, p.198-199 / PDF p.195-196)

U09-058	여러 행에 걸친 패턴에도 개행 문자를 포함할 수 있지만 그런 입력을 한 행씩만 읽어서는 안 된다.	(9.4.1, p.199 / PDF p.196)
U09-059	regex는 패턴을 효율적으로 실행할 수 있는 상태 기계로 실행 시간에 컴파일한다고 설명한다.	(9.4.1, p.199 / PDF p.196)
U09-060	원문의 기본 표기법은 ECMAScript의 ECMA 표준을 따른다.	(9.4.2, p.199-200 / PDF p.196-197)
U09-061	특수 문자는 보통 문자와 다른 역할을 갖는다.	(9.4.2, p.199-200 / PDF p.196-197)
U09-062	괄호는 부분 패턴을 만들고, 대괄호는 문자 클래스를, 중괄호는 반복 횟수를 지정한다.	(9.4.2, p.199-200 / PDF p.196-197)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-063	점은 문자 하나, 세로줄은 대안, ^와 $는 각각 행의 시작과 끝을 나타낸다. ^는 문자 클래스의 반전에도 쓰인다.	(9.4.2, p.199-200 / PDF p.196-197)
U09-064	{n}	(9.4.2, p.199-206 / PDF p.196-203)
U09-065	정확히 n번	(9.4.2, p.199-206 / PDF p.196-203)
U09-066	{n,}	(9.4.2, p.199-206 / PDF p.196-203)
U09-067	n번 이상	(9.4.2, p.199-206 / PDF p.196-203)
U09-068	{n,m}	(9.4.2, p.199-206 / PDF p.196-203)
U09-069	n번 이상 m번 이하	(9.4.2, p.199-206 / PDF p.196-203)
U09-070	*	(9.4.2, p.199-206 / PDF p.196-203)
U09-071	0번 이상	(9.4.2, p.199-206 / PDF p.196-203)
U09-072	+	(9.4.2, p.199-206 / PDF p.196-203)
U09-073	1번 이상	(9.4.2, p.199-206 / PDF p.196-203)
U09-074	?	(9.4.2, p.199-206 / PDF p.196-203)
U09-075	0번 또는 1번	(9.4.2, p.199-206 / PDF p.196-203)
U09-076	접미 반복이 없으면 패턴은 기본적으로 한 번 등장한다.	(9.4.2, p.201-202 / PDF p.198-199)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-077	반복 표기 뒤의 ?는 가장 짧은 매치를 고르는 지연 또는 비욕심쟁이 매칭을 뜻한다.	(9.4.2, p.201-202 / PDF p.198-199)
U09-078	기본의 욕심쟁이 매칭은 가장 긴 매치를 찾는다.	(9.4.2, p.201-202 / PDF p.198-199)
U09-079	원문 abababd에서 (ab)+는 전체를, (ab)+?는 첫 ab를 매치한다고 설명한다.	(9.4.2, p.201-202 / PDF p.198-199)
U09-080	문자 클래스에는 alnum·alpha·blank·cntrl·digit·graph·lower·print·punct·space·upper·xdigit 등 이름이 있다.	(9.4.2, p.202-203 / PDF p.199-200)
U09-081	이름을 [: :]로 감싼 뒤 문자 클래스의 [ ] 안에서 사용한다.	(9.4.2, p.202-203 / PDF p.199-200)
U09-082	숫자·공백·단어 문자의 축약어는 각각 \d·\s·\w이고, \D·\S·\W는 각각 그 여집합이다.	(9.4.2, p.202-203 / PDF p.199-200)
U09-083	원문은 비표준인 \l·\u·\L·\U도 구분하고 이식성을 위해 이름을 사용하라고 조언한다.	(9.4.2, p.202-203 / PDF p.199-200)
U09-084	식별자 예는 첫 문자가 밑줄 또는 알파벳이고, 이후가 밑줄·알파벳·숫자의 시퀀스라는 조건을 설명한다.	(9.4.2, p.203-204 / PDF p.200-201)
U09-085	alpha와 alnum에 밑줄이 포함되지 않는다는 점, [:alpha:]를 문자 클래스 바깥에 적으면 의도가 달라진다는 점을 잘못된 패턴과 비교한다.	(9.4.2, p.203-204 / PDF p.200-201)
U09-086	입력 전체를 검사하는 함수에는 regex_match()를 사용한다.	(9.4.2, p.203-204 / PDF p.200-201)
U09-087	group은 괄호로 구분되어 sub_match에 저장될 수 있다.	(9.4.2, p.205-206 / PDF p.202-203)
U09-088	그룹을 묶되 부분 매치를 저장할 필요가 없으면 (?: 형태를 사용한다.	(9.4.2, p.205-206 / PDF p.202-203)
U09-089	원문은 그룹 개수·중첩·반복에 따른 저장과 \1로 첫 그룹을 다시 참조하는 태그 예를 비교한다.	(9.4.2, p.205-206 / PDF p.202-203)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-090	태그 사이를 .*로 탐욕적으로 잡으면 첫 <부터 마지막 >까지 걸칠 수 있으므로, 의도한 짧은 부분을 찾는 데 .*?를 사용한다고 설명한다.	(9.4.2, p.205-206 / PDF p.202-203)
U09-091	[원문 확인 필요] 일부 식별자 검사 코드와 패턴에서 본문 설명과 기호의 대응을 다시 확인해야 한다.	(9.4.2, p.200,204-205 / PDF p.197,201-202)
U09-092	이 교재에서는 판독에 의심이 있는 코드의 기호를 임의로 고쳐 재작성하지 않고, 해당 페이지에서 명시된 규칙을 중심으로 정리했다.	(9.4.2, p.200,204-205 / PDF p.197,201-202)
U09-093	regex_iterator는 시퀀스에서 패턴을 차례로 찾아 순회한다.	(9.4.3, p.206-207 / PDF p.203-204)
U09-094	string에 대해 사용하는 원문 sregex_iterator 예는 공백 뒤의 단어를 부분 매치로 출력한다.	(9.4.3, p.206-207 / PDF p.203-204)

U09-095	정규 표현식 반복자로 단어 찾기 (원문 코드 발췌)
	void test()
	{
	string input = "aa as; asd ++e^asdf asdfg";
	regex pat {R"(\s+(\w+))"};
	for (sregex_iterator p(input.begin(),input.end(),pat);
	p!=sregex_iterator{}; ++p)
	cout << (*p)[1] << '\n';
	}
		(9.4.3, p.206 / PDF p.203)

U09-096	원문의 첫 패턴 출력은 as·asd·asdfg다.	(9.4.3, p.206-207 / PDF p.203-204)
U09-097	앞 공백이 없는 aa는 빠진다.	(9.4.3, p.206-207 / PDF p.203-204)
U09-098	패턴을 R"((\w+))"로 단순화한 다음 페이지의 출력은 aa·as·asd·e·asdf·asdfg다.	(9.4.3, p.206-207 / PDF p.203-204)
U09-099	이 결과는 원문에 제시된 목록만 옮긴 것이다.	(9.4.3, p.206-207 / PDF p.203-204)
U09-100	regex_iterator는 양방향 반복자를 요구하므로 입력 반복자만 제공하는 istream을 직접 순회할 수 없다고 설명한다.	(9.4.3, p.206-207 / PDF p.203-204)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-101	쓰기는 지원하지 않고 끝 표시는 기본 생성한 regex_iterator{}를 쓴다.	(9.4.3, p.206-207 / PDF p.203-204)
U09-102	문자 시퀀스를 저장할 때 string을 사용하고 C 스타일 함수 대신 문자열 연산을 사용한다.	(9.5, p.207-208 / PDF p.204-205)
U09-103	부분 문자열의 복사는 substr(), 교체는 replace()로 표현한다.	(9.5, p.207-208 / PDF p.204-205)
U09-104	구간 검사가 필요할 때 at()을 사용한다.	(9.5, p.207-208 / PDF p.204-205)
U09-105	여러 저장 형태의 문자 시퀀스를 읽는 함수는 string_view를 사용하고 소유·수명 문제를 기억한다.	(9.5, p.207-208 / PDF p.204-205)
U09-106	문자열 리터럴의 s와 문자열 뷰 리터럴의 sv를 구별한다.	(9.5, p.207-208 / PDF p.204-205)
U09-107	패턴은 로우 문자열로 명료하게 표현하고 전체 매칭과 부분 검색을 구별한다.	(9.5, p.207-208 / PDF p.204-205)
U09-108	필요한 부분 패턴만 저장하고 욕심쟁이·지연 매칭의 차이를 확인한다.	(9.5, p.207-208 / PDF p.204-205)
U09-109	역할	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-110	문자 시퀀스 저장·조작	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-111	여러 저장 형태의 문자 시퀀스 읽기	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-112	소유	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)

번호	문장·표 항목·원문 코드·수식	원문 위치
U09-113	문자를 직접 관리	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-114	문자를 소유하지 않음	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-115	수정	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-116	인덱싱·replace 등으로 변경	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-117	읽기 전용	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-118	수명	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-119	자기 객체의 문자열	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-120	가리키는 문자 영역의 수명에 의존	(문자열 저장과 문자열 뷰, p.190-196 / PDF p.187-193)
U09-121	string_view는 연속된 문자 시퀀스를 나타내는 포인터와 길이의 쌍으로 볼 수 있으며, 문자를 소유하지 않는다.	(9.3, p.194-195 / PDF p.191-192)
U09-122	string	(9.2, p.190,192 / PDF p.187,189)
U09-123	string_view	(9.3, p.194-195 / PDF p.191-192)

제10장. 입력과 출력

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-001	입출력 스트림 라이브러리는 텍스트와 수치 값의 형식화·비형식화 및 버퍼링된 입출력을 지원한다.	(10.1, p.210-211 / PDF p.207-208)
U10-002	cout은 표준 출력, cerr은 에러 보고 스트림이다.	(10.2, p.211,213 / PDF p.208,210)
U10-003	ostream은 타입 있는 값을 문자 또는 바이트의 스트림으로 바꾸고, istream은 반대 방향으로 문자 스트림을 타입 있는 객체로 변환한다.	(10.1, p.210-211 / PDF p.207-208)
U10-004	이 연산은 타입 안전성과 타입 민감성을 갖고 사용자 정의 타입에도 확장할 수 있다.	(10.1, p.210-211 / PDF p.207-208)
U10-005	스트림은 표준 입력·출력, 파일, string 버퍼에 적용된다.	(10.1, p.210-211 / PDF p.207-208)
U10-006	소멸자가 자신이 소유한 버퍼와 파일 핸들 같은 자원을 해제하므로 RAII의 한 예다.	(10.1, p.210-211 / PDF p.207-208)
U10-007	이진 입출력·여러 문자 타입·로케일·고급 버퍼링도 지원하지만 여기서는 상세히 다루지 않는다고 범위를 정한다.	(10.1, p.210-211 / PDF p.207-208)
U10-008	<<는 ostream에 값을 내보내는 연산이다.	(10.2, p.211-213 / PDF p.208-210)
U10-009	cout은 표준 출력, cerr은 에러 보고 스트림이다.	(10.2, p.211-213 / PDF p.208-210)
U10-010	cout에 정수 10을 쓰면 기본적으로 정수의 바이트 표현 대신 문자 1과 0으로 변환하여 출력한다.	(10.2, p.211-213 / PDF p.208-210)
U10-011	한 출력 표현식의 결과를 다음 <<에 연결할 수 있어 관련 항목을 한 문장에 묶을 수 있다.	(10.2, p.211-213 / PDF p.208-210)
U10-012	문자 상수를 출력하면 문자가 출력되지만 char 값을 int로 변환하여 출력하면 정수 값이 출력된다.	(10.2, p.212-213 / PDF p.209-210)
U10-013	원문은 문자 a·정수에 저장한 b·문자 c를 출력하는 예에서, 사용한 ASCII 인코딩을 전제로 a98c가 나온다고 명시한다.	(10.2, p.212-213 / PDF p.209-210)
U10-014	이 결과를 모든 인코딩에 관한 주장으로 넓히지 않는다.	(10.2, p.212-213 / PDF p.209-210)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-015	>>는 istream에서 값을 가져오는 연산이며 cin은 표준 입력 스트림이다.	(10.3, p.213-215 / PDF p.210-212)
U10-016	오른쪽 피연산자의 타입이 어떤 입력을 받아 어디에 저장하는지를 결정한다.	(10.3, p.213-215 / PDF p.210-212)
U10-017	정수와 부동소수점 입력을 연결해서 읽을 수 있다.	(10.3, p.213-215 / PDF p.210-212)
U10-018	정수 입력은 숫자가 아닌 문자를 만나면 끝나고, >>는 기본적으로 앞 공백을 지나친다.	(10.3, p.213-215 / PDF p.210-212)
U10-019	string에 대한 >>는 공백에서 읽기를 끝내므로 Eric Bloodaxe를 입력해도 첫 단어 Eric만 읽는다.	(10.3, p.214-215 / PDF p.211-212)
U10-020	한 행 전체가 필요하면 getline()을 사용한다.	(10.3, p.214-215 / PDF p.211-212)
U10-021	행을 마친 개행 문자는 버리고 다음 행을 읽을 수 있는 상태가 된다.	(10.3, p.214-215 / PDF p.211-212)
U10-022	문자열은 입력에 따라 늘어나므로 최대 길이를 미리 계산할 필요가 없다고 설명한다.	(10.3, p.214-215 / PDF p.211-212)

U10-023	한 행 전체를 읽는 getline (원문 코드 발췌)
	void hello_line()
	{
	cout << "이름을 입력하세요\n";
	string str;
	getline(cin,str);
	cout << "Hello, " << str << "!\n";
	}
		(10.3, p.215 / PDF p.212)

U10-024	원문은 문자 단위 조작보다 형식화된 입출력이 대체로 오류 가능성이 낮고 효율적이며 코드도 짧다고 설명한다.	(10.3, p.215 / PDF p.212)
U10-025	istream은 메모리 관리와 구간 검사를 맡고, 메모리 안에서 형식화를 수행할 때는 문자열 스트림을 사용한다.	(10.3, p.215 / PDF p.212)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-026	스트림은 연산의 성공 여부를 나타내는 상태를 보유한다.	(10.4, p.215-217 / PDF p.212-214)
U10-027	is>>i는 is의 참조를 반환하고, 이를 조건으로 검사하면 다음 연산을 수행할 수 있는지를 확인한다.	(10.4, p.215-217 / PDF p.212-214)
U10-028	정수가 아닌 입력이나 입력 끝에서 반복을 멈추는 구조다.	(10.4, p.215-217 / PDF p.212-214)

U10-029	입력 상태를 루프 조건으로 사용 (원문 코드 발췌)
	vector<int> read_ints(istream& is)
	{
	vector<int> res;
	for (int i; is>>i;)
	res.push_back(i);
	return res;
	}
		(10.4, p.215-216 / PDF p.212-213)

U10-030	입출력 상태에는 형식화 정보·에러 상태·버퍼링 종류 등 작업에 필요한 정보가 들어간다.	(10.4, p.216 / PDF p.213)
U10-031	사용자는 에러를 알리거나 회복 가능한 실패를 무시하기 위해 상태를 설정할 수 있다.	(10.4, p.216 / PDF p.213)
U10-032	원문의 종료 표식 버전은 eof()면 정상 종료하고, 정수 입력 fail()이면 clear()로 상태를 리셋하고 unget()으로 되돌린 뒤 문자열 표식인지 확인한다.	(10.4, p.216 / PDF p.213)
U10-033	[원문 확인 필요] 종료 표식 버전은 입력 스트림 매개변수 is를 받지만 뒷부분의 문자열 읽기와 실패 상태 설정에는 cin을 사용한다.	(10.4, p.216-217 / PDF p.213-214)
U10-034	이 교재에서는 일반화된 다른 코드로 고치지 않고 원문 설명의 흐름과 해당 표기를 구분해 둔다.	(10.4, p.216-217 / PDF p.213-214)
U10-035	종료 표식 호출 예는 read_ints(cin,"stop")이다.	(10.4, p.216-217 / PDF p.213-214)
U10-036	사용자 정의 타입에도 의미 있는 텍스트 표현이 있다면 <<와 >>를 정의할 수 있다.	(10.5, p.217-219 / PDF p.214-216)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-037	ostream이나 istream 자체를 수정할 필요는 없다.	(10.5, p.217-219 / PDF p.214-216)
U10-038	Entry의 출력은 {"이름", 번호} 형식을 사용하고 스트림 참조를 반환한다.	(10.5, p.217-219 / PDF p.214-216)
U10-039	입력은 형식 검사와 에러 처리가 필요해 더 복잡하다.	(10.5, p.217-219 / PDF p.214-216)

번호	문장·표 항목·원문 코드·수식	원문 위치

U10-040	사용자 정의 타입의 입출력 연산 (1/2) (원문 코드 발췌)
	struct Entry {
	string name;
	int number;
	};
	
	ostream& operator<<(ostream& os, const Entry& e)
	{
	return os << "{\"" << e.name << "\", " << e.number << "}";
	}
	
	istream& operator>>(istream& is, Entry& e)
	// { "이름" ,번호 } 쌍 읽기. { " " , 와 } 로 형식화됨
	{
	char c, c2;
	if (is>>c && c=='{' && is>>c2 && c2=='"') { // { "로 시작
	string name; // string의 기본 값은 빈 문자열: ""
	while (is.get(c) && c!='"') // " 이전의 모든 문자는 이름으로 저장
	name+=c;
	
	if (is>>c && c==',') {
	int number = 0;
	if (is>>number>>c && c=='}') { // 번호와 } 읽기
	e = {name, number}; // 항목에 대입
	return is;
	}
	}
	}
	is.setstate(ios_base::failbit); // 스트림에 실패를 기록함
	return is;
		(10.5, p.217-219 / PDF p.214-216)

U10-041	사용자 정의 타입의 입출력 연산 (2/2) (원문 코드 발췌) }	(10.5, p.217-219 / PDF p.214-216)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-042	조건에서 is>>c를 검사하면 읽기의 성공 여부도 확인한다. >>는 앞 공백을 지나치지만 get(c)는 그렇지 않으므로, 이름 바깥 공백은 건너뛰고 따옴표 안 공백은 이름에 보존한다.	(10.5, p.218-219 / PDF p.215-216)
U10-043	잘못된 형식이면 failbit를 설정하여 호출 측의 스트림 조건에 실패를 전달한다.	(10.5, p.218-219 / PDF p.215-216)
U10-044	원문은 John Marwood Cleese와 Michael Edward Palin을 공백 포함 이름의 예로 제시한다.	(10.5, p.218-219 / PDF p.215-216)
U10-045	조정자는 입력·출력 형식을 제어한다.	(10.6, p.219-221 / PDF p.216-218)
U10-046	표준 조정자는 <ios>·<istream>·<ostream> 등에 있고 인자를 받는 조정자는 <iomanip>에 있다.	(10.6, p.219-221 / PDF p.216-218)
U10-047	정수의 기본 10진 출력 외에 hex와 oct로 진법을 지정할 수 있다.	(10.6, p.219-221 / PDF p.216-218)
U10-048	정수 출력 진법을 바꾸는 조정자 (원문 코드 발췌) cout << 1234 << ',' << hex << 1234 << ',' << oct << 1234 << '\n'; // 1234,4d2,2322 출력	(10.6, p.219 / PDF p.216)
U10-049	defaultfloat	(10.6, p.219-221 / PDF p.216-218)
U10-050	값을 잘 보존하도록 구현체가 스타일 선택	(10.6, p.219-221 / PDF p.216-218)
U10-051	최대 자릿수	(10.6, p.219-221 / PDF p.216-218)
U10-052	scientific	(10.6, p.219-221 / PDF p.216-218)
U10-053	소수점 앞 한 자리와 지수	(10.6, p.219-221 / PDF p.216-218)
U10-054	소수점 다음 최대 자릿수	(10.6, p.219-221 / PDF p.216-218)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-055	fixed	(10.6, p.219-221 / PDF p.216-218)
U10-056	정수부·소수점·소수 부분	(10.6, p.219-221 / PDF p.216-218)
U10-057	원문은 hexfloat를 이용한 16진 부동소수점 표기도 제시한다.	(10.6, p.220-221 / PDF p.217-218)
U10-058	부동소수점 값은 버림이 아닌 반올림으로 출력한다.	(10.6, p.220-221 / PDF p.217-218)
U10-059	precision()은 정수 값의 출력에 영향을 주지 않는다.	(10.6, p.220-221 / PDF p.217-218)
U10-060	precision과 부동소수점 형식 조정은 지속적이므로 이후 출력에도 영향을 주며, 다음 출력 한 번에만 적용된다고 가정해서는 안 된다.	(10.6, p.220-221 / PDF p.217-218)
U10-061	<fstream>의 ifstream은 읽기, ofstream은 쓰기, fstream은 읽기와 쓰기용이다.	(10.7, p.221-222 / PDF p.218-219)
U10-062	파일을 연 다음에는 스트림 상태를 확인하여 실제로 연결되었는지 확인해야 한다.	(10.7, p.221-222 / PDF p.218-219)
U10-063	성공한 파일 스트림은 일반적인 istream·ostream처럼 사용할 수 있다.	(10.7, p.221-222 / PDF p.218-219)

U10-064	파일 스트림을 연 뒤 상태 확인 (원문 코드 발췌)
	ofstream ofs {"target"}; // "o"는 "출력(output)"을 의미
	if (!ofs)
	error("'target'을 쓰기용으로 열 수 없음");
	
	ifstream ifs {"source"}; // "i"는 "입력(input)"을 의미
	if (!ifs)
	error("'source'를 읽기용으로 열 수 없음 ");
		(10.7, p.221 / PDF p.218)

U10-065	파일 스트림은 소유한 자원을 소멸자로 정리한다.	(10.7, p.221-222,231 / PDF p.218-219,228)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-066	원문의 조언은 파일 스트림을 복사하지 않는 것이다.	(10.7, p.221-222,231 / PDF p.218-219,228)
U10-067	세밀한 위치 제어나 열기 모드는 이 책의 범위를 벗어나며, 파일 이름과 파일 시스템 조작은 10.10으로 연결한다.	(10.7, p.221-222,231 / PDF p.218-219,228)
U10-068	<sstream>의 istringstream은 string에서 읽고 ostringstream은 string에 쓰며 stringstream은 둘 다 한다.	(10.8, p.222-223 / PDF p.219-220)
U10-069	ostringstream에 형식화된 출력을 수행하고 str()로 저장된 문자열을 얻는 예는 GUI 등에 표시할 문자열을 만드는 용도다.	(10.8, p.222-223 / PDF p.219-220)
U10-070	반대로 입력 문자열을 istringstream에 넣어 형식화된 입력을 수행할 수 있다.	(10.8, p.222-223 / PDF p.219-220)

U10-071	문자열 스트림으로 일반적인 타입 변환 (원문 코드 발췌)
	template<typename Target =string, typename Source =string>
	Target to(Source arg) // Source를 Target으로 변환
	{
	stringstream interpreter;
	Target result;
	
	if (!(interpreter << arg) // arg를 스트림에 출력
	|| !(interpreter >> result) // 스트림에서 result를 읽기
	|| !(interpreter >> std::ws).eof()) // 스트림에 남은 것이 있는가?
	throw runtime_error{"to<>() failed"};
	
	return result;
	}
		(10.8, p.223 / PDF p.220)

U10-072	to()는 Source를 스트림에 출력하고 Target을 읽는 두 작업을 결합하며, 변환 후 공백 이외의 내용이 남으면 실패로 처리한다.	(10.8, p.223 / PDF p.220)
U10-073	원문은 문자열로 표현 가능한 타입 사이의 일반적인 변환 사례로 설명한다.	(10.8, p.223 / PDF p.220)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-074	함수 템플릿 인자가 추론 가능하거나 기본 값이 있으면 생략할 수 있으므로, 기본 Target=string과 Source 추론을 통해 to(1.2)처럼 쓸 수 있다.	(10.8, p.223 / PDF p.220)
U10-075	C++ 표준 라이브러리는 printf·scanf 계열도 지원한다.	(10.9, p.223-224 / PDF p.220-221)
U10-076	원문은 대개 타입과 보안 면에서 안전하지 않고 입력을 안전하고 편리하게 구현하기 어려우며 사용자 정의 타입을 지원하지 않아 권장하지 않는다.	(10.9, p.223-224 / PDF p.220-221)
U10-077	C 스타일 입출력을 사용하지 않으면서 성능을 고려할 때 다음 호출을 제시한다.	(10.9, p.223-224 / PDF p.220-221)
U10-078	C 스타일 입출력과의 동기화 설정 (원문 코드 발췌) ios_base::sync_with_stdio(false); // 큰 오버헤드를 피할 수 있음	(10.9, p.224 / PDF p.221)
U10-079	<filesystem>은 시스템마다 다른 경로·파일 종류·권한·탐색 기능에 일관된 인터페이스를 제공한다.	(10.10, p.224-225 / PDF p.221-222)
U10-080	path는 운영체제의 기본 문자 집합과 관례를 다루는 경로 표현이다.	(10.10, p.224-225 / PDF p.221-222)
U10-081	명령행의 이름이나 문자열에서 만들고 파일 열기에 사용할 수 있다.	(10.10, p.224-225 / PDF p.221-222)
U10-082	실제 사용 전에는 유효성을 검사하지 않으며 무엇이 유효한지는 실행 시스템 관례에 달려 있다.	(10.10, p.224-225 / PDF p.221-222)
U10-083	파일 시스템은 다른 프로그램과 함께 사용된다.	(10.10, p.225 / PDF p.222)
U10-084	exists()로 파일 존재를 확인했어도 다음 is_regular_file() 호출 전에 파일이 없어질 수 있다.	(10.10, p.225 / PDF p.222)
U10-085	이처럼 두 연산 사이의 외부 상태 변화 가능성을 원문이 경고한다.	(10.10, p.225 / PDF p.222)

번호	문장·표 항목·원문 코드·수식	원문 위치

U10-086	디렉터리 순회와 파일 시스템 예외 (원문 코드 발췌)
	void print_directory(path p)
	try
	{
	if (is_directory(p)) {
	cout << p << ":\n";
	for (const directory_entry& x : directory_iterator{p})
	cout << " " << x.path() << '\n';
	}
	}
	catch (const filesystem_error& ex) {
	cerr << ex.what() << '\n';
	}
		(10.10, p.226 / PDF p.223)

U10-087	directory_entry는 한 항목이고 directory_iterator는 디렉터리 항목을 순회한다.	(10.10, p.226-227 / PDF p.223-224)
U10-088	하위 디렉터리까지 필요하면 recursive_directory_iterator를 사용한다.	(10.10, p.226-227 / PDF p.223-224)
U10-089	사전 순서 출력이 필요하면 원문은 path들을 vector에 담아 정렬하는 방법을 제시한다.	(10.10, p.226-227 / PDF p.223-224)
U10-090	순회가 자동으로 정렬된다고 설명하지 않는다.	(10.10, p.226-227 / PDF p.223-224)
U10-091	/= 와 +=	(10.10, p.224-230 / PDF p.221-227)
U10-092	파일 이름 구분자를 넣어 연결 / 구분자 없이 연결	(10.10, p.224-230 / PDF p.221-227)
U10-093	native(), string(), generic_string()	(10.10, p.224-230 / PDF p.221-227)
U10-094	네이티브 표현 또는 제네릭 표현의 문자열 얻기	(10.10, p.224-230 / PDF p.221-227)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-095	filename(), stem(), extension()	(10.10, p.224-230 / PDF p.221-227)
U10-096	파일 이름·꼬리·확장자 부분 다루기	(10.10, p.224-230 / PDF p.221-227)
U10-097	begin(), end()	(10.10, p.224-230 / PDF p.221-227)
U10-098	경로 구성 요소 시퀀스의 시작·끝	(10.10, p.224-230 / PDF p.221-227)
U10-099	==, !=, 비교 연산·스트림 입출력	(10.10, p.224-230 / PDF p.221-227)
U10-100	상등·사전 순서 비교 및 경로 입출력	(10.10, p.224-230 / PDF p.221-227)
U10-101	u8path(s)	(10.10, p.224-230 / PDF p.221-227)
U10-102	UTF-8로 인코딩된 s에서 경로 생성	(10.10, p.224-230 / PDF p.221-227)
U10-103	원문 표는 value_type이 POSIX에서 char, 윈도우에서 wchar_t라고 설명한다.	(10.10, p.227-228 / PDF p.224-225)
U10-104	string_type은 해당 문자 타입의 basic_string이며 const_iterator는 const 양방향 반복자, iterator는 그 별칭이다.	(10.10, p.227-228 / PDF p.224-225)
U10-105	[원문 확인 필요] extension()의 옮긴이 설명은 “.을 제외한 확장자”로 인쇄되어 있어 원문 표기를 그대로 알리고 임의로 수정하지 않는다.	(10.10, p.227-228 / PDF p.224-225)
U10-106	exists, file_size	(10.10, p.224-230 / PDF p.221-227)
U10-107	존재 여부·바이트 수 조회	(10.10, p.224-230 / PDF p.221-227)
U10-108	copy, copy_file	(10.10, p.224-230 / PDF p.221-227)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-109	파일 또는 디렉터리 복사 / 파일 내용 복사	(10.10, p.224-230 / PDF p.221-227)
U10-110	create_directory, create_directories	(10.10, p.224-230 / PDF p.221-227)
U10-111	상위 경로가 존재해야 하는 생성 / 중간 경로도 생성	(10.10, p.224-230 / PDF p.221-227)
U10-112	current_path 계열	(10.10, p.224-230 / PDF p.221-227)
U10-113	현재 작업 디렉터리 처리	(10.10, p.224-230 / PDF p.221-227)
U10-114	remove	(10.10, p.224-230 / PDF p.221-227)
U10-115	파일 또는 빈 디렉터리 삭제	(10.10, p.224-230 / PDF p.221-227)
U10-116	원문은 대부분의 연산에 실패하면 filesystem_error를 던지는 기본 버전과 error_code를 추가 인자로 받아 결과를 확인하는 버전이 있다고 설명한다.	(10.10, p.229 / PDF p.226)
U10-117	실패가 자주 일어나 예외를 던지는 일이 예외적 상황과 맞지 않는 경우에는 에러 코드를 사용하라고 조언한다.	(10.10, p.229 / PDF p.226)
U10-118	권한 등을 받는 세부 오버로드는 여기서 다루지 않는다.	(10.10, p.229 / PDF p.226)
U10-119	파일 종류는 is_block_file·is_character_file·is_directory·is_empty·is_fifo·is_other·is_regular_file·is_socket·is_symlink로 질의하고 status_known으로 상태를 아는지 확인한다.	(10.10, p.229-230 / PDF p.226-227)
U10-120	알 수 없는 종류는 기타로 분류한다.	(10.10, p.229-230 / PDF p.226-227)
U10-121	경로 이름·자연어·인코딩은 복잡하지만 이 추상화가 이식성과 단순함을 제공한다고 정리한다.	(10.10, p.229-230 / PDF p.226-227)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-122	일반 출력은 cout, 에러 보고는 cerr을 사용하고 <<와 >>를 연결해 의도를 명확하게 한다.	(10.11, p.230-232 / PDF p.227-229)
U10-123	형식이 잘못된 입력을 고려하고 회복 가능한 입력 실패는 스트림 상태로 처리한다.	(10.11, p.230-232 / PDF p.227-229)
U10-124	문자 수준 입력은 꼭 필요할 때만 사용한다.	(10.11, p.230-232 / PDF p.227-229)
U10-125	원문은 endl을 피하라고 조언한다.	(10.11, p.230-232 / PDF p.227-229)
U10-126	사용자 정의 타입에 의미 있는 텍스트 표현이 있으면 입출력 연산자를 제공한다.	(10.11, p.230-232 / PDF p.227-229)
U10-127	정밀도와 형식 설정은 이후 출력에도 지속된다.	(10.11, p.230-232 / PDF p.227-229)
U10-128	파일 사용 전에 연결 상태를 확인하고 메모리상 형식화에는 문자열 스트림을 쓴다.	(10.11, p.230-232 / PDF p.227-229)
U10-129	특정 운영체제 인터페이스보다 <filesystem>을 사용한다.	(10.11, p.230-232 / PDF p.227-229)
U10-130	읽기	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-131	ifstream	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-132	istringstream	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-133	쓰기	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-134	ofstream	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-135	ostringstream	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-136	읽기·쓰기	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-137	fstream	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-138	stringstream	(파일과 문자열 스트림의 대응, p.221-222 / PDF p.218-219)
U10-139	기본 버전	(파일 시스템 에러 보고, p.229 / PDF p.226)
U10-140	실패 시 filesystem_error 예외	(파일 시스템 에러 보고, p.229 / PDF p.226)
U10-141	error_code 인자 버전	(파일 시스템 에러 보고, p.229 / PDF p.226)
U10-142	인자로 전달한 에러 코드 확인	(파일 시스템 에러 보고, p.229 / PDF p.226)
U10-143	cin은 표준 입력 스트림이며, >>는 istream에서 값을 가져오는 연산이다.	(10.3, p.213,215 / PDF p.210,212)
U10-144	조정자는 입력·출력 형식을 제어한다.	(10.6, p.219,221 / PDF p.216,218)
U10-145	path는 운영체제의 기본 문자 집합과 관례를 다루는 경로 표현이다.	(10.10, p.225,229 / PDF p.222,226)

번호	문장·표 항목·원문 코드·수식	원문 위치
U10-146	문자 수준 입력은 꼭 필요할 때만 사용한다. 원문은 endl을 피하라고 조언한다.	(10.11, p.230-232 / PDF p.227-229)
U10-147	파일 스트림	(10.7, p.221-222 / PDF p.218-219)
U10-148	문자열 스트림	(10.8, p.222-223 / PDF p.219-220)

제11장. 컨테이너

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-001	컨테이너는 객체를 저장하는 일이 주목적인 클래스다.	(11.1, p.234 / PDF p.231)
U11-002	vector<T>는 T 타입 요소가 메모리에 연속적으로 저장되는 시퀀스다.	(11.2, p.234-235 / PDF p.231-232)
U11-003	작업에 적합한 컨테이너와 기본 연산을 선택하는 일은 프로그램 구성의 중요한 단계다.	(11.1, p.234 / PDF p.231)
U11-004	이 장은 이름과 전화번호를 담는 Entry의 전화번호부로 표준 컨테이너를 비교한다.	(11.1, p.234 / PDF p.231)
U11-005	예제에서는 불필요한 복잡성을 피하려고 전화번호를 int로 표현한다.	(11.1, p.234 / PDF p.231)
U11-006	전형적인 구현은 첫 요소, 사용 중인 요소의 끝, 할당 공간의 끝을 가리키는 정보와 할당자로 구성된다.	(11.2, p.234-235 / PDF p.231-232)
U11-007	기본 할당자는 new와 delete로 메모리를 관리한다.	(11.2, p.234-235 / PDF p.231-232)
U11-008	인덱스는 0에서 시작하며 size()는 실제 요소 수를 알려준다.	(11.2, p.235-236 / PDF p.232-233)
U11-009	요소 목록을 주는 {} 초기화와 초기 크기를 지정하는 () 초기화를 구별한다.	(11.2, p.235-236 / PDF p.232-233)
U11-010	크기만 지정하면 요소는 타입의 기본 값으로 초기화되고, 두 번째 인자로 별도의 초기 값을 지정할 수 있다.	(11.2, p.235-236 / PDF p.232-233)
U11-011	구간 for는 vector가 제공하는 요소 구간 전체를 순회한다.	(11.2, p.235-236 / PDF p.232-233)

U11-012	vector의 구간 for 순회 (원문 코드 발췌)
	void print_book(const vector<Entry>& book)
	{
	for (const auto& x : book) // "auto"에 대해서는 1.4절 참고
	cout << x << '\n';
	}
		(11.2, p.235 / PDF p.232)

U11-013	size와 capacity는 다르다.	(11.2, p.236-237 / PDF p.233-234)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-014	size는 실제 요소 수, capacity는 재할당 없이 저장할 수 있는 슬롯 수다.	(11.2, p.236-237 / PDF p.233-234)
U11-015	reserve()는 요소를 위한 공간을 확보하며, 새 공간을 할당하면 기존 요소를 그곳으로 이동시킨다.	(11.2, p.236-237 / PDF p.233-234)
U11-016	push_back()은 끝에 요소를 추가하고 size를 1 늘린다.	(11.2, p.236-237 / PDF p.233-234)
U11-017	원문의 간략한 Vector 구현은 공간이 부족할 때 용량을 늘려 매번 재할당하는 일을 피한다.	(11.2, p.236-237 / PDF p.233-234)

U11-018	용량을 확보한 뒤 요소를 생성하는 push_back (원문 코드 발췌)
	template<typename T>
	void Vector<T>::push_back(const T& t)
	{
	if (capacity()<size()+1) // t를 저장할 공간이 있는지 확인
	reserve(size()==0?8:2*size()); // 용량을 2배로 증가
	new(space) T{t}; // *space 를 t로 초기화
	++space;
	}
		(11.2, p.237 / PDF p.234)

U11-019	성능 향상을 기대한다는 이유만으로 reserve()를 호출할 필요는 없다.	(11.2, p.237-238 / PDF p.234-235)
U11-020	원문은 vector의 증가 전략을 신뢰하고, 요소 포인터나 반복자의 유효성을 위해 재할당을 막아야 할 때 명시적으로 reserve()를 사용하도록 권한다.	(11.2, p.237-238 / PDF p.234-235)
U11-021	대입과 초기화로 vector를 복사하면 각 요소도 복사되어 독립된 복사본이 생긴다.	(11.2, p.237-238 / PDF p.234-235)
U11-022	요소가 많으면 비용이 클 수 있으므로 복사가 바람직하지 않을 때 참조·포인터 또는 이동 연산을 사용한다.	(11.2, p.237-238 / PDF p.234-235)
U11-023	표준 컨테이너에는 내장 타입, 사용자 정의 타입, 포인터 등 거의 모든 타입을 요소로 사용할 수 있다.	(11.2.1, p.238-239 / PDF p.235-236)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-024	요소를 삽입하면 값이 컨테이너 안으로 복사된다.	(11.2.1, p.238-239 / PDF p.235-236)
U11-025	정수 7을 넣으면 요소 자체가 7의 값을 가지며, 다른 곳의 7을 가리키는 참조나 포인터를 저장하는 것이 아니다.	(11.2.1, p.238-239 / PDF p.235-236)
U11-026	이 방식은 간결하고 빠르게 접근할 수 있는 저장 구조를 만든다.	(11.2.1, p.238-239 / PDF p.235-236)
U11-027	가상 함수에 의존하는 클래스 계층의 다형적인 동작을 유지하려면 객체 자체 대신 포인터나 스마트 포인터를 요소로 저장한다.	(11.2.1, p.238-239 / PDF p.235-236)
U11-028	원문은 vector<Shape>로 Circle이나 Smiley를 저장하려는 방식을 피하고, vector<unique_ptr<Shape>>를 사용하는 형태를 제시한다.	(11.2.1, p.238-239 / PDF p.235-236)
U11-029	표준 vector의 operator[]는 구간 검사를 보장하지 않는다.	(11.2.2, p.239-240 / PDF p.236-237)
U11-030	book[book.size()]처럼 마지막 요소 다음을 참조하는 코드는 유효한 접근이 아니다.	(11.2.2, p.239-240 / PDF p.236-237)
U11-031	at()은 구간 밖 인덱스에 out_of_range를 던진다.	(11.2.2, p.239-240 / PDF p.236-237)
U11-032	원문의 Vec는 vector의 생성자를 상속하고 operator[]를 at()에 연결한다.	(11.2.2, p.239-240 / PDF p.236-237)

번호	문장·표 항목·원문 코드·수식	원문 위치

U11-033	at()에 연결하여 구간 검사하는 Vec (원문 코드 발췌)
	template<typename T>
	class Vec : public std::vector<T> {
	public:
	using vector<T>::vector; // (Vec라는 이름으로) vector의 생성자를 사용
	
	T& operator[](int i) // 구간 검사
	{ return vector<T>::at(i); }
	
	const T& operator[](int i) const // const 객체용 구간 검사; 4.2.1절
	{ return vector<T>::at(i); }
	};
		(11.2.2, p.239 / PDF p.236)

U11-034	Vec의 구간 밖 접근은 예외를 발생시켜 호출자가 잡을 수 있게 한다.	(11.2.2, p.240-241 / PDF p.237-238)
U11-035	잡지 않은 예외가 있으면 프로그램은 잘 정의된 방식으로 종료된다.	(11.2.2, p.240-241 / PDF p.237-238)
U11-036	원문은 main()의 try 블록과 기본 예외 핸들러에서 cerr로 진단을 출력하는 예도 제시한다.	(11.2.2, p.240-241 / PDF p.237-238)
U11-037	구간 검사의 비용은 하드웨어·최적화·접근 방식에 따라 다르므로, 표준은 모든 인덱싱에 검사를 강제하지 않는다.	(11.2.2, p.240-241 / PDF p.237-238)
U11-038	일부 구현은 디버그 검사 기능을 제공한다.	(11.2.2, p.240-241 / PDF p.237-238)
U11-039	구간 for나 begin()/end() 범위를 이용한 순회는 추가적인 구간 검사 비용 없이 구간 밖 접근을 피한다.	(11.2.2, p.241 / PDF p.238)
U11-040	코드에서 at()을 직접 사용할 수 있다면 Vec를 따로 만들 필요가 없다.	(11.2.2, p.241 / PDF p.238)
U11-041	list는 이중 연결 리스트다.	(11.3, p.241-242 / PDF p.238-239)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-042	다른 요소를 이동시키지 않고 시퀀스 중간에 삽입·삭제할 때 사용한다.	(11.3, p.241-242 / PDF p.238-239)
U11-043	vector처럼 인덱스로 접근하는 대신 처음부터 원하는 요소를 찾는다.	(11.3, p.241-242 / PDF p.238-239)
U11-044	begin()은 첫 요소를, end()는 마지막 요소 다음을 가리키는 반복자를 반환한다.	(11.3, p.241-242 / PDF p.238-239)

U11-045	반복자로 전화번호부 선형 탐색 (원문 코드 발췌)
	int get_number(const string& s)
	{
	for (auto p = phone_book.begin(); p!=phone_book.end(); ++p)
	if (p->name==s)
	return p->number;
	return 0; // 0 은 "수를 찾지 못함"을 의미
	}
		(11.3, p.243 / PDF p.240)

U11-046	반복자 p에 대해 *p는 요소를 참조하고 ++p는 다음 요소로 진행한다.	(11.3, p.243 / PDF p.240)
U11-047	p->m은 (*p).m과 같다.	(11.3, p.243 / PDF p.240)
U11-048	insert(p,elem)은 elem을 복사한 요소를 p 앞에 넣으며, p가 end()여도 된다.	(11.3, p.243 / PDF p.240)
U11-049	erase(p)는 p의 요소를 제거하고 소멸시킨다.	(11.3, p.243 / PDF p.240)
U11-050	원문의 전화번호부 예제는 vector로도 작성할 수 있고, 짧은 시퀀스에서는 삽입·삭제까지 vector 쪽이 더 효율적일 수 있으므로 직관만으로 list를 선택하지 않는다.	(11.3, p.243 / PDF p.240)
U11-051	forward_list는 순방향 순회만 제공하는 단일 연결 리스트다.	(11.3, p.243-244 / PDF p.240-241)
U11-052	이전 요소 포인터가 없어 공간을 아끼며 빈 객체의 크기는 포인터 하나다.	(11.3, p.243-244 / PDF p.240-241)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-053	요소 개수도 저장하지 않으므로 개수를 알려면 직접 세어야 한다.	(11.3, p.243-244 / PDF p.240-241)
U11-054	원문은 이 특성이 필요할 때 선택하도록 한다.	(11.3, p.243-244 / PDF p.240-241)
U11-055	map은 키와 값의 쌍을 담고 조회에 최적화된 연관 컨테이너다.	(11.4, p.244-245 / PDF p.241-242)
U11-056	원문은 일반적인 구현을 균형 이진 탐색 트리, 특히 레드-블랙 트리로 설명한다.	(11.4, p.244-245 / PDF p.241-242)
U11-057	연관 배열 또는 딕셔너리라고도 한다.	(11.4, p.244-245 / PDF p.241-242)
U11-058	키 타입의 값으로 인덱싱하면 대응하는 매핑된 타입의 값을 얻는다.	(11.4, p.244-245 / PDF p.241-242)

U11-059	map의 키로 전화번호 조회 (원문 코드 발췌)
	int get_number(const string& s)
	{
	return phone_book[s];
	}
		(11.4, p.245 / PDF p.242)

U11-060	없는 키에 대한 []는 조회만 하는 연산이 아니다.	(11.4, p.245 / PDF p.242)
U11-061	키를 못 찾으면 값 타입의 기본 값을 가진 항목이 삽입된다.	(11.4, p.245 / PDF p.242)
U11-062	전화번호부에서는 int의 기본 값 0이 유효하지 않은 번호를 나타낸다.	(11.4, p.245 / PDF p.242)
U11-063	이런 항목이 들어가는 것을 원치 않으면 [] 대신 find()와 insert()를 사용한다.	(11.4, p.245 / PDF p.242)
U11-064	map의 조회 비용은 요소 수 n에 대해 O(log(n))이다.	(11.5, p.245-246 / PDF p.242-243)
U11-065	unordered_map은 해시 테이블을 이용하며 순서 비교 함수를 요구하지 않는다.	(11.5, p.245-246 / PDF p.242-243)
U11-066	map과 비슷하게 초기화하고 인덱싱할 수 있지만 요소 사이의 순서를 유지하지 않는다.	(11.5, p.245-246 / PDF p.242-243)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-067	표준 라이브러리는 string·내장 타입·여러 라이브러리 타입의 기본 해시 함수를 제공한다.	(11.5, p.245-246 / PDF p.242-243)
U11-068	사용자 정의 타입에는 해시 함수 객체를 전달하거나 std::hash를 특수화할 수 있다.	(11.5, p.246-248 / PDF p.243-245)
U11-069	원문의 Record 예제는 이름과 product_code의 표준 해시를 배타적 논리합(^)으로 결합한다.	(11.5, p.246-248 / PDF p.243-245)
U11-070	좋은 해시 함수에는 데이터에 대한 지식과 실험이 필요할 수 있다.	(11.5, p.246-248 / PDF p.243-245)
U11-071	좋은 함수를 쓸 때 큰 unordered_map은 map보다 훨씬 빠를 수 있지만, 잘못된 해시 함수에서는 최악의 경우가 map보다 훨씬 느릴 수 있다.	(11.5, p.246-248 / PDF p.243-245)
U11-072	vector<T> / list<T> / forward_list<T>	(11.6, p.248 / PDF p.245)
U11-073	가변 크기 벡터 / 이중 연결 리스트 / 단일 연결 리스트	(11.6, p.248 / PDF p.245)
U11-074	deque<T>	(11.6, p.248 / PDF p.245)
U11-075	양방향 큐	(11.6, p.248 / PDF p.245)
U11-076	set<T> / multiset<T>	(11.6, p.248 / PDF p.245)
U11-077	값만 가진 집합 / 값의 중복을 허용하는 집합	(11.6, p.248 / PDF p.245)
U11-078	map<K,V> / multimap<K,V>	(11.6, p.248 / PDF p.245)
U11-079	키와 값의 연관 배열 / 키의 중복을 허용하는 map	(11.6, p.248 / PDF p.245)
U11-080	unordered_map / unordered_multimap	(11.6, p.248 / PDF p.245)
U11-081	해시 조회를 사용하는 map / multimap	(11.6, p.248 / PDF p.245)
U11-082	unordered_set / unordered_multiset	(11.6, p.248 / PDF p.245)
U11-083	해시 조회를 사용하는 set / multiset	(11.6, p.248 / PDF p.245)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-084	컨테이너는 std에 정의되고 <vector>, <list>, <map> 등에서 제공된다.	(11.6, p.249-250 / PDF p.246-247)
U11-085	queue·stack·priority_queue는 컨테이너 어댑터이며 array와 bitset은 고정 크기 구조다.	(11.6, p.249-250 / PDF p.246-247)
U11-086	여러 컨테이너는 일관된 표기와 의미를 갖도록 설계되어, 알고리즘을 개별 컨테이너 타입과 독립적으로 기술할 수 있다.	(11.6, p.249-250 / PDF p.246-247)
U11-087	value_type	(11.6, p.249-250 / PDF p.246-247)
U11-088	요소 타입	(11.6, p.249-250 / PDF p.246-247)
U11-089	begin()/end(), cbegin()/cend()	(11.6, p.249-250 / PDF p.246-247)
U11-090	첫 요소 / 마지막 요소 다음; const 반복자도 제공	(11.6, p.249-250 / PDF p.246-247)
U11-091	size()/empty()	(11.6, p.249-250 / PDF p.246-247)
U11-092	요소 수 / 비어 있는지 검사	(11.6, p.249-250 / PDF p.246-247)
U11-093	capacity()/reserve(k)	(11.6, p.249-250 / PDF p.246-247)
U11-094	새 할당 없는 저장 용량 / 용량 확보	(11.6, p.249-250 / PDF p.246-247)
U11-095	resize(k)	(11.6, p.249-250 / PDF p.246-247)
U11-096	요소 수 변경; 추가 요소는 value_type{}	(11.6, p.249-250 / PDF p.246-247)
U11-097	[k] / at(k)	(11.6, p.249-250 / PDF p.246-247)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-098	구간 검사 없는 접근 / 구간 밖에서 예외	(11.6, p.249-250 / PDF p.246-247)
U11-099	push_back(x) / emplace_back(a)	(11.6, p.249-250 / PDF p.246-247)
U11-100	끝에 x 추가 / 인자로 새 요소를 직접 생성	(11.6, p.249-250 / PDF p.246-247)
U11-101	insert(p,x) / erase(p)	(11.6, p.249-250 / PDF p.246-247)
U11-102	p 앞에 삽입 / p 요소 제거	(11.6, p.249-250 / PDF p.246-247)
U11-103	= / == / <	(11.6, p.249-250 / PDF p.246-247)
U11-104	대입 / 요소의 상등성 / 사전 순 비교	(11.6, p.249-250 / PDF p.246-247)
U11-105	각 컨테이너에 맞는 연산을 선택해야 한다.	(11.6, p.250 / PDF p.247)
U11-106	vector는 연속 저장에 따른 저렴한 인덱싱·순회를 제공하지만 중간 삽입·삭제 때 요소 이동이 필요하다.	(11.6, p.250 / PDF p.247)
U11-107	list는 그 반대의 성격을 갖는다.	(11.6, p.250 / PDF p.247)
U11-108	빈 경우가 많다면 한 워드만 필요한 forward_list를 고려할 수 있다.	(11.6, p.250 / PDF p.247)
U11-109	배치 연산은 객체를 먼저 만들어 복사하는 대신, 전달한 생성자 인자로 컨테이너의 새 저장 공간에 요소를 직접 만든다.	(11.6, p.250 / PDF p.247)
U11-110	표준 컨테이너를 익히고 기본 선택으로 vector를 고려한다.	(11.7, p.251-252 / PDF p.248-249)
U11-111	성능은 직관이 아니라 측정으로 확인한다.	(11.7, p.251-252 / PDF p.248-249)
U11-112	순회에는 구간 for나 시작/끝 반복자를 사용한다.	(11.7, p.251-252 / PDF p.248-249)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-113	구간 검사가 필요하면 at()을 사용하며 []가 검사한다고 가정하지 않는다.	(11.7, p.251-252 / PDF p.248-249)
U11-114	vector의 크기 변경 후 반복자를 그대로 사용하지 않는다.	(11.7, p.251-252 / PDF p.248-249)
U11-115	포인터·반복자 유효성을 위한 재할당 방지와 단순 성능 추측을 구별한다.	(11.7, p.251-252 / PDF p.248-249)
U11-116	컨테이너 요소는 값이다.	(11.7, p.251-252 / PDF p.248-249)
U11-117	다형적인 동작이 필요하면 포인터를 저장한다.	(11.7, p.251-252 / PDF p.248-249)
U11-118	컨테이너 전달은 참조로, 반환은 값으로 한다.	(11.7, p.251-252 / PDF p.248-249)
U11-119	많은 데이터의 빠른 조회에는 순서 없는 컨테이너, 순서 있는 순회에는 map·set 같은 순서 있는 연관 컨테이너를 고려한다.	(11.7, p.251-252 / PDF p.248-249)
U11-120	크기로 생성할 때 ()와 요소 목록으로 생성할 때 {}를 구별한다.	(11.7, p.251-252 / PDF p.248-249)
U11-121	해시 함수의 적절성은 실험으로 확인한다.	(11.7, p.251-252 / PDF p.248-249)
U11-122	조회 구조	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-123	균형 이진 탐색 트리	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-124	해시 테이블	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-125	필요한 함수	(map과 unordered_map, p.245-248 / PDF p.242-245)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-126	순서 비교 함수; 기본적으로 <	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-127	해시 함수	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-128	요소 순서	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-129	순서 있는 시퀀스	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-130	순서 유지 안 함	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-131	성능 조건	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-132	조회 O(log(n))	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-133	좋은 해시는 큰 데이터에서 빠름; 나쁜 해시는 최악의 경우 훨씬 느림	(map과 unordered_map, p.245-248 / PDF p.242-245)
U11-134	list는 이중 연결 리스트이며, 다른 요소를 이동시키지 않고 시퀀스 중간에 삽입·삭제할 때 사용한다.	(11.3, p.241,243 / PDF p.238,240)
U11-135	unordered_map은 해시 테이블을 이용하며 요소 사이의 순서를 유지하지 않는다.	(11.5, p.246,248 / PDF p.243,245)
U11-136	표준 컨테이너를 익히고 기본 선택으로 vector를 고려한다. 성능은 직관이 아니라 측정으로 확인한다.	(11.7, p.251-252 / PDF p.248-249)

번호	문장·표 항목·원문 코드·수식	원문 위치
U11-137	순회에는 구간 for나 시작/끝 반복자를 사용한다. 구간 검사가 필요하면 at()을 사용하며 []가 검사한다고 가정하지 않는다.	(11.7, p.251-252 / PDF p.248-249)
U11-138	vector의 크기 변경 후 반복자를 그대로 사용하지 않는다. 포인터·반복자 유효성을 위한 재할당 방지와 단순 성능 추측을 구별한다.	(11.7, p.251-252 / PDF p.248-249)
U11-139	컨테이너 요소는 값이다. 다형적인 동작이 필요하면 포인터를 저장한다. 컨테이너 전달은 참조로, 반환은 값으로 한다.	(11.7, p.251-252 / PDF p.248-249)
U11-140	map	(11.4, p.244-245 / PDF p.241-242)
U11-141	unordered_map	(11.5, p.246,248 / PDF p.243,245)

제12장. 알고리즘

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-001	입력 시퀀스는 첫 요소를 포함하고 끝 위치는 포함하지 않는 반개구간 [begin:end)이며, 첫 요소와 마지막 요소 다음을 가리키는 반복자 쌍으로 나타낸다.	(12.1, p.254-255 / PDF p.251-252)
U12-002	반복자는 특정 타입의 객체이며 저장 구조에 맞는 정보를 가진다.	(12.3, p.259-260 / PDF p.256-257)
U12-003	표준 라이브러리 알고리즘은 컨테이너에 저장한 요소의 정렬·출력·검색·부분집합 추출 같은 공통 작업을 제공한다.	(12.1, p.254-255 / PDF p.251-252)
U12-004	sort는 비교 연산으로 입력 구간을 정렬하고 unique_copy는 이웃한 중복을 복사하지 않는다.	(12.1, p.254-255 / PDF p.251-252)
U12-005	기존 출력 위치를 지정하면 그 뒤 요소를 덮어쓰므로 충분한 출력 공간이 있어야 한다.	(12.1, p.254-255 / PDF p.251-252)
U12-006	back_inserter는 컨테이너 뒤에 삽입하는 반복자를 만들어 필요에 따라 컨테이너를 확장한다.	(12.1, p.254-255 / PDF p.251-252)
U12-007	이 방식은 미리 크기를 정하거나 C 스타일 realloc()으로 직접 관리할 필요를 줄인다.	(12.1, p.254-255 / PDF p.251-252)

U12-008	정렬 후 중복 없이 새 list에 삽입 (원문 코드 발췌)
	list<Entry> f(vector<Entry>& vec)
	{
	list<Entry> res;
	sort(vec.begin(),vec.end());
	unique_copy(vec.begin(),vec.end(),back_inserter(res)); // res 뒤에 붙이기
	return res;
	}
		(12.1, p.255 / PDF p.252)

U12-009	begin()과 end()는 컨테이너에서 반복자를 얻는 대표적인 함수다.	(12.2, p.255-256 / PDF p.252-253)
U12-010	find는 찾은 요소의 반복자를 반환하고, 찾지 못하면 입력 구간의 end()를 반환한다.	(12.2, p.255-256 / PDF p.252-253)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-011	그러므로 반환값을 끝 반복자와 비교하여 검색 성공을 판정한다.	(12.2, p.255-256 / PDF p.252-253)
U12-012	원문의 find_all은 일치하는 값 자체 대신 그 위치를 가리키는 반복자들의 vector를 반환한다.	(12.2, p.256-258 / PDF p.253-255)
U12-013	이 반복자로 원래 요소를 바꿀 수도 있으므로, 변경할 문자열이나 컨테이너는 const가 아닌 참조로 받는다.	(12.2, p.256-258 / PDF p.253-255)
U12-014	처음에는 string 전용으로 작성하고 이후 컨테이너 C와 값 V의 템플릿으로 일반화한다.	(12.2, p.256-258 / PDF p.253-255)

U12-015	반복자 타입 별칭을 사용한 제네릭 find_all (원문 코드 발췌)
	template<typename T>
	using Iterator = typename T::iterator; // T의 반복자
	
	template<typename C, typename V>
	vector<Iterator<C>> find_all(C& c, V v) // c에서 v가 나타나는 곳을 모두 찾기
	{
	vector<Iterator<C>> res;
	for (auto p = c.begin(); p!=c.end(); ++p)
	if (*p==v)
	res.push_back(p);
	return res;
	}
		(12.2, p.258 / PDF p.255)

U12-016	typename C::iterator의 typename은 iterator가 값이 아닌 타입임을 컴파일러에 알려준다.	(12.2, p.258-259 / PDF p.255-256)
U12-017	타입 별칭 Iterator<T>는 이 세부사항을 숨긴다.	(12.2, p.258-259 / PDF p.255-256)
U12-018	알고리즘은 반복자로 데이터에 적용되므로 저장 컨테이너를 알 필요가 없고, 컨테이너 역시 어떤 알고리즘이 쓰이는지 알 필요 없이 반복자를 제공한다.	(12.2, p.258-259 / PDF p.255-256)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-019	이 분리가 일반적이고 유연한 소프트웨어의 바탕이다.	(12.2, p.258-259 / PDF p.255-256)
U12-020	vector에서는 단순 포인터가 합리적인 구현일 수 있고, vector 포인터와 인덱스를 함께 저장하면 구간 검사를 할 수 있다.	(12.3, p.259-260 / PDF p.256-257)
U12-021	list에서는 다음 링크를 따라가야 하므로 단순 요소 포인터가 아니라 링크를 가리키는 정보가 필요하다.	(12.3, p.259-260 / PDF p.256-257)
U12-022	구현은 달라도 ++로 다음 요소를 가리키고 *로 요소를 참조하는 공통 의미와 명명 규칙을 따른다.	(12.3, p.259-260 / PDF p.256-257)
U12-023	이러한 요구를 표현한 것이 Iterator 컨셉이다.	(12.3, p.259-260 / PDF p.256-257)
U12-024	사용자에게 내부 구현 전체가 필요한 것은 아니며 각 컨테이너의 iterator·const_iterator라는 관례적인 이름을 이용하면 된다.	(12.3, p.259-260 / PDF p.256-257)
U12-025	요소 시퀀스는 컨테이너에만 있는 것이 아니다.	(12.4, p.261 / PDF p.258)
U12-026	입력 스트림도 값의 시퀀스를 만들고 출력 스트림도 시퀀스를 받는다.	(12.4, p.261 / PDF p.258)
U12-027	ostream_iterator는 스트림과 출력 타입을 지정해 만들며, 역참조 위치에 대입하면 스트림으로 출력한다. ++는 배열 포인터와 같은 사용 방식을 흉내 낸다.	(12.4, p.261 / PDF p.258)
U12-028	istream_iterator는 입력 스트림을 읽기 전용 컨테이너처럼 다루게 한다.	(12.4, p.261 / PDF p.258)

번호	문장·표 항목·원문 코드·수식	원문 위치

U12-029	파일 단어를 정렬하고 중복 없이 출력 (원문 코드 발췌)
	int main()
	{
	string from, to;
	cin >> from >> to; // 원본과 출력 파일 이름 가져오기
	
	ifstream is {from}; // 파일 "from"의 입력 스트림
	istream_iterator<string> ii {is}; // 스트림에 사용할 입력 반복자
	istream_iterator<string> eos {}; // 입력(의 끝을 나타내는) 보초 값
	
	ofstream os {to}; // 파일 "to"의 출력 스트림
	ostream_iterator<string> oo {os,"\n"}; // 스트림의 출력 반복자
	
	vector<string> b {ii,eos}; // b는 입력으로부터 초기화된 벡터
	sort(b.begin(),b.end()); // 버퍼를 정렬
	
	unique_copy(b.begin(),b.end(),oo); // 버퍼를 출력에 복사, 중복된 값은 버림
	
	return !is.eof() || !os; // 에러 상태 반환(1.2.1절, 10.4절)
	}
		(12.4, p.262 / PDF p.259)

U12-030	기본 생성한 istream_iterator는 입력 끝의 보초 값으로 쓰인다.	(12.4, p.262-263 / PDF p.259-260)
U12-031	ostream_iterator의 두 번째 인자는 출력 값 사이의 구분자다.	(12.4, p.262-263 / PDF p.259-260)
U12-032	예제는 파일의 단어를 vector로 읽고 정렬한 뒤 unique_copy로 중복을 제거해 출력한다.	(12.4, p.262-263 / PDF p.259-260)
U12-033	원문은 set을 사용해 입력 단계에서 중복을 없애고 순서를 유지하면 sort·unique_copy 대신 copy만 쓸 수 있다는 대안도 제시한다.	(12.4, p.262-263 / PDF p.259-260)
U12-034	코드 길이를 줄이는 선택은 취향과 가독성을 함께 고려한다.	(12.4, p.262-263 / PDF p.259-260)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-035	술어는 알고리즘이 각 요소에 적용할 조건을 전달하는 수단이다.	(12.5, p.263-264 / PDF p.260-261)
U12-036	find가 지정 값과 같은 요소를 찾는 데 비해 find_if는 전달된 술어가 참이 되는 첫 요소를 찾는다.	(12.5, p.263-264 / PDF p.260-261)
U12-037	원문의 map 예제는 값 부분이 42보다 큰 쌍을 찾는다.	(12.5, p.263-264 / PDF p.260-261)
U12-038	비교 기준을 저장한 함수 객체와 람다 표현식 모두 사용할 수 있다.	(12.5, p.263-264 / PDF p.260-261)
U12-039	람다 술어를 전달한 find_if (원문 코드 발췌) auto p = find_if(m.begin(), m.end(), [](const pair<string,int>& r) { return r.second>42; });	(12.5, p.264 / PDF p.261)
U12-040	술어는 적용 대상 요소를 변경하면 안 된다.	(12.5, p.264 / PDF p.261)
U12-041	원문의 Greater_than은 비교값을 val에 저장하고 pair의 second를 이 값과 비교한다.	(12.5, p.264 / PDF p.261)
U12-042	람다 버전도 같은 비교를 직접 표현한다.	(12.5, p.264 / PDF p.261)
U12-043	이 책에서 표준 알고리즘은 요소 시퀀스에 적용되는 함수 템플릿 연산이다.	(12.6, p.264-266 / PDF p.261-263)
U12-044	std에 정의되고 <algorithm>에서 제공된다.	(12.6, p.264-266 / PDF p.261-263)
U12-045	컨테이너뿐 아니라 string과 내장 배열에도 적용할 수 있다.	(12.6, p.264-266 / PDF p.261-263)
U12-046	for_each	(12.6, p.265 / PDF p.262)
U12-047	모든 요소에 함수 적용	(12.6, p.265 / PDF p.262)
U12-048	find / find_if	(12.6, p.265 / PDF p.262)
U12-049	지정 값 / 술어를 만족하는 첫 위치	(12.6, p.265 / PDF p.262)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-050	count / count_if	(12.6, p.265 / PDF p.262)
U12-051	지정 값 / 술어에 맞는 요소 개수	(12.6, p.265 / PDF p.262)
U12-052	replace / replace_if	(12.6, p.265 / PDF p.262)
U12-053	지정 값 / 술어에 맞는 요소를 새 값으로 대체	(12.6, p.265 / PDF p.262)
U12-054	copy / copy_if / move	(12.6, p.265 / PDF p.262)
U12-055	전체 복사 / 조건에 맞는 요소 복사 / 이동	(12.6, p.265 / PDF p.262)
U12-056	unique_copy	(12.6, p.265 / PDF p.262)
U12-057	이웃한 중복을 제외하고 복사	(12.6, p.265 / PDF p.262)
U12-058	sort	(12.6, p.265 / PDF p.262)
U12-059	기본 < 또는 주어진 비교 연산으로 정렬	(12.6, p.265 / PDF p.262)
U12-060	equal_range	(12.6, p.265 / PDF p.262)
U12-061	정렬된 구간에서 같은 값의 부분 구간을 찾음; 기본적으로 이진 탐색	(12.6, p.265 / PDF p.262)
U12-062	merge	(12.6, p.265 / PDF p.262)
U12-063	정렬된 두 구간을 기본 또는 지정 비교 연산으로 병합	(12.6, p.265 / PDF p.262)
U12-064	일부 알고리즘은 요소 값을 변경하지만 입력 시퀀스 자체에 요소를 추가하거나 제거하지는 않는다.	(12.6, p.266 / PDF p.263)
U12-065	반복자만으로는 소유 컨테이너를 알 수 없기 때문이다.	(12.6, p.266 / PDF p.263)
U12-066	크기를 바꾸려면 back_inserter처럼 컨테이너를 아는 수단이나 push_back·erase 같은 컨테이너 연산이 필요하다.	(12.6, p.266 / PDF p.263)
U12-067	원문은 직접 루프를 만들기 전에 잘 설계된 표준 알고리즘으로 표현할 수 있는지 살피도록 권한다.	(12.6, p.266 / PDF p.263)
U12-068	이 절은 원문 집필 시점의 Ranges Technical Specification과 C++20에 대한 전망을 설명한다.	(12.7, p.266 / PDF p.263)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-069	원문은 <experimental/ranges>의 컨셉을 소개하며 비슷한 기능이 C++20의 std에 들어가기를 기대한다고 쓴다.	(12.7, p.266 / PDF p.263)
U12-070	이 노트는 그 시점의 명칭과 조건을 그대로 다룬다.	(12.7, p.266 / PDF p.263)
U12-071	Range는 begin/end 시퀀스의 일반화다. {begin,end}, {begin,n} 또는 {begin,pred}로 구간을 정의할 수 있다.	(12.7, p.266-267 / PDF p.263-264)
U12-072	n은 요소 수, pred는 반복자가 끝에 도달했는지 판정하는 술어다.	(12.7, p.266-267 / PDF p.263-264)
U12-073	마지막 방식은 동적으로 정의되거나 무한한 시퀀스도 표현한다.	(12.7, p.266-267 / PDF p.263-264)
U12-074	이 구간을 받는 알고리즘은 sort(v)와 같은 호출을 가능하게 한다.	(12.7, p.266-267 / PDF p.263-264)
U12-075	Same / DerivedFrom / ConvertibleTo	(12.7, p.267-268 / PDF p.264-265)
U12-076	같은 타입 / 파생 관계 / 변환 가능	(12.7, p.267-268 / PDF p.264-265)
U12-077	CommonReference / Common	(12.7, p.267-268 / PDF p.264-265)
U12-078	공통 참조 타입 / 공통 타입을 가짐	(12.7, p.267-268 / PDF p.264-265)
U12-079	Integral / SignedIntegral / UnsignedIntegral	(12.7, p.267-268 / PDF p.264-265)
U12-080	정수 / 부호 있는 정수 / 부호 없는 정수	(12.7, p.267-268 / PDF p.264-265)
U12-081	Assignable / SwappableWith / Swappable	(12.7, p.267-268 / PDF p.264-265)
U12-082	대입 가능 / 두 타입 값의 교환 가능 / 같은 타입 사이의 교환 가능	(12.7, p.267-268 / PDF p.264-265)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-083	Common은 여러 타입을 공통 타입 c로 변환한 뒤에도 비교가 합리적이어야 하는 알고리즘을 기술하는 데 쓰인다.	(12.7, p.268 / PDF p.265)
U12-084	원문은 string과 char*, int와 double의 비교를 들고 string과 int는 비교하지 않는다고 구별한다.	(12.7, p.268 / PDF p.265)
U12-085	라이브러리가 적당한 정의를 제공하지 않는 경우가 아니라면 common_type_t를 직접 특수화할 필요가 없다.	(12.7, p.268 / PDF p.265)
U12-086	Boolean	(12.7, p.268-269 / PDF p.265-266)
U12-087	불리언 값으로 사용 가능	(12.7, p.268-269 / PDF p.265-266)
U12-088	WeaklyEqualityComparableWith / WeaklyEqualityComparable	(12.7, p.268-269 / PDF p.265-266)
U12-089	서로 다른 타입 / 같은 타입 값의 ==·!= 비교	(12.7, p.268-269 / PDF p.265-266)
U12-090	EqualityComparableWith / EqualityComparable	(12.7, p.268-269 / PDF p.265-266)
U12-091	상등성 비교; 같은 타입 버전은 T,T 관계	(12.7, p.268-269 / PDF p.265-266)
U12-092	StrictTotallyOrderedWith / StrictTotallyOrdered	(12.7, p.268-269 / PDF p.265-266)
U12-093	<·<=·>·>=를 이용한 완전 순서; 같은 타입 버전은 T,T 관계	(12.7, p.268-269 / PDF p.265-266)
U12-094	Destructible / Constructible / DefaultConstructible	(12.7, p.268-269 / PDF p.265-266)
U12-095	소멸 및 주소 연산 / 주어진 인자로 생성 / 기본 생성	(12.7, p.268-269 / PDF p.265-266)
U12-096	MoveConstructible / CopyConstructible	(12.7, p.268-269 / PDF p.265-266)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-097	이동 생성 / 복사와 이동 생성	(12.7, p.268-269 / PDF p.265-266)
U12-098	Movable / Copyable	(12.7, p.268-269 / PDF p.265-266)
U12-099	이동 생성·대입·교환 / 복사 생성 및 복사 대입을 포함	(12.7, p.268-269 / PDF p.265-266)
U12-100	Semiregular / Regular	(12.7, p.268-269 / PDF p.265-266)
U12-101	Copyable와 기본 생성 / 여기에 상등성 비교를 포함	(12.7, p.268-269 / PDF p.265-266)
U12-102	원문은 WeaklyEqualityComparableWith와 WeaklyEqualityComparable를 혼용하면 오버로드 기회가 줄 수 있다고 경고한다.	(12.7, p.269 / PDF p.266)
U12-103	Regular 타입은 int처럼 동작해 사용이 단순해지는 이상적인 성격으로 설명하며, ==가 없는 기본적인 클래스는 Semiregular로 구분한다.	(12.7, p.269 / PDF p.266)
U12-104	Invocable<F,Args>	(12.7, p.270 / PDF p.267)
U12-105	Args 인자 목록으로 F 호출	(12.7, p.270 / PDF p.267)
U12-106	InvocableRegular<F,Args>	(12.7, p.270 / PDF p.267)
U12-107	호출 가능하고 상등성을 보존	(12.7, p.270 / PDF p.267)
U12-108	Predicate<F,Args>	(12.7, p.270 / PDF p.267)
U12-109	호출하여 bool 반환	(12.7, p.270 / PDF p.267)
U12-110	Relation<F,T,U>	(12.7, p.270 / PDF p.267)
U12-111	Predicate<F,T,U>	(12.7, p.270 / PDF p.267)
U12-112	StrictWeakOrder<F,T,U>	(12.7, p.270 / PDF p.267)
U12-113	Relation이 엄격하고 약한 순서를 제공	(12.7, p.270 / PDF p.267)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-114	상등성 보존은 x==y일 때 f(x)==f(y)가 되는 성질이다.	(12.7, p.270 / PDF p.267)
U12-115	비교 연산에는 일반적으로 엄격하고 약한 순서를 가정한다.	(12.7, p.270 / PDF p.267)
U12-116	원문은 Relation과 StrictWeakOrder의 차이가 의미론적인 것이며, 이 차이를 이름으로 의도에 드러낸다고 설명한다.	(12.7, p.270 / PDF p.267)
U12-117	Iterator / Sentinel / SizedSentinel	(12.7, p.270-271 / PDF p.267-268)
U12-118	증가·역참조 / 구간 끝의 보초 / 차이 연산이 가능한 보초	(12.7, p.270-271 / PDF p.267-268)
U12-119	InputIterator / OutputIterator	(12.7, p.270-271 / PDF p.267-268)
U12-120	입력의 읽기 / 출력의 쓰기	(12.7, p.270-271 / PDF p.267-268)
U12-121	ForwardIterator / BidirectionalIterator	(12.7, p.270-271 / PDF p.267-268)
U12-122	여러 번 순방향 순회 / 여기에 -- 지원	(12.7, p.270-271 / PDF p.267-268)
U12-123	RandomAccessIterator	(12.7, p.270-271 / PDF p.267-268)
U12-124	양방향 반복자에 원문 표의 산술·인덱싱 연산을 추가	(12.7, p.270-271 / PDF p.267-268)
U12-125	Permutable / Mergeable / Sortable	(12.7, p.270-271 / PDF p.267-268)
U12-126	이동·교환 / 정렬 구간 병합 / 지정 관계로 정렬 가능	(12.7, p.270-271 / PDF p.267-268)
U12-127	Range / SizedRange	(12.7, p.270-271 / PDF p.267-268)
U12-128	시작 반복자와 보초 / 크기를 상수 시간에 알 수 있음	(12.7, p.270-271 / PDF p.267-268)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-129	View / BoundedRange	(12.7, p.270-271 / PDF p.267-268)
U12-130	상수 시간 복사·이동·대입 / 반복자와 보초의 타입이 같음	(12.7, p.270-271 / PDF p.267-268)
U12-131	InputRange / OutputRange / ForwardRange / BidirectionalRange / RandomAccessRange	(12.7, p.270-271 / PDF p.267-268)
U12-132	각 이름에 대응하는 반복자 컨셉을 만족하는 구간	(12.7, p.270-271 / PDF p.267-268)
U12-133	보초는 주어진 반복자에서 시작해 종료 술어를 만족할 때까지 순회하게 하는 개념이다.	(12.7, p.271-272 / PDF p.268-269)
U12-134	원문은 C 스타일 문자열에서 *p==0을 끝 조건으로 삼는 예를 든다.	(12.7, p.271-272 / PDF p.268-269)
U12-135	반복자의 종류는 알고리즘을 더 적절하게 선택하는 데 활용한다.	(12.7, p.271-272 / PDF p.268-269)
U12-136	원문은 Ranges를 기다리지 않고도 컨테이너를 받아 내부에서 begin()/end() 버전을 호출하는 간단한 sort 래퍼를 작성할 수 있다고 설명한다.	(12.8, p.272 / PDF p.269)
U12-137	기본 비교 버전과 술어 버전을 별도 Estd 네임스페이스에 두면 std 사용 시 충돌을 피하고 나중에 Range로 교체하기도 쉽다.	(12.8, p.272 / PDF p.269)
U12-138	각 데이터 항목의 계산이 서로 독립적이면 같은 작업을 병렬로 수행할 수 있다.	(12.9, p.273-274 / PDF p.270-271)
U12-139	병렬 실행은 여러 스레드에서 작업하고 벡터화된 실행(SIMD)은 한 스레드에서 벡터화로 작업한다.	(12.9, p.273-274 / PDF p.270-271)
U12-140	<execution>은 seq·par·par_unseq 실행 정책을 제공한다.	(12.9, p.273-274 / PDF p.270-271)

번호	문장·표 항목·원문 코드·수식	원문 위치

U12-141	정렬의 실행 정책 (원문 코드 발췌)
	sort(v.begin(),v.end()); // 순차
	sort(seq,v.begin(),v.end()); // 순차(기본과 동일)
	sort(par,v.begin(),v.end()); // 병렬
	sort(par_unseq,v.begin(),v.end()); // 병렬 혹은 벡터화
		(12.9, p.273 / PDF p.270)

U12-142	정책은 실행에 대한 힌트이며 실제 병렬성은 컴파일러와 런타임 스케줄러가 정한다.	(12.9, p.273-274 / PDF p.270-271)
U12-143	효과는 알고리즘·요소 수·하드웨어·사용률에 따라 달라지므로 반드시 실험으로 확인한다.	(12.9, p.273-274 / PDF p.270-271)
U12-144	원문은 12.6절 표의 알고리즘 가운데 equal_range를 병렬 정책 대상에서 제외해 설명한다.	(12.9, p.273-274 / PDF p.270-271)
U12-145	병렬 실행을 요청할 때 데이터 경합과 데드락을 피해야 한다.	(12.9, p.273-274 / PDF p.270-271)
U12-146	알고리즘의 입력은 반복자로 정의한 반개구간이며, 검색 실패는 일반적으로 입력 끝으로 표시한다.	(12.10, p.274 / PDF p.271)
U12-147	알고리즘은 전달된 시퀀스 자체에 요소를 추가·제거하지 않는다.	(12.10, p.274 / PDF p.271)
U12-148	직접 루프를 만들기 전에 표준 제네릭 알고리즘을 고려한다.	(12.10, p.274 / PDF p.271)
U12-149	함수 객체와 술어로 동작을 확장하되 술어가 인자를 바꾸지 않게 한다.	(12.10, p.274 / PDF p.271)
U12-150	반복자 쌍 방식이 번거로우면 컨테이너/구간 알고리즘을 사용한다.	(12.10, p.274 / PDF p.271)
U12-151	실행 수단	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-152	여러 스레드; 여러 코어일 수도 있음	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-153	한 스레드에서 벡터화(SIMD)	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)

번호	문장·표 항목·원문 코드·수식	원문 위치
U12-154	정책 요청	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-155	par	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-156	par_unseq가 병렬 또는 벡터화를 허용	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-157	보장 조건	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-158	가능한 경우; 정책은 힌트	(병렬 실행과 벡터화된 실행, p.273 / PDF p.270)
U12-159	ostream_iterator는 스트림과 출력 타입을 지정해 만들며, 역참조 위치에 대입하면 스트림으로 출력한다.	(12.4, p.261,263 / PDF p.258,260)
U12-160	표준 알고리즘은 요소 시퀀스에 적용되는 함수 템플릿 연산이며, std에 정의되고 <algorithm>에서 제공된다.	(12.6, p.265-266 / PDF p.262-263)
U12-161	find_if는 전달된 술어가 참이 되는 첫 요소를 찾는다.	(12.5, p.263-264 / PDF p.260-261)
U12-162	직접 루프를 만들기 전에 표준 제네릭 알고리즘을 고려한다. 함수 객체와 술어로 동작을 확장하되 술어가 인자를 바꾸지 않게 한다.	(12.10, p.274 / PDF p.271)
U12-163	find	(12.2, p.255-256 / PDF p.252-253)
U12-164	find_if	(12.5, p.263-264 / PDF p.260-261)

제13장. 유틸리티

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-001	자원은 획득한 후 언젠가 해제해야 하는 것으로 메모리·소켓·스레드 핸들·파일 핸들이 예다.	(13.2, p.276-277 / PDF p.273-274)
U13-002	unique_ptr은 유일한 소유권을, shared_ptr은 공유된 소유권을 표현한다.	(13.2.1, p.277,281 / PDF p.274,278)
U13-003	이 장은 컨테이너·입출력 같은 큰 분류에 딱 들어맞지 않지만 널리 쓰이는 표준 라이브러리 유틸리티를 다룬다.	(13.1, p.276 / PDF p.273)
U13-004	설계와 프로그램을 표현하는 일반적인 용어로 쓰이는 타입과, 다른 라이브러리 기능을 구현하는 재료가 포함된다.	(13.1, p.276 / PDF p.273)
U13-005	유틸리티는 지나치게 복잡하거나 다른 기능에 의존하지 않아야 한다.	(13.1, p.276 / PDF p.273)
U13-006	누수는 장시간 프로그램의 성능 저하나 충돌뿐 아니라 짧은 프로그램의 자원 부족도 일으킬 수 있다.	(13.2, p.276-277 / PDF p.273-274)
U13-007	표준 라이브러리는 자원을 책임지는 객체의 생성·소멸과 자원의 생애 주기를 연결해 누수를 막는다.	(13.2, p.276-277 / PDF p.273-274)
U13-008	RAII는 생성자에서 자원을 획득하고 소멸자에서 해제하는 관례적 관리 방식이다.	(13.2, p.276-277 / PDF p.273-274)
U13-009	원문의 scoped_lock은 생성 시 mutex를 얻고, return·함수 끝·예외 등 어떤 방식으로 스코프를 벗어나도 소멸자로 해제한다.	(13.2, p.276-277 / PDF p.273-274)
U13-010	vector·map·string·iostream의 저장 공간이나 파일 핸들도 같은 원리에 따라 관리된다.	(13.2, p.276-277 / PDF p.273-274)
U13-011	<memory>의 unique_ptr은 유일한 소유권을, shared_ptr은 공유된 소유권을 표현한다.	(13.2.1, p.277-278 / PDF p.274-275)
U13-012	자유 저장소의 객체를 스마트 포인터에 맡기면 예외나 이른 반환에서 해제를 빠뜨리는 일을 막을 수 있다.	(13.2.1, p.277-278 / PDF p.274-275)
U13-013	다만 지역 객체나 컨테이너 같은 더 높은 수준의 자원 핸들로 표현할 수 있다면 그것을 먼저 사용한다.	(13.2.1, p.277-278 / PDF p.274-275)

번호	문장·표 항목·원문 코드·수식	원문 위치

U13-014	내장 포인터와 unique_ptr의 해제 비교 (원문 코드 발췌)
	void f(int i, int j) // X* vs. unique_ptr<X>
	{
	X* p = new X; // 새로운 X 할당
	unique_ptr<X> sp {new X}; // 새로운 X를 할당하고, 그 포인터를 unique_ptr에 넘겨줌
	// ...
	
	if (i<99) throw Z{}; // 여기서 예외를 던지거나
	if (j<77) return; // "일찍" 반환될 수 있음
	// ... p와 sp 사용 ..
	delete p; // *p 소멸
	}
		(13.2.1, p.278 / PDF p.275)

U13-015	unique_ptr은 개별 객체나 배열의 핸들이며 소유권 전달에 이동을 사용한다.	(13.2.1, p.278-279 / PDF p.275-276)
U13-016	shared_ptr은 복사하여 소유권을 나누고 마지막 소유자가 소멸할 때 자원을 해제한다.	(13.2.1, p.278-279 / PDF p.275-276)
U13-017	함수에 shared_ptr를 넘기면 함수가 종료된 뒤에도 복사본이 남을 수 있으므로, 자원 소멸 시점을 단순히 호출 함수의 끝으로 예상하면 안 된다.	(13.2.1, p.278-279 / PDF p.275-276)
U13-018	공유에는 추가 비용과 생애주기 예측의 어려움이 있어 꼭 공유해야 할 때만 사용한다.	(13.2.1, p.278-279 / PDF p.275-276)
U13-019	make_unique와 make_shared는 객체 생성과 포인터 인계를 함께 처리해 잘못된 포인터 전달이나 인계 누락을 줄인다.	(13.2.1, p.280-281 / PDF p.277-278)
U13-020	원문은 make_shared가 소유자 수를 세기 위한 별도 할당을 줄일 수 있다는 이점도 설명한다.	(13.2.1, p.280-281 / PDF p.277-278)
U13-021	포인터가 필요한 대표적인 경우는 다형적인 객체를 참조하거나 객체를 공유하는 경우다.	(13.2.1, p.280-281 / PDF p.277-278)
U13-022	객체 컬렉션 반환은 컨테이너로 처리할 수 있다.	(13.2.1, p.280-281 / PDF p.277-278)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-023	스마트 포인터는 여전히 포인터다.	(13.2.1, p.280 / PDF p.277)
U13-024	특히 shared_ptr 자체는 소유자들의 읽기·쓰기 규칙을 명시하지 않으므로, 수명 관리만으로 자원 경합 같은 문제까지 해결되지는 않는다.	(13.2.1, p.280 / PDF p.277)
U13-025	이동과 복사의 선택은 대부분 암묵적이지만, 복사할 수 없는 unique_ptr을 다른 소유자로 넘길 때처럼 명시적인 이동이 필요할 수 있다.	(13.2.2, p.281-282 / PDF p.278-279)
U13-026	std::move() 자체가 자원을 이동시키는 것은 아니다.	(13.2.2, p.281-282 / PDF p.278-279)
U13-027	전달한 객체를 rvalue 참조로 변환해 이동 생성자나 이동 대입 연산이 선택되게 유도한다.	(13.2.2, p.281-282 / PDF p.278-279)

U13-028	이동을 이용한 swap (원문 코드 발췌)
	template <typename T>
	void swap(T& a, T& b)
	{
	T tmp {move(a)}; // T의 생성자에는 rvalue로 전달되므로 이동을 수행
	a = move(b); // T의 대입 연산에는 rvalue로 전달되므로 이동을 수행
	b = move(tmp); // T의 대입 연산에는 rvalue로 전달되므로 이동을 수행
	}
		(13.2.2, p.282 / PDF p.279)

U13-029	이동 후에도 원본 객체는 남는다.	(13.2.2, p.282-283 / PDF p.279-280)
U13-030	원문은 move한 문자열을 나중에 다시 인덱싱하는 위험을 들어, 확실하고 큰 성능 개선이 있을 때만 명시적으로 move를 사용하도록 한다.	(13.2.2, p.282-283 / PDF p.279-280)
U13-031	이동 후 상태는 일반적으로 정해지지 않지만 표준 라이브러리 타입은 소멸·대입 가능한 상태로 남는다고 설명한다.	(13.2.2, p.282-283 / PDF p.279-280)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-032	이 원문은 vector·string 같은 컨테이너를 이동 후 비어 있는 상태로 설명한다.	(13.2.2, p.282-283 / PDF p.279-280)
U13-033	완벽한 포워딩은 인자의 성격을 바꾸지 않고 다른 함수로 넘기는 것이다.	(13.2.2, p.283 / PDF p.280)
U13-034	std::forward()는 lvalue와 rvalue의 차이를 처리한다는 점에서 move와 구별된다.	(13.2.2, p.283 / PDF p.280)
U13-035	포워딩할 때는 std::forward를 사용하고 같은 객체를 두 번 포워딩하지 않는다.	(13.2.2, p.283 / PDF p.280)
U13-036	포워딩 뒤에 그 객체를 다시 사용하지 않도록 주의한다.	(13.2.2, p.283 / PDF p.280)
U13-037	포인터와 요소 수를 따로 전달하는 인터페이스는 두 값이 실제 배열과 일치한다는 관례에 의존해 구간 에러를 일으키기 쉽다.	(13.3, p.284-285 / PDF p.281-282)
U13-038	gsl::span은 요소 시퀀스의 포인터와 길이를 함께 표현하는 구간 참조다.	(13.3, p.284-285 / PDF p.281-282)
U13-039	vector나 내장 배열 등의 연속 요소를 가리키며, 그 요소를 소유하지 않는다.	(13.3, p.284-285 / PDF p.281-282)
U13-040	string_view나 반복자 쌍과 유사하지만 문자 읽기 전용에 한정되지 않는다.	(13.3, p.284-285 / PDF p.281-282)
U13-041	배열에서 span을 직접 만들면 컴파일러가 요소 수를 세므로 간단하고 안전하다.	(13.3, p.285-286 / PDF p.282-283)
U13-042	별도로 포인터·길이로 span을 만들 때는 그 선택이 명시적으로 드러난다.	(13.3, p.285-286 / PDF p.282-283)
U13-043	함수끼리 span을 넘기면 포인터와 개수를 각각 넘기는 것보다 단순하다.	(13.3, p.285-286 / PDF p.282-283)
U13-044	인덱싱의 구간 검사는 에러 시 gsl::fail_fast를 던진다고 원문이 설명하며, 성능이 중요한 코드에서는 검사를 막을 수 있다고 한다.	(13.3, p.285-286 / PDF p.282-283)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-045	이 절의 span은 원문 시점에 표준으로 제안된 GSL 타입이다.	(13.3, p.284-286 / PDF p.281-283)
U13-046	원문은 향후 std::span의 구간 에러 처리에 계약을 사용하기를 기대한다고 쓴다.	(13.3, p.284-286 / PDF p.281-283)
U13-047	이 전망을 현재의 표준 사실로 바꾸어 보충하지 않는다.	(13.3, p.284-286 / PDF p.281-283)
U13-048	문자 구간에는 gsl::string_span을 제시한다.	(13.3, p.284-286 / PDF p.281-283)
U13-049	T[N] / array<T,N>	(13.4, p.287 / PDF p.284)
U13-050	연속 저장된 T 요소 N개의 고정 크기 시퀀스; array는 내장 배열의 여러 문제를 줄임	(13.4, p.287 / PDF p.284)
U13-051	bitset<N> / vector<bool>	(13.4, p.287 / PDF p.284)
U13-052	N비트의 고정 시퀀스 / vector 특수화로 저장한 비트 시퀀스	(13.4, p.287 / PDF p.284)
U13-053	pair<T,U> / tuple<T...>	(13.4, p.287 / PDF p.284)
U13-054	두 요소 / 임의의 여러 타입을 가진 요소 시퀀스	(13.4, p.287 / PDF p.284)
U13-055	basic_string<C> / valarray<T>	(13.4, p.287 / PDF p.284)
U13-056	문자열 연산이 있는 문자 시퀀스 / 수치 연산을 제공하는 수치 배열	(13.4, p.287 / PDF p.284)
U13-057	여러 컨테이너는 서로 다르거나 겹치는 요구를 만족한다.	(13.4, p.287 / PDF p.284)
U13-058	pair와 tuple은 이형이며, 다른 컨테이너는 같은 타입의 요소를 담는 동형이다.	(13.4, p.287 / PDF p.284)
U13-059	array·vector·tuple의 요소는 연속 할당되고 forward_list·map은 연결 구조다.	(13.4, p.287 / PDF p.284)
U13-060	정리 범위 제한: 제공 자료에는 책 288~289쪽이 없다.	(13.4.1, p.287,290 / PDF p.284-285)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-061	array 설명의 앞부분을 확인할 수 없으므로 287쪽 표, 290쪽의 이어지는 설명, 313쪽 조언에서 확인되는 내용만 정리한다.	(13.4.1, p.287,290 / PDF p.284-285)
U13-062	array는 크기를 알고 있어 표준 알고리즘과 함께 쓸 수 있고 =로 복사할 수 있다.	(13.4.1, p.290 / PDF p.285)
U13-063	원문이 강조하는 장점은 내장 배열에서 생길 수 있는 예기치 않은 포인터 변환을 막는다는 것이다.	(13.4.1, p.290 / PDF p.285)
U13-064	Circle 배열을 Shape*로 받아 인덱싱하면 sizeof(Shape)<sizeof(Circle)이라는 원문의 전제하에 잘못된 오프셋이 생기지만 array는 그 포인터 변환을 허용하지 않는다. 290쪽의 이어지는 문장은 스택이 제한된 자원이며 스택 오버플로우가 생길 수 있다는 주의도 포함한다.	(13.4.1, p.290 / PDF p.285)
U13-065	bitset<N>은 컴파일 시간에 정한 N비트의 구간에 연산을 제공한다.	(13.4.2, p.290-291 / PDF p.285-286)
U13-066	참/거짓·켜짐/꺼짐 등의 플래그 집합을 나타내며, 내장 정수에 담기 어려운 비트 집합을 다루는 데 편리하다.	(13.4.2, p.290-291 / PDF p.285-286)
U13-067	비트에 번호 대신 이름을 붙여야 한다면 원문은 set이나 열거형을 권한다.	(13.4.2, p.290-291 / PDF p.285-286)
U13-068	정수나 문자열로 초기화할 수 있고 비트별 연산과 시프트를 제공한다.	(13.4.2, p.291-292 / PDF p.286-287)
U13-069	시프트는 0을 밀어 넣는다.	(13.4.2, p.291-292 / PDF p.286-287)
U13-070	to_ullong()과 to_string()은 생성자의 반대 방향으로 변환한다.	(13.4.2, p.291-292 / PDF p.286-287)
U13-071	출력은 높은 자릿수의 비트가 왼쪽에 오며 bitset 자체의 출력 연산자도 사용할 수 있다.	(13.4.2, p.291-292 / PDF p.286-287)

번호	문장·표 항목·원문 코드·수식	원문 위치

U13-072	정수를 bitset 문자열로 출력 (원문 코드 발췌)
	void binary(int i)
	{
	bitset<8*sizeof(int)> b = i; // 8비트 바이트를 가정(14.7절 참고)
	cout << b.to_string() << '\n'; // i를 비트로 출력
	}
		(13.4.2, p.291 / PDF p.286)

U13-073	값들의 모음 자체가 필요하면 간단한 struct 또는 pair·tuple을 사용할 수 있다.	(13.4.3, p.292-293 / PDF p.287-288)
U13-074	pair는 first와 second라는 일관된 멤버 이름을 가진 두 값의 묶음이다.	(13.4.3, p.292-293 / PDF p.287-288)
U13-075	equal_range는 정렬된 입력에서 같은 값에 해당하는 부분 구간을 두 반복자의 pair로 반환한다.	(13.4.3, p.292-293 / PDF p.287-288)
U13-076	요소가 연산을 지원하면 pair의 대입·비교도 가능하다.	(13.4.3, p.292-293 / PDF p.287-288)
U13-077	타입 추론이나 make_pair로 중복된 타입 명시를 줄일 수 있다.	(13.4.3, p.293-294 / PDF p.288-289)
U13-078	둘보다 많거나 적은 요소에는 tuple을 사용한다.	(13.4.3, p.293-294 / PDF p.288-289)
U13-079	원문은 생성자 인자에서의 타입 추론이 C++17부터 제공되며 이전 코드에서는 make_tuple을 사용할 수 있다고 설명한다.	(13.4.3, p.293-294 / PDF p.288-289)

U13-080	tuple 생성과 타입 추론 (원문 코드 발췌)
	tuple<string,int,double> t1 {"Shark",123,3.14}; // 타입을 명시적으로 지정
	auto t2 = make_tuple(string{"Herring"},10,1.23); // 타입을
	// tuple<string,int,double>로 추론
	tuple t3 {"Cod"s,20,9.99}; // 타입을 tuple<string,int,double>로 추론
		(13.4.3, p.294 / PDF p.289)

U13-081	get<0>()처럼 0부터 시작하는 상수 인덱스로 tuple 요소에 접근한다.	(13.4.3, p.294-295 / PDF p.289-290)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-082	그 타입이 tuple 안에서 유일할 때는 get<string>()처럼 타입으로도 지정할 수 있고 쓰기에도 get을 사용한다.	(13.4.3, p.294-295 / PDF p.289-290)
U13-083	pair도 get으로 접근 가능하고 pair·tuple 모두 구조화된 바인딩을 사용할 수 있다.	(13.4.3, p.294-295 / PDF p.289-290)
U13-084	제네릭할 필요가 없다면 이름 있는 멤버를 둔 struct가 더 읽기 쉽고 유지보수하기 좋을 수 있다.	(13.4.3, p.294-295 / PDF p.289-290)
U13-085	표준 라이브러리는 지정한 선택지 중 하나를 담는 variant, 특정 타입의 값 또는 값 없음을 표현하는 optional, 타입 집합에 제한이 없는 any를 제공한다.	(13.5, p.295 / PDF p.290)
U13-086	각각 <variant>·<optional>·<any>에 정의되며, 관련 기능을 표현하지만 인터페이스가 통일되어 있지는 않다.	(13.5, p.295 / PDF p.290)
U13-087	variant<A,B,C>는 union을 직접 사용하는 것보다 안전하고 편리한 선택지다.	(13.5.1, p.295-296 / PDF p.290-291)
U13-088	대입·초기화한 값의 타입을 기억하며 사용자는 어떤 선택지가 저장되었는지 질의하고 값을 추출한다.	(13.5.1, p.295-296 / PDF p.290-291)
U13-089	원문은 문자열 또는 에러 코드를 반환하는 경우와, 컴파일러 노드의 Expression·Statement·Declaration·Type을 구분하는 경우를 든다.	(13.5.1, p.295-296 / PDF p.290-291)
U13-090	저장 타입별로 여러 번 검사하는 대신 visit과 오버로드된 함수 집합으로 행동을 선택할 수 있다.	(13.5.1, p.297-298 / PDF p.292-293)
U13-091	원문의 overloaded는 람다 등으로부터 오버로드 집합을 만드는 보조 타입이며 표준 기능 자체는 아니라고 명시한다.	(13.5.1, p.297-298 / PDF p.292-293)
U13-092	추론 가이드도 이 구현에 사용된다.	(13.5.1, p.297-298 / PDF p.292-293)
U13-093	visit 이후 오버로드 규칙으로 가장 적절한 람다가 호출된다.	(13.5.1, p.297-298 / PDF p.292-293)
U13-094	잘못된 타입으로 접근하면 bad_variant_access를 던진다.	(13.5.1, p.297-298 / PDF p.292-293)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-095	가상 호출보다 빠를 가능성은 실측해야 한다.	(13.5.1, p.297-298 / PDF p.292-293)
U13-096	optional<A>는 A 또는 아무것도 없음을 나타낸다.	(13.5.2, p.298-299 / PDF p.293-294)
U13-097	객체를 반환하거나 반환하지 않는 함수에 유용하며 빈 {}가 값 없음에 해당한다.	(13.5.2, p.298-299 / PDF p.293-294)
U13-098	조건 검사로 값 유무를 살피고 *로 저장 값에 접근하므로 포인터와 비슷한 사용 형태를 가진다.	(13.5.2, p.298-299 / PDF p.293-294)

U13-099	값이 있는 optional만 합산 (원문 코드 발췌)
	int cat(optional<int> a, optional<int> b)
	{
	int res = 0;
	if (a) res+=*a;
	if (b) res+=*b;
	return res;
	}
	
	int x = cat(17,19);
	int y = cat(17,{});
	int z = cat({},{});
		(13.5.2, p.299 / PDF p.294)

U13-100	원문은 값이 없는 optional에 접근하면 동작을 예측할 수 없고 예외도 던지지 않으므로, 이 접근 방식에서 타입 안전성을 보장하지 않는다고 경고한다.	(13.5.2, p.299 / PDF p.294)
U13-101	따라서 예제처럼 먼저 존재 여부를 검사하는 조건을 유지한다.	(13.5.2, p.299 / PDF p.294)
U13-102	any는 임의의 타입을 저장하며 저장된 객체가 있으면 그 타입을 기억한다.	(13.5.3, p.299-300 / PDF p.294-295)
U13-103	제약을 제거한 variant처럼 설명할 수 있다.	(13.5.3, p.299-300 / PDF p.294-295)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-104	any_cast로 타입을 지정하여 값을 추출한다.	(13.5.3, p.299-300 / PDF p.294-295)
U13-105	원문은 저장된 타입과 다른 타입으로 접근하면 bad_any_access를 던진다고 표기하고, 예외에 의존하지 않는 접근 방법도 있다고 언급한다.	(13.5.3, p.299-300 / PDF p.294-295)
U13-106	[원문 확인 필요] 여기서 예외 이름 bad_any_access는 제공된 책 300쪽의 표기다.	(13.5.3, p.300 / PDF p.295)
U13-107	이 노트에서는 의심되는 명칭을 다른 이름으로 조용히 바꾸지 않았다.	(13.5.3, p.300 / PDF p.295)
U13-108	컨테이너의 기본 할당은 new·delete를 사용하지만, 특별한 성능·보안·스레드별 할당·메모리 구조 요구가 있으면 할당자를 지정할 수 있다.	(13.6, p.300 / PDF p.295)
U13-109	원문은 고급 할당자 전반 대신 이벤트 큐에서 풀 할당자로 단편화를 줄인 실제 사례를 제시한다.	(13.6, p.300 / PDF p.295)
U13-110	원래 시스템은 vector를 가진 이벤트를 shared_ptr로 전달하여 마지막 사용자가 자동 소멸시키는 단순한 구조였지만 단편화로 많은 메모리를 소비했다.	(13.6, p.301-302 / PDF p.296-297)
U13-111	풀 할당자는 같은 크기의 여러 객체를 한꺼번에 할당해 관리한다.	(13.6, p.301-302 / PDF p.296-297)
U13-112	C++17의 pmr 하위 네임스페이스에는 다형성 메모리 자원과 synchronized_pool_resource가 있어 컨테이너를 전면 교체하지 않고 할당자를 바꿀 수 있다.	(13.6, p.301-302 / PDF p.296-297)
U13-113	원문 사례에서 프로듀서 16개와 컨슈머 4개가 이벤트 100,000개를 전달한 뒤 사용 메모리가 6GB에서 3MB 미만으로 줄었다.	(13.6, p.301-302 / PDF p.296-297)
U13-114	이는 단편화 낭비를 줄인 결과이며 실제로 필요했던 메모리양이 바뀐 것은 아니다.	(13.6, p.301-302 / PDF p.296-297)
U13-115	이 수치는 해당 사례의 관측값으로만 이해한다.	(13.6, p.301-302 / PDF p.296-297)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-116	<chrono>의 clock::now()는 시각인 time_point를 반환하고 두 시각의 차이는 시간 구간인 duration이 된다.	(13.7, p.302-303 / PDF p.297-298)
U13-117	시계별로 단위가 다르므로 duration_cast로 적절한 단위에 맞춰 표현한다.	(13.7, p.302-303 / PDF p.297-298)
U13-118	성능을 판단할 때는 추측 대신 실제 시간을 측정한다.	(13.7, p.302-303 / PDF p.297-298)

U13-119	chrono로 작업 시간 측정 (원문 코드 발췌)
	using namespace std::chrono; // 하위 네임스페이스 std::chrono; 3.4절 참고
	
	auto t0 = high_resolution_clock::now();
	do_work();
	auto t1 = high_resolution_clock::now();
	cout << duration_cast<milliseconds>(t1-t0).count() << " msec\n";
		(13.7, p.302 / PDF p.297)

U13-120	시간 단위 접미사는 std::chrono_literals에 정의되어 10ms·33us 같은 표현을 가능하게 한다.	(13.7, p.303 / PDF p.298)
U13-121	원문은 년·월·달력·시간대·윤초를 다루는 확장 기능을 C++20 표준에 추가 중이라고 소개한다.	(13.7, p.303 / PDF p.298)
U13-122	이는 원문 시점의 상태이며 현재 상황으로 보충하지 않는다.	(13.7, p.303 / PDF p.298)
U13-123	함수를 다른 함수의 인자로 넘길 때 호출 형태나 타입이 정확히 맞지 않으면 람다, mem_fn, function을 이용하는 세 방법을 고려한다.	(13.8, p.303-304 / PDF p.298-299)
U13-124	표준 알고리즘은 f(x) 형태로 인자를 호출하지만 멤버 함수는 x->f() 형태를 사용한다.	(13.8.1, p.304 / PDF p.299)
U13-125	람다는 이 호출 방식의 차이를 연결할 수 있다.	(13.8.1, p.304 / PDF p.299)

번호	문장·표 항목·원문 코드·수식	원문 위치

U13-126	람다로 멤버 함수 호출 연결 (원문 코드 발췌)
	void draw_all(vector<Shape*>& v)
	{
	for_each(v.begin(),v.end(),[](Shape* p) { p->draw(); });
	}
		(13.8.1, p.304 / PDF p.299)

U13-127	mem_fn(mf)는 멤버 함수 mf를 비멤버 함수처럼 호출할 수 있는 함수 객체로 바꾼다.	(13.8.2, p.304-305 / PDF p.299-300)
U13-128	원문은 mem_fn(&Shape::draw)를 for_each에 넘겨 람다와 같은 연결을 수행한다.	(13.8.2, p.304-305 / PDF p.299-300)
U13-129	람다 도입 이전에 객체지향 호출과 함수 스타일 호출을 연결하던 대표 수단이다.	(13.8.2, p.304-305 / PDF p.299-300)
U13-130	function 객체는 ()로 호출할 수 있는 대상을 저장한다.	(13.8.3, p.305 / PDF p.300)
U13-131	함수·람다를 넣고 그 호출 타입을 명시하거나 초기 값에서 추론할 수 있다.	(13.8.3, p.305 / PDF p.300)
U13-132	콜백이나 연산을 인자로 전달하는 데 유용하지만 직접 함수 호출보다 실행 비용이 있고, function은 객체이므로 자체에 오버로딩을 적용할 수 없다.	(13.8.3, p.305 / PDF p.300)
U13-133	함수 객체의 오버로딩에는 앞선 overloaded를 고려한다.	(13.8.3, p.305 / PDF p.300)
U13-134	타입 함수는 타입 자체를 인자로 받거나 타입을 반환하며 컴파일 시간에 평가된다.	(13.9, p.306 / PDF p.301)
U13-135	numeric_limits는 수치 타입 정보를, sizeof는 객체의 바이트 수를 알려준다.	(13.9, p.306 / PDF p.301)
U13-136	이런 기능은 컴파일 시간 계산을 통한 타입 검사와 성능 향상의 재료다.	(13.9, p.306 / PDF p.301)
U13-137	관련 기법을 메타프로그래밍, 템플릿과 관련되면 템플릿 메타프로그래밍이라고 한다.	(13.9, p.306 / PDF p.301)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-138	원문은 아직 컨셉이 범용 표준이 아닌 시점이어서 타입 함수가 널리 쓰인다고 설명한다.	(13.9, p.306 / PDF p.301)
U13-139	표준 sort는 임의 접근 반복자를 요구하지만 forward_list는 순방향 반복자만 제공한다.	(13.9.1, p.306-309 / PDF p.301-304)
U13-140	원문은 임의 접근 구간에는 바로 sort를, 순방향 구간에는 vector로 복사·정렬·원래 구간으로 재복사를 선택하는 sort_helper 두 버전을 제시한다.	(13.9.1, p.306-309 / PDF p.301-304)
U13-141	반복자 종류를 나타내는 태그로 적절한 오버로드를 컴파일 시간에 선택하는 기법이 태그 디스패치다.	(13.9.1, p.306-309 / PDF p.301-304)

U13-142	포인터용 iterator_traits (원문 코드 발췌)
	template<class T>
	struct iterator_traits<T*> {
	 using difference_type = ptrdiff_t;
	 using value_type = T;
	 using pointer = T*;
	 using reference = T&;
	 using iterator_category = random_access_iterator_tag;
	};
		(13.9.1, p.309 / PDF p.304)

U13-143	컨테이너의 iterator·value_type 멤버는 타입 별칭으로 표현할 수 있다.	(13.9.1, p.308-310 / PDF p.303-305)
U13-144	포인터처럼 이런 멤버 타입이 없는 경우에는 <iterator>의 iterator_traits 특수화를 통해 값 타입·참조·포인터·반복자 종류를 얻는다.	(13.9.1, p.308-310 / PDF p.303-305)
U13-145	따라서 int*도 random_access_iterator_tag를 가진 반복자로 다룰 수 있다.	(13.9.1, p.308-310 / PDF p.303-305)
U13-146	원문은 같은 구분이 컨셉으로 더 직접 표현될 수 있고 표준화가 진행 중이라고 설명한다.	(13.9.1, p.308-310 / PDF p.303-305)
U13-147	<type_traits>는 is_arithmetic·is_class·is_pod·is_literal_type·has_virtual_destructor·is_base_of 같은 타입 술어를 제공한다.	(13.9.2, p.310-311 / PDF p.305-306)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-148	템플릿은 이 값으로 타입의 성질을 검사할 수 있다.	(13.9.2, p.310-311 / PDF p.305-306)
U13-149	원문은 산술 타입만 받는 complex의 static_assert를 예로 들고, std::is_arithmetic<T>::value를 반환하는 Is_arithmetic() 래퍼로 표기를 간단히 한다.	(13.9.2, p.310-311 / PDF p.305-306)

U13-150	타입 술어의 표기를 단순화한 래퍼 (원문 코드 발췌)
	template<typename T>
	constexpr bool Is_arithmetic()
	{
	return std::is_arithmetic<T>::value;
	}
		(13.9.2, p.311 / PDF p.306)

U13-151	타입 술어는 static_assert·컴파일 시간 if·enable_if의 조건으로 사용된다.	(13.9.3, p.311-312 / PDF p.306-307)
U13-152	enable_if는 조건에 따라 정의를 선택하는 데 널리 쓰인다.	(13.9.3, p.311-312 / PDF p.306-307)
U13-153	원문의 스마트 포인터 설명은 T가 클래스일 때에만 화살표 연산을 제공하려는 의도를 든다.	(13.9.3, p.311-312 / PDF p.306-307)
U13-154	함수 바깥에서는 컴파일 시간 if를 쓸 수 없다는 문맥에서 enable_if를 소개한다.	(13.9.3, p.311-312 / PDF p.306-307)
U13-155	enable_if는 표기가 낯설고 많은 경우 컨셉으로 대체될 수 있지만, 원문 시점의 템플릿 메타프로그래밍과 여러 표준 라이브러리 구성 요소의 기반이다.	(13.9.3, p.312 / PDF p.307)
U13-156	이 기법은 SFINAE(Substitution Failure Is Not An Error) 언어 기능에 의존한다.	(13.9.3, p.312 / PDF p.307)
U13-157	자원에는 RAII 자원 핸들을 사용한다.	(13.10, p.312-314 / PDF p.307-309)
U13-158	포인터가 필요하면 unique_ptr를 우선하고, 소유권을 공유해야 할 때만 shared_ptr를 쓴다.	(13.10, p.312-314 / PDF p.307-309)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-159	생성에는 make_unique·make_shared를 사용한다.	(13.10, p.312-314 / PDF p.307-309)
U13-160	move를 남용하지 않으며 forward는 포워딩에만 사용한다.	(13.10, p.312-314 / PDF p.307-309)
U13-161	이동·포워딩한 객체를 나중에 읽지 않는다.	(13.10, p.312-314 / PDF p.307-309)
U13-162	포인터와 개수를 별도로 전달하기보다 span을 사용한다.	(13.10, p.312-314 / PDF p.307-309)
U13-163	고정 크기에는 array, N비트에는 bitset을 고려한다.	(13.10, p.312-314 / PDF p.307-309)
U13-164	pair·tuple보다 이름 있는 struct가 읽기 쉬운 경우를 구별한다.	(13.10, p.312-314 / PDF p.307-309)
U13-165	union을 직접 쓰기보다 variant를 고려한다.	(13.10, p.312-314 / PDF p.307-309)
U13-166	단편화에는 할당자를 고려하고, 효율성은 실제 시간 측정으로 판단한다.	(13.10, p.312-314 / PDF p.307-309)
U13-167	duration은 단위를 명시하고 필요하면 duration_cast로 바꾼다.	(13.10, p.312-314 / PDF p.307-309)
U13-168	호출을 연결할 때 람다·mem_fn을, 호출 대상을 저장할 때 function을 쓴다.	(13.10, p.312-314 / PDF p.307-309)
U13-169	타입 특성·enable_if·컨셉과 타입 별칭의 역할을 구분한다.	(13.10, p.312-314 / PDF p.307-309)
U13-170	소유권	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-171	유일한 소유자	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-172	여러 소유자가 공유	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-173	전달	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-174	이동; 복사할 수 없음	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-175	복사하여 공유	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-176	자원 해제	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-177	소유 핸들의 수명에 연결	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-178	마지막 소유자가 소멸할 때	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-179	선택	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-180	포인터가 필요하고 소유자가 하나일 때 우선	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)
U13-181	공유 소유권이 실제로 필요할 때	(스마트 포인터의 소유권, p.277-281 / PDF p.274-278)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-182	variant	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-183	미리 지정한 여러 타입 중 하나	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-184	타입 질의·get·visit	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-185	optional	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-186	한 타입의 값 또는 없음	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-187	존재 검사 후 *	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-188	any	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-189	미리 제한하지 않은 타입	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-190	any_cast로 지정 타입 추출	(세 가지 선택지 타입, p.295-300 / PDF p.290-295)
U13-191	gsl::span은 요소 시퀀스의 포인터와 길이를 함께 표현하는 구간 참조이며, 그 요소를 소유하지 않는다.	(13.3, p.284,286 / PDF p.281,283)

번호	문장·표 항목·원문 코드·수식	원문 위치
U13-192	자원에는 RAII 자원 핸들을 사용한다. 포인터가 필요하면 unique_ptr를 우선하고, 소유권을 공유해야 할 때만 shared_ptr를 쓴다. 생성에는 make_unique·make_shared를 사용한다.	(13.10, p.312-314 / PDF p.307-309)
U13-193	move를 남용하지 않으며 forward는 포워딩에만 사용한다. 이동·포워딩한 객체를 나중에 읽지 않는다.	(13.10, p.312-314 / PDF p.307-309)
U13-194	포인터와 개수를 별도로 전달하기보다 span을 사용한다. 고정 크기에는 array, N비트에는 bitset을 고려한다.	(13.10, p.312-314 / PDF p.307-309)
U13-195	pair·tuple보다 이름 있는 struct가 읽기 쉬운 경우를 구별한다. union을 직접 쓰기보다 variant를 고려한다.	(13.10, p.312-314 / PDF p.307-309)
U13-196	optional<A>	(13.5.2, p.298-299 / PDF p.293-294)
U13-197	any	(13.5.3, p.299-300 / PDF p.294-295)

제14장. 수치

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-001	<numeric>은 시퀀스의 합·내적·부분합·이웃한 차이 등을 일반화한다.	(14.3, p.318-319 / PDF p.313-314)
U14-002	reduce는 accumulate와 달리 요소 연산의 순서가 정해지지 않는다.	(14.3.1, p.319-320 / PDF p.314-315)
U14-003	수치 계산은 데이터베이스·네트워킹·장비 제어·그래픽스·시뮬레이션·금융 분석 등 큰 시스템의 일부로 쓰인다.	(14.1, p.316 / PDF p.311)
U14-004	C++는 계산 과정에 복잡한 자료 구조가 함께 필요할 때 장점을 가진다.	(14.1, p.316 / PDF p.311)
U14-005	이 장은 그런 작업을 지원하는 표준 라이브러리 수치 기능을 정리한다.	(14.1, p.316 / PDF p.311)
U14-006	<cmath>는 float·double·long double에 대한 표준 수학 함수를 제공한다.	(14.2, p.316-317 / PDF p.311-312)
U14-007	대응하는 복소수 버전은 <complex>에서 찾을 수 있고 반환 타입은 인자 타입과 같다.	(14.2, p.316-317 / PDF p.311-312)
U14-008	abs / ceil / floor	(14.2, p.316-317 / PDF p.311-312)
U14-009	절댓값 / x 이상인 가장 작은 정수 / x 이하인 가장 큰 정수	(14.2, p.316-317 / PDF p.311-312)
U14-010	sqrt	(14.2, p.316-317 / PDF p.311-312)
U14-011	제곱근; x는 음수가 아니어야 함	(14.2, p.316-317 / PDF p.311-312)
U14-012	cos / sin / tan	(14.2, p.316-317 / PDF p.311-312)
U14-013	코사인 / 사인 / 탄젠트	(14.2, p.316-317 / PDF p.311-312)
U14-014	acos / asin / atan	(14.2, p.316-317 / PDF p.311-312)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-015	역코사인 / 역사인 / 역탄젠트; acos는 음이 아닌 결과, asin은 0에서 가장 가까운 결과	(14.2, p.316-317 / PDF p.311-312)
U14-016	sinh / cosh / tanh	(14.2, p.316-317 / PDF p.311-312)
U14-017	하이퍼볼릭 사인 / 코사인 / 탄젠트	(14.2, p.316-317 / PDF p.311-312)
U14-018	exp / log / log10	(14.2, p.316-317 / PDF p.311-312)
U14-019	밑 e의 지수 / 자연 로그 / 밑 10의 로그; log의 x는 양수	(14.2, p.316-317 / PDF p.311-312)
U14-020	원문은 <cerrno>의 errno로 에러를 나타내며 EDOM은 정의역 에러, ERANGE는 구간 에러라고 설명한다.	(14.2, p.317 / PDF p.312)
U14-021	예제는 먼저 errno를 0으로 지운 뒤 음수의 sqrt와 너무 큰 pow 결과를 검사한다.	(14.2, p.317 / PDF p.312)
U14-022	<cstdlib>에도 몇 가지 수학 함수가 있고, <cmath>에는 beta·riemann_zeta·sph_bessel 같은 특수 수학 함수도 있다.	(14.2, p.317 / PDF p.312)
U14-023	각 알고리즘에는 흔히 쓰는 기본 연산 버전과, 적용 연산을 인자로 받는 일반화 버전이 있다.	(14.3, p.318-319 / PDF p.313-314)
U14-024	모든 표준 라이브러리 시퀀스에 적용할 수 있다.	(14.3, p.318-319 / PDF p.313-314)
U14-025	accumulate(b,e,i) / accumulate(b,e,i,f)	(14.3, p.318 / PDF p.313)
U14-026	초기 값 i를 바탕으로 합산 / + 대신 f 사용	(14.3, p.318 / PDF p.313)
U14-027	inner_product	(14.3, p.318 / PDF p.313)
U14-028	두 시퀀스의 대응 요소 곱을 합산하고 초기 값에 더함; +·*를 별도 연산으로 바꾸는 버전 제공	(14.3, p.318 / PDF p.313)
U14-029	partial_sum	(14.3, p.318 / PDF p.313)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-030	각 위치까지의 부분합; + 대신 f를 지정할 수 있음	(14.3, p.318 / PDF p.313)
U14-031	adjacent_difference	(14.3, p.318 / PDF p.313)
U14-032	이웃한 요소의 차이; 비어 있지 않으면 첫 출력은 첫 입력 값; - 대신 f 지정 가능	(14.3, p.318 / PDF p.313)
U14-033	iota	(14.3, p.318 / PDF p.313)
U14-034	연속적인 값으로 구간을 채우는 기능; 원문 표는 각 요소에 ++v를 대입한다고 표기	(14.3, p.318 / PDF p.313)
U14-035	gcd / lcm	(14.3, p.318 / PDF p.313)
U14-036	정수의 최대공약수 / 최소공배수	(14.3, p.318 / PDF p.313)
U14-037	accumulate의 초기 값과 합산 (원문 코드 발췌) list<double> lst {1, 2, 3, 4, 5, 9999.99999}; auto s = accumulate(lst.begin(),lst.end(),0.0); // 합 계산: 10014.9999	(14.3, p.318 / PDF p.313)
U14-038	[원문 확인 필요] 318쪽 iota 표는 ++v와 v+1, v+2, ...로 설명한다.	(14.3, p.318 / PDF p.313)
U14-039	이 노트는 의심되는 시작 값 표기를 임의로 고치거나 다른 실행 결과를 만들지 않았다.	(14.3, p.318 / PDF p.313)
U14-040	<numeric>의 병렬 수치 알고리즘은 기존 함수와 비슷하지만 일부 의미가 다르다.	(14.3.1, p.319-320 / PDF p.314-315)
U14-041	초기 값과 실행 정책을 주는 형태, 기본 값을 사용하는 형태가 제공된다.	(14.3.1, p.319-320 / PDF p.314-315)
U14-042	reduce	(14.3.1, p.319 / PDF p.314)
U14-043	순서가 정해지지 않는 누적	(14.3.1, p.319 / PDF p.314)
U14-044	exclusive_scan / inclusive_scan	(14.3.1, p.319 / PDF p.314)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-045	i번째 합에서 i번째 입력 제외 / 포함	(14.3.1, p.319 / PDF p.314)
U14-046	transform_reduce	(14.3.1, p.319 / PDF p.314)
U14-047	각 요소에 f를 적용한 뒤 reduce	(14.3.1, p.319 / PDF p.314)
U14-048	transform_exclusive_scan / transform_inclusive_scan	(14.3.1, p.319 / PDF p.314)
U14-049	각 요소에 f를 적용한 뒤 각각의 scan 실행	(14.3.1, p.319 / PDF p.314)

U14-050	reduce와 병렬 실행 정책 (원문 코드 발췌)
	vector<double> v {1, 2, 3, 4, 5, 9999.99999};
	auto s = reduce(v.begin(),v.end()); // double을 누적자로 사용해 합을 계산
	
	vector<double> large;
	// ... 많은 값들로 large를 채운다 ...
	auto s2 = reduce(par_unseq, large.begin(),large.end()); // 가능한 병렬화 정도로 합을 계산
		(14.3.1, p.320 / PDF p.315)

U14-051	일반 병렬 알고리즘과 마찬가지로 실행 정책을 지정할 수 있다.	(14.3.1, p.319-320 / PDF p.314-315)
U14-052	기본 연산 대신 별도의 연산을 받거나 기본 정책·기본 값을 쓰는 형태도 있다고 원문은 설명하며, 표에서는 편의상 일부를 생략했다.	(14.3.1, p.319-320 / PDF p.314-315)
U14-053	<complex>의 complex<Scalar>는 실수부와 허수부 타입을 매개변수로 받는 복소수 템플릿이다.	(14.4, p.320-321 / PDF p.315-316)
U14-054	원문은 float·double·long double에 대한 복소수 사용을 보여 준다.	(14.4, p.320-321 / PDF p.315-316)
U14-055	일반적인 산술 연산과 sqrt·pow 같은 수학 함수를 사용할 수 있고 생성자의 실수부·허수부에는 기본 인자가 제공된다.	(14.4, p.320-321 / PDF p.315-316)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-056	<random>의 난수 생성기는 엔진과 분포를 결합한다.	(14.5, p.321 / PDF p.316)
U14-057	엔진은 난수 또는 의사 난수 시퀀스를 만들고, 분포는 그 값을 수학적 분포의 특정 구간으로 매핑한다.	(14.5, p.321 / PDF p.316)
U14-058	원문은 같은 확률로 정수를 만드는 uniform_int_distribution, 종형 곡선의 normal_distribution, exponential_distribution을 예로 든다.	(14.5, p.321 / PDF p.316)
U14-059	엔진과 분포를 직접 조합한 주사위 예제에 이어 원문은 두 구성 요소를 Rand_int 클래스에 묶어 간단한 사용 인터페이스를 제시한다.	(14.5, p.321-322 / PDF p.316-317)
U14-060	이 클래스는 기본 난수 엔진, 정수 분포, 호출 연산자, 시드 설정 연산을 가진다.	(14.5, p.321-322 / PDF p.316-317)

U14-061	엔진과 분포를 묶은 Rand_int (원문 코드 발췌)
	class Rand_int {
	public:
	Rand_int(int low, int high) :dist{low,high} { }
	int operator()() { return dist(re); } // int 생성
	void seed(int s) { re.seed(s); } // 난수 엔진에 새로운 시드 선택
	private:
	default_random_engine re;
	uniform_int_distribution<> dist;
	};
		(14.5, p.322 / PDF p.317)

U14-062	원문의 히스토그램 예는 0부터 9까지의 값을 200번 생성하고 각 값의 빈도만큼 별표를 출력한다.	(14.5, p.322-323 / PDF p.317-318)
U14-063	결과는 통계적 변동이 있는 분포로 제시되므로 이 노트에서 새로운 난수값이나 고정 출력표를 만들지 않는다.	(14.5, p.322-323 / PDF p.317-318)
U14-064	조언에서는 충분한 무작위성을 확인하고 C의 rand()를 사용하지 않도록 권한다.	(14.5, p.322-323 / PDF p.317-318)
U14-065	일반 컨테이너 vector는 값을 저장하고 반복자·알고리즘 구조와 결합하는 데 초점을 두며 수학적인 벡터 연산을 직접 제공하지 않는다.	(14.6, p.323-324 / PDF p.318-319)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-066	<valarray>의 valarray는 일반성이 다소 낮은 대신 수치 계산 최적화에 더 적합하도록 제공되는 배열 템플릿이다.	(14.6, p.323-324 / PDF p.318-319)

U14-067	valarray의 배열 수준 산술 (원문 코드 발췌)
	void f(valarray<double>& a1, valarray<double>& a2)
	{
	valarray<double> a = a1*3.14+a2/a1; // 수치 배열 연산자 *, +, /, =
	a2 += a1*3.14;
	a = abs(a);
	double d = a2[7];
	// ...
	}
		(14.6, p.324 / PDF p.319)

U14-068	valarray는 배열 수준의 산술 연산과 일반적인 수학 함수, 요소 인덱싱을 제공한다.	(14.6, p.324 / PDF p.319)
U14-069	다차원 계산을 구현하기 위한 건너뛰기 접근(stride access)도 지원한다.	(14.6, p.324 / PDF p.319)
U14-070	원문은 연산·요소 타입의 유연성보다 실행 효율이 중요할 때의 선택으로 설명한다.	(14.6, p.324 / PDF p.319)
U14-071	<limits>의 numeric_limits는 내장 수치 타입의 속성을 기술한다.	(14.7, p.324 / PDF p.319)
U14-072	원문은 char의 부호 유무와 int가 필요한 큰 값을 표현할 수 있는지 static_assert로 검사한다.	(14.7, p.324 / PDF p.319)
U14-073	max()가 constexpr 함수이므로 컴파일 시간 조건으로 사용할 수 있다.	(14.7, p.324 / PDF p.319)
U14-074	수치 타입의 성질을 컴파일 시간에 검사 (원문 코드 발췌) static_assert(numeric_limits<char>::is_signed,"부호 없는 문자!"); static_assert(100000<numeric_limits<int>::max(),"작은 int!");	(14.7, p.324 / PDF p.319)
U14-075	수치 문제를 수학적으로 확신할 수 없다면 전문적인 조언과 실험을 활용한다.	(14.8, p.325 / PDF p.320)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-076	기본 언어 기능만으로 직접 만들기보다 라이브러리를 사용한다.	(14.8, p.325 / PDF p.320)
U14-077	시퀀스를 계산하는 루프 전에 accumulate·inner_product·partial_sum·adjacent_difference를 고려한다.	(14.8, p.325 / PDF p.320)
U14-078	복소수에는 std::complex를 사용한다.	(14.8, p.325 / PDF p.320)
U14-079	난수 생성은 엔진과 분포를 결합하고 무작위성에 주의를 기울인다.	(14.8, p.325 / PDF p.320)
U14-080	수치 효율에 우선순위가 있으면 valarray를 고려한다.	(14.8, p.325 / PDF p.320)
U14-081	numeric_limits로 수치 타입이 용도에 맞는지 검사한다.	(14.8, p.325 / PDF p.320)
U14-082	목적	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-083	일반적인 값 저장·컨테이너 구조	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-084	수치 계산 최적화	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-085	일반성	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-086	유연한 일반 컨테이너	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-087	일반성은 상대적으로 낮음	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-088	수치 배열 연산	(vector와 valarray, p.323-325 / PDF p.318-320)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-089	수학 벡터 연산을 직접 제공하지 않음	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-090	배열 산술·수학 함수·stride access	(vector와 valarray, p.323-325 / PDF p.318-320)
U14-091	구분	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-092	순차 수치 알고리즘	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-093	병렬 수치 알고리즘	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-094	적용 순서	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-095	순차 처리의 누적	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-096	요소 연산 순서가 정해지지 않음	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-097	정책	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-098	원문의 순차 예	(accumulate와 reduce, p.318-320 / PDF p.313-315)

번호	문장·표 항목·원문 코드·수식	원문 위치
U14-099	실행 정책 지정 가능	(accumulate와 reduce, p.318-320 / PDF p.313-315)
U14-100	엔진은 난수 또는 의사 난수 시퀀스를 만들고, 분포는 그 값을 수학적 분포의 특정 구간으로 매핑한다.	(14.5, p.321,323 / PDF p.316,318)
U14-101	수치 문제를 수학적으로 확신할 수 없다면 전문적인 조언과 실험을 활용한다. 기본 언어 기능만으로 직접 만들기보다 라이브러리를 사용한다.	(14.8, p.325 / PDF p.320)
U14-102	복소수에는 std::complex를 사용한다. 난수 생성은 엔진과 분포를 결합하고 무작위성에 주의를 기울인다.	(14.8, p.325 / PDF p.320)
U14-103	수치 효율에 우선순위가 있으면 valarray를 고려한다. numeric_limits로 수치 타입이 용도에 맞는지 검사한다.	(14.8, p.325 / PDF p.320)
U14-104	reduce	(14.3.1, p.319-320 / PDF p.314-315)
U14-105	valarray	(14.6, p.323-324 / PDF p.318-319)

제15장. 동시성

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-001	동시성은 여러 태스크를 동시에 실행하는 것이다.	(15.1, p.327-328 / PDF p.321-322)
U15-002	태스크는 다른 계산과 동시에 실행할 수 있는 작업이며, thread는 이를 표현하는 시스템 수준 개념이다.	(15.2, p.328-329 / PDF p.322-323)
U15-003	동시성은 여러 태스크를 동시에 실행하는 것으로, 여러 프로세서로 처리량을 늘리거나 대기 중에도 다른 일을 진행하여 반응성을 높이는 데 사용한다.	(15.1, p.327-328 / PDF p.321-322)
U15-004	원문의 표준 라이브러리는 정교한 고수준 모델보다는 시스템 수준 동시성을 이식성과 타입 안전성을 고려해 제공한다.	(15.1, p.327-328 / PDF p.321-322)
U15-005	C++는 한 주소 공간의 여러 스레드를 지원하며 메모리 모델과 원자적 연산을 제공한다.	(15.1, p.327-328 / PDF p.321-322)
U15-006	데이터 경합은 변경 가능한 데이터에 대한 무질서한 동시 접근이다.	(15.1, p.327-328 / PDF p.321-322)
U15-007	원자적 연산은 락 없는 프로그래밍의 기반이지만 원문은 이를 전문가에게 맡기도록 조언한다.	(15.1, p.327-328 / PDF p.321-322)
U15-008	동시성 자체가 해답은 아니며, 순차 해결이 더 간단하고 빠를 수 있다.	(15.1, p.327-328 / PDF p.321-322)
U15-009	직접 스레드를 관리하는 대신 병렬 알고리즘을 이용하는 방법도 있다.	(15.1, p.327-328 / PDF p.321-322)
U15-010	<thread>의 std::thread에 함수 또는 함수 객체를 넘겨 실행을 시작한다.	(15.2, p.328-329 / PDF p.322-323)
U15-011	join()은 해당 스레드의 종료를 기다린다.	(15.2, p.328-329 / PDF p.322-323)

번호	문장·표 항목·원문 코드·수식	원문 위치

U15-012	함수와 함수 객체를 스레드에서 실행 (원문 코드 발췌)
	void f(); // 함수
	
	struct F { // 함수 객체
	  void operator()(); // F의 호출 연산자(6.3.2절)
	};
	
	void user()
	{
	  thread t1 {f}; // 별도 스레드에서 f() 실행
	  thread t2 {F()}; // 별도 스레드에서 F()() 실행
	
	  t1.join(); // t1을 기다림
	  t2.join(); // t2를 기다림
	}
		(15.2, p.329 / PDF p.323)

U15-013	한 프로그램의 스레드는 주소 공간을 공유하므로 객체를 통한 통신이 가능하지만 락 등으로 접근을 제어해야 한다.	(15.2, p.329-330 / PDF p.323-324)
U15-014	원문은 두 태스크가 동기화 없이 cout을 쓰면 실행 순서가 정해지지 않아 출력이 뒤섞일 수 있음을 보여 준다.	(15.2, p.329-330 / PDF p.323-324)
U15-015	명확한 통신 외에는 태스크를 분리하고, 인자를 받아 결과를 반환하는 함수처럼 다루는 것이 가장 단순한 접근이다.	(15.2, p.329-330 / PDF p.323-324)
U15-016	태스크에는 값·포인터·참조로 데이터를 전달할 수 있다.	(15.3, p.330-331 / PDF p.324-325)
U15-017	thread의 가변 템플릿 생성자는 실행할 함수와 그 인자들을 받는다.	(15.3, p.330-331 / PDF p.324-325)
U15-018	<functional>의 ref()를 쓰면 객체를 값으로 전달하는 대신 참조로 취급한다.	(15.3, p.330-331 / PDF p.324-325)
U15-019	함수 객체는 생성 시 받은 참조를 멤버에 저장할 수도 있다.	(15.3, p.330-331 / PDF p.324-325)

번호	문장·표 항목·원문 코드·수식	원문 위치

U15-020	참조 인자를 전달하는 두 방식 (원문 코드 발췌)
	void f(vector<double>& v); // v로 무언가를 수행하는 함수
	
	struct F { // 함수 객체: v로 무언가를 수행
	vector<double>& v;
	F(vector<double>& vv) :v{vv} { }
	void operator()(); // 적용 연산자; 6.3.2절
	};
	
	int main()
	{
	vector<double> some_vec {1,2,3,4,5,6,7,8,9};
	vector<double> vec2 {10,11,12,13,14};
	
	thread t1 {f,ref(some_vec)}; // f(some_vec)을 별도의 스레드에서 실행
	thread t2 {F{vec2}}; // F(vec2)()를 별도의 스레드에서 실행
	
	t1.join();
	t2.join();
	}
		(15.3, p.330-331 / PDF p.324-325)

U15-021	원문의 F{vec2}가 vec2를 참조하는 동안 다른 태스크가 vec2에 접근해서는 안 된다.	(15.3, p.330-331 / PDF p.324-325)
U15-022	vec2를 값으로 넘기면 이 공유 위험을 피할 수 있다.	(15.3, p.330-331 / PDF p.324-325)
U15-023	컴파일러는 함수가 주어진 인자로 호출 가능한지 검사하고 스레드가 실행할 함수 객체를 구성한다.	(15.3, p.330-331 / PDF p.324-325)
U15-024	비const 참조로 입력을 받아 그것을 변경하는 방식으로 결과를 돌려줄 수도 있다.	(15.4, p.331-332 / PDF p.325-326)
U15-025	원문은 입력을 const 참조로 받고 결과 저장 위치를 별도 인자로 넘기는 편이 더 명확하다고 설명한다.	(15.4, p.331-332 / PDF p.325-326)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-026	함수 포인터 인자·함수 객체 멤버·참조 캡처 람다의 세 방식을 나란히 보여 준다.	(15.4, p.331-332 / PDF p.325-326)

번호	문장·표 항목·원문 코드·수식	원문 위치

U15-027	별도 출력 위치로 결과를 돌려받기 (원문 코드 발췌)
	void f(const vector<double>& v, double* res); // v에서 입력을 받아 *res에 결과를 저장
	
	class F {
	public:
	F(const vector<double>& vv, double* p) :v{vv}, res{p} { }
	void operator()(); // *res에 결과 저장
	private:
	const vector<double>& v; // 입력
	double* res; // 출력을 저장할 곳
	};
	
	double g(const vector<double>&); // 반환 값 사용
	
	void user(vector<double>& vec1, vector<double> vec2, vector<double> vec3)
	{
	double res1;
	double res2;
	double res3;
	
	thread t1 {f,cref(vec1),&res1}; // f(vec1,&res1)을 별도 스레드로 실행
	thread t2 {F{vec2,&res2}}; // F{vec2,&res2}()를 별도 스레드로 실행
	thread t3 { [&](){ res3 = g(vec3); }}; // 지역 변수를 참조로 전달
	
	t1.join();
	t2.join();
	t3.join();
	
	cout << res1 << ' ' << res2 << ' ' << res3 << '\n';
	}
		(15.4, p.332 / PDF p.326)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-028	모든 join() 뒤에 결과를 사용하므로 각 태스크의 완료를 기다린다.	(15.4, p.331-332 / PDF p.325-326)
U15-029	원문은 이런 참조 기반 결과 전달이 흔하고 동작하지만 덜 우아하다고 평가하고, 이후 future·promise로 더 높은 수준의 결과 전달을 설명한다.	(15.4, p.331-332 / PDF p.325-326)
U15-030	공유 데이터를 동시에 변경할 때는 한 번에 한 태스크만 해당 데이터에 접근하도록 동기화한다.	(15.5, p.333 / PDF p.327)
U15-031	변경 불가능한 데이터를 여러 태스크가 읽는 경우는 구별한다.	(15.5, p.333 / PDF p.327)
U15-032	<mutex>의 mutex는 상호 배제 객체이며 lock()으로 획득하고 unlock()으로 해제한다.	(15.5, p.333 / PDF p.327)
U15-033	scoped_lock은 이 획득·해제를 RAII로 묶는다.	(15.5, p.333 / PDF p.327)

U15-034	scoped_lock으로 공유 데이터 보호 (원문 코드 발췌)
	mutex m; // 제어용 mutex
	int sh; // 공유 데이터
	
	void f()
	{
	scoped_lock lck {m}; // mutex 획득
	sh += 7; // 공유 데이터 조작
	} // 암묵적인 mutex 해제
		(15.5, p.333 / PDF p.327)

U15-035	다른 스레드가 mutex를 가진 동안 획득 시도는 대기하며, 해제되면 기다리던 스레드가 진행한다.	(15.5, p.333-334 / PDF p.327-328)
U15-036	mutex와 보호 데이터의 관계는 관례적이므로 같은 Record 안에 두거나 이름·주석으로 관계를 명확히 해야 한다.	(15.5, p.333-334 / PDF p.327-328)
U15-037	명시적인 lock/unlock보다 자원 핸들이 간단하고 안전하다.	(15.5, p.333-334 / PDF p.327-328)
U15-038	서로 다른 순서로 여러 mutex를 획득하면 상대가 가진 mutex를 기다리느라 모두 멈추는 교착 상태(데드락)가 생길 수 있다.	(15.5, p.334 / PDF p.328)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-039	원문의 scoped_lock은 여러 mutex를 한 번에 받고 모두 획득한 뒤 진행하며, 일부만 획득한 채로 블록되지 않는다.	(15.5, p.334 / PDF p.328)
U15-040	소멸 시에는 모두 해제한다.	(15.5, p.334 / PDF p.328)
U15-041	공유 데이터는 호출·반환보다 낮은 수준의 통신이며 락 비용도 있다.	(15.5, p.334-335 / PDF p.328-329)
U15-042	데이터 복사보다 더 빠르다고 측정 없이 가정하지 않는다.	(15.5, p.334-335 / PDF p.328-329)
U15-043	shared_mutex는 읽기 스레드의 shared_lock을 함께 허용하고, 쓰기 스레드의 unique_lock에는 배타적 접근을 제공하여 읽기/쓰기 락을 구현한다.	(15.5, p.334-335 / PDF p.328-329)
U15-044	시간 경과는 가장 간단한 이벤트다.	(15.6, p.335-336 / PDF p.329-330)
U15-045	this_thread는 현재 스레드를 가리키며 sleep_for로 기다릴 수 있다.	(15.6, p.335-336 / PDF p.329-330)
U15-046	chrono의 시각 차이와 duration_cast로 경과 시간을 표시할 수 있고 별도 스레드를 만들 필요는 없다.	(15.6, p.335-336 / PDF p.329-330)
U15-047	<condition_variable>의 condition_variable은 다른 태스크의 행동으로 조건이나 이벤트가 생기기를 기다리는 수단이다.	(15.6, p.336-337 / PDF p.330-331)
U15-048	원문의 메시지 큐는 queue·조건 변수·mutex를 함께 사용한다.	(15.6, p.336-337 / PDF p.330-331)
U15-049	소비자는 unique_lock을 획득한 뒤 큐가 비어 있지 않다는 술어로 기다린다.	(15.6, p.336-337 / PDF p.330-331)
U15-050	wait는 대기 중 락을 해제하고 깨어날 때 다시 획득한다.	(15.6, p.336-337 / PDF p.330-331)

번호	문장·표 항목·원문 코드·수식	원문 위치

U15-051	조건 변수로 메시지 큐를 기다리는 소비자 (원문 코드 발췌)
	void consumer()
	{
	while(true) {
	unique_lock lck {mutex}; // mutex 획득
	mcond.wait(lck,[] { return !mqueue.empty(); }); // lck을 해제하고 대기;
	// 깨어날 때 lck을 다시 획득
	// mqueue가 비어있는 동안 깨어나지 않음
	auto m = mqueue.front(); // 메시지 가져오기
	mqueue.pop();
	lck.unlock(); // lck 해제
	// ... m을 처리 ...
	}
	}
		(15.6, p.337 / PDF p.331)

U15-052	깨어났다는 이유만으로 필요한 조건이 성립하는 것은 아니다.	(15.6, p.337-338 / PDF p.331-332)
U15-053	다른 태스크가 먼저 큐에 접근했을 수 있으므로 !mqueue.empty()를 다시 확인한다.	(15.6, p.337-338 / PDF p.331-332)
U15-054	메시지를 가져와 제거한 뒤에는 락을 해제하고 메시지를 처리해 임계 영역 시간을 줄인다.	(15.6, p.337-338 / PDF p.331-332)
U15-055	생산자는 락으로 큐 삽입을 보호하고 notify_one()으로 통지한다.	(15.6, p.337-338 / PDF p.331-332)
U15-056	unique_lock은 lock()/unlock()으로 낮은 수준의 제어가 가능하며 원문은 하나의 mutex만 다룬다고 설명한다.	(15.6, p.337 / PDF p.331)
U15-057	[원문 확인 필요] 337쪽은 scoped_lock과 비교하면서 unique_lock은 복사가 가능하다고 서술한다.	(15.6, p.337 / PDF p.331)
U15-058	이 문장은 의심되는 원문 표기로 남겨 두며, 이 노트에서 다른 규칙으로 조용히 교체하지 않는다.	(15.6, p.337 / PDF p.331)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-059	<future>는 직접 thread·락을 다루는 것보다 높은 수준의 통신 수단을 제공한다.	(15.7, p.338 / PDF p.332)
U15-060	future·promise는 태스크 사이의 결과 전달, packaged_task는 태스크 실행과 결과의 연결, async는 함수 호출과 비슷한 태스크 시작에 사용한다.	(15.7, p.338 / PDF p.332)
U15-061	promise에 값을 넣으면 연결된 future에서 그 값을 볼 수 있다.	(15.7.1, p.338-339 / PDF p.332-333)
U15-062	future<X>::get()은 필요한 경우 값 계산이 끝날 때까지 기다리고 X를 얻는다.	(15.7.1, p.338-339 / PDF p.332-333)
U15-063	값 대신 예외가 전달되거나 값을 계산할 수 없으면 get()에서 예외를 던질 수 있다.	(15.7.1, p.338-339 / PDF p.332-333)
U15-064	promise의 set_value()와 set_exception()은 future의 get()에 대응한다.	(15.7.1, p.339-340 / PDF p.333-334)
U15-065	태스크는 결과 또는 예외 중 하나를 전달한다.	(15.7.1, p.339-340 / PDF p.333-334)
U15-066	current_exception()은 현재 잡힌 예외를 참조하며, 전달된 예외는 get()의 호출 경로 어딘가에서 처리해야 한다.	(15.7.1, p.339-340 / PDF p.333-334)
U15-067	락을 명시적으로 조작하지 않고도 시스템이 전달을 수행한다.	(15.7.1, p.339-340 / PDF p.333-334)

번호	문장·표 항목·원문 코드·수식	원문 위치

U15-068	promise로 값 또는 예외 전달 (원문 코드 발췌)
	void f(promise<X>& px) // 태스크: 결과를 px에 넣기
	{
	// ...
	try {
	X res;
	// ... res에 값을 계산 ...
	px.set_value(res);
	}
	catch (...) { // res를 계산할 수 없는 경우
	px.set_exception(current_exception()); // future의 스레드로 예외 전달
	}
	}
		(15.7.1, p.339-340 / PDF p.333-334)

U15-069	packaged_task는 태스크의 반환값이나 예외를 promise에 넣는 래퍼를 제공하고, get_future()로 대응 future를 돌려준다.	(15.7.2, p.340-341 / PDF p.334-335)
U15-070	템플릿 인자는 태스크의 호출 타입이며 생성자 인자는 수행할 함수다.	(15.7.2, p.340-341 / PDF p.334-335)
U15-071	원문의 accum 예제는 vector 구간의 두 절반을 별도 태스크로 합산한다.	(15.7.2, p.340-341 / PDF p.334-335)

번호	문장·표 항목·원문 코드·수식	원문 위치

U15-072	packaged_task로 두 구간의 계산과 결과 연결 (원문 코드 발췌)
	double accum(double* beg, double* end, double init)
	// 초깃값 init부터 시작해 [beg:end)를 모두 더함
	{
	return accumulate(beg,end,init);
	}
	
	double comp2(vector<double>& v)
	{
	using Task_type = double(double*,double*,double); // 태스크의 타입
	
	packaged_task<Task_type> pt0 {accum}; // 태스크(즉, accum)를 패키징
	packaged_task<Task_type> pt1 {accum};
	
	future<double> f0 {pt0.get_future()}; // pt0의 future를 저장
	future<double> f1 {pt1.get_future()}; // pt1의 future를 저장
	
	double* first = &v[0];
	thread t1 {move(pt0),first,first+v.size()/2,0}; // pt0를 위한 스레드 시작
	thread t2 {move(pt1),first+v.size()/2,first+v.size(),0}; // pt1을 위한 스레드 시작
	
	// ...
	
	return f0.get()+f1.get(); // 결과 가져오기
	}
		(15.7.2, p.341 / PDF p.335)

U15-073	packaged_task는 promise를 소유하는 자원 핸들이므로 복사할 수 없고 thread로 전달할 때 move가 필요하다.	(15.7.2, p.341-342 / PDF p.335-336)
U15-074	명시적인 락 없이 태스크와 결과에 집중할 수 있으며 별도 스레드의 두 작업은 잠재적으로 병렬 실행된다.	(15.7.2, p.341-342 / PDF p.335-336)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-075	async()는 잠재적으로 비동기 실행할 태스크를 함수 호출과 유사하게 시작한다.	(15.7.3, p.342 / PDF p.336)
U15-076	호출 시작·결과 가져오기·실제 실행을 분리하여 thread나 락보다 태스크 자체에 집중하게 한다.	(15.7.3, p.342 / PDF p.336)
U15-077	원문은 충분히 큰 vector를 네 구간으로 나누고 future의 결과를 모으는 예를 든다.	(15.7.3, p.342 / PDF p.336)

U15-078	async로 네 구간의 결과를 모으기 (원문 코드 발췌)
	double comp4(vector<double>& v)
	// v가 충분히 크면 많은 태스크를 시작
	{
	if (v.size()<10000) // 동시성을 활용할 만한가?
	return accum(v.begin(),v.end(),0.0);
	
	auto v0 = &v[0];
	auto sz = v.size();
	
	auto f0 = async(accum,v0,v0+sz/4,0.0); // 첫 번째 1/4
	auto f1 = async(accum,v0+sz/4,v0+sz/2,0.0); // 두 번째 1/4
	auto f2 = async(accum,v0+sz/2,v0+sz*3/4,0.0); // 세 번째 1/4
	auto f3 = async(accum,v0+sz*3/4,v0+sz,0.0); // 네 번째 1/4
	
	return f0.get()+f1.get()+f2.get()+f3.get(); // 결과를 모아서 병합
	}
		(15.7.3, p.342 / PDF p.336)

U15-079	원문은 락을 이용해 데이터를 공유하는 태스크에 async를 사용할 수 없다는 한계로 서술하며, 스레드 수는 호출 시점의 시스템 자원을 고려해 결정되므로 알 수 없다고 설명한다.	(15.7.3, p.343 / PDF p.337)
U15-080	v.size()<10000 같은 기준은 스레드 시작 비용과 계산 비용에 대한 원시적인 가정으로, 성능 판단을 잘못할 수 있음을 경고한다.	(15.7.3, p.343 / PDF p.337)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-081	accumulate 같은 표준 연산을 직접 병렬화하기보다 reduce(par_unseq, ...) 같은 병렬 알고리즘이 더 잘 동작할 수 있다고 원문은 권한다.	(15.7.3, p.343 / PDF p.337)
U15-082	async는 성능을 위한 병렬 계산만을 위한 것이 아니며 메인 작업 중 사용자 입력을 받는 태스크 같은 반응성 용도도 있다.	(15.7.3, p.343 / PDF p.337)
U15-083	처리량·반응성에 동시성이 필요한지 살피고 가능한 높은 추상화 수준에서 작업한다.	(15.8, p.343-345 / PDF p.337-339)
U15-084	순차 해법과 병렬 알고리즘을 먼저 고려한다.	(15.8, p.343-345 / PDF p.337-339)
U15-085	공유 데이터와 데이터 경합을 가능한 피한다.	(15.8, p.343-345 / PDF p.337-339)
U15-086	thread의 종료는 join으로 기다린다.	(15.8, p.343-345 / PDF p.337-339)
U15-087	mutex는 보호하는 데이터와 가까이 두고 RAII 핸들로 관리한다.	(15.8, p.343-345 / PDF p.337-339)
U15-088	여러 락은 scoped_lock으로 함께 획득한다.	(15.8, p.343-345 / PDF p.337-339)
U15-089	이벤트 통신에는 condition_variable과 unique_lock을 쓰고 조건 없이 기다리지 않는다.	(15.8, p.343-345 / PDF p.337-339)
U15-090	임계 영역 시간을 최소화한다.	(15.8, p.343-345 / PDF p.337-339)
U15-091	태스크를 동시에 실행할 수 있는 함수로 생각한다.	(15.8, p.343-345 / PDF p.337-339)
U15-092	promise로 결과를 제공하고 future로 얻으며 packaged_task로 값·예외 전달을 묶는다.	(15.8, p.343-345 / PDF p.337-339)
U15-093	간단한 태스크 시작에는 async를 사용하고 thread·mutex를 직접 다루는 일을 줄인다.	(15.8, p.343-345 / PDF p.337-339)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-094	promise	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-095	값이나 예외를 넣는 쪽	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-096	future	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-097	값이 준비될 때까지 기다리고 결과·예외를 받는 쪽	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-098	packaged_task	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-099	태스크와 promise를 묶고 대응 future를 제공	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-100	async	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-101	함수 호출처럼 태스크를 시작하고 결과 획득을 분리	(태스크 통신 수단의 역할, p.338-343 / PDF p.332-337)
U15-102	scoped_lock	(원문에서 구별하는 락 핸들, p.333-337 / PDF p.327-331)
U15-103	스코프 기반 mutex 관리; 여러 mutex를 함께 획득	(원문에서 구별하는 락 핸들, p.333-337 / PDF p.327-331)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-104	unique_lock	(원문에서 구별하는 락 핸들, p.333-337 / PDF p.327-331)
U15-105	조건 변수 wait와 명시적인 lock/unlock; 한 mutex 관리	(원문에서 구별하는 락 핸들, p.333-337 / PDF p.327-331)
U15-106	shared_lock	(원문에서 구별하는 락 핸들, p.333-337 / PDF p.327-331)
U15-107	shared_mutex에서 여러 읽기 접근을 공유	(원문에서 구별하는 락 핸들, p.333-337 / PDF p.327-331)
U15-108	mutex는 상호 배제 객체이며 lock()으로 획득하고 unlock()으로 해제한다.	(15.5, p.333,335 / PDF p.327,329)
U15-109	condition_variable은 다른 태스크의 행동으로 조건이나 이벤트가 생기기를 기다리는 수단이다.	(15.6, p.336-337 / PDF p.330-331)
U15-110	promise에 값을 넣으면 연결된 future에서 그 값을 볼 수 있으며, future<X>::get()은 필요한 경우 계산이 끝날 때까지 기다린다.	(15.7.1, p.339-340 / PDF p.333-334)
U15-111	async()는 잠재적으로 비동기 실행할 태스크를 함수 호출과 유사하게 시작한다.	(15.7.3, p.342-343 / PDF p.336-337)
U15-112	처리량·반응성에 동시성이 필요한지 살피고 가능한 높은 추상화 수준에서 작업한다. 순차 해법과 병렬 알고리즘을 먼저 고려한다.	(15.8, p.343-345 / PDF p.337-339)
U15-113	공유 데이터와 데이터 경합을 가능한 피한다. thread의 종료는 join으로 기다린다.	(15.8, p.343-345 / PDF p.337-339)
U15-114	mutex는 보호하는 데이터와 가까이 두고 RAII 핸들로 관리한다. 여러 락은 scoped_lock으로 함께 획득한다.	(15.8, p.343-345 / PDF p.337-339)
U15-115	이벤트 통신에는 condition_variable과 unique_lock을 쓰고 조건 없이 기다리지 않는다. 임계 영역 시간을 최소화한다.	(15.8, p.343-345 / PDF p.337-339)

번호	문장·표 항목·원문 코드·수식	원문 위치
U15-116	future·promise	(15.7.1, p.339-340 / PDF p.333-334)
U15-117	packaged_task	(15.7.2, p.340-341 / PDF p.334-335)

제16장. 역사와 호환성

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-001	ISO C++ 위원회는 WG21로 불린다.	(16.1.3, p.355,357 / PDF p.348,350)
U16-002	두 번째 표준 C++11은 2011년 8월 21:0으로 승인됐다.	(16.1.3, p.354-355 / PDF p.347-348)
U16-003	C++는 시뮬러의 프로그램 조직화 기능과 C의 시스템 프로그래밍 효율성·유연성을 함께 제공하도록 설계됐다.	(16.1, p.347-348 / PDF p.340-341)
U16-004	클래스와 파생 클래스, 가상 함수는 시뮬러에서 영향을 받았고, 템플릿과 예외는 이후에 다른 아이디어를 바탕으로 추가됐다.	(16.1, p.347-348 / PDF p.340-341)
U16-005	원문은 언어의 진화를 실제 활용 사례와 연결한다.	(16.1, p.347-348 / PDF p.340-341)
U16-006	사용자와 동료의 의견을 듣고 당면한 문제를 해결하는 과정에서 언어와 라이브러리가 함께 발전했다.	(16.1, p.347-348 / PDF p.340-341)
U16-007	아래 연혁과 기능 상태는 원문이 서술하던 시점을 그대로 따른다.	(16.1, p.347-348 / PDF p.340-341)
U16-008	1979	(16.1.1, p.348-350 / PDF p.341-343)
U16-009	C with Classes 작업 시작. 클래스·파생 클래스, public/private, 생성자·소멸자, 타입 검사되는 함수 선언을 도입했다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-010	1984~1985	(16.1.1, p.348-350 / PDF p.341-343)
U16-011	이름을 C++로 바꾸고 가상 함수, 함수·연산자 오버로딩, 참조를 포함했다. 1985년 10월 14일 첫 상용 릴리스가 나왔다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-012	1989~1991	(16.1.1, p.348-350 / PDF p.341-343)
U16-013	ARM과 두 번째 The C++ Programming Language가 등장했다. 템플릿 기반 제네릭 프로그래밍, 예외 기반 에러 처리, 일반적인 자원 관리 관례인 RAII가 소개됐다.	(16.1.1, p.348-350 / PDF p.341-343)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-014	1997~1998	(16.1.1, p.348-350 / PDF p.341-343)
U16-015	네임스페이스·dynamic_cast·템플릿 개선과 STL을 포함한 ISO C++가 소개되고, 1998년 첫 ISO 표준이 제정됐다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-016	2002~2003	(16.1.1, p.348-350 / PDF p.341-343)
U16-017	C++0x 개정 작업을 시작했다. 2003년 표준 버그 수정과 Technical Report의 새 라이브러리 구성 요소는 이후 C++11로 이어졌다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-018	2011~2013	(16.1.1, p.348-350 / PDF p.341-343)
U16-019	C++11은 초기화, 이동, auto, 구간 for, 가변 템플릿, 람다, 타입 별칭, 동시성 메모리 모델 등을 포함했다. 원문은 완전한 구현체가 2013년에 등장했다고 서술한다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-020	2014~2015	(16.1.1, p.348-350 / PDF p.341-343)
U16-021	C++14의 가변 템플릿·자릿수 구분자·제네릭 람다·라이브러리 개선, Core Guidelines 프로젝트 시작, 컨셉트 TS 승인을 기록한다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-022	2017	(16.1.1, p.348-350 / PDF p.341-343)
U16-023	C++17과 첫 구현체, 모듈 TS 및 Ranges TS 승인을 기록한다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-024	2020	(16.1.1, p.348-350 / PDF p.341-343)
U16-025	원문에서는 ISO C++20 표준을 예정 사항으로 적었다. 현재 상태로 바꾸지 않는다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-026	C++11은 개발 중 C++0x로 불렸다.	(16.1.1, p.348-350 / PDF p.341-343)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-027	원문은 완료 시점에 대한 낙관과 긴 지연을 설명하면서, C++14·C++17은 정해진 시점에 완료됐다고 대조한다.	(16.1.1, p.348-350 / PDF p.341-343)
U16-028	처음 해결하려던 문제는 멀티프로세서와 로컬 네트워크에 유닉스 커널 서비스를 배포하는 일이었다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-029	시스템 부분과 상호작용을 정확히 명시하면서 하드웨어를 직접 다루고 높은 성능을 얻어야 했다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-030	시뮬러는 조직화에 적합했으나 성능이 문제였고, C는 하드웨어와 성능에 적합했으나 모듈화가 어려웠다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-031	이 두 요구를 결합한 것이 C with Classes였다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-032	초기에는 이후의 연산자 오버로딩·참조·가상 함수·템플릿·예외 등이 없었다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-033	C++라는 이름은 릭 매시티가 1983년 여름 제안했고, C의 증가 연산자 ++를 사용해 C의 확장을 표현했다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-034	설계·문서화·구현은 함께 진행됐으며, 초창기의 진화는 공식 위원회보다 사용자 문제에 대한 토론에 기초했다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-035	초기 설계의 중심에는 함수 인자 타입 검사, 클래스의 public/private에 의한 인터페이스와 구현 분리, 생성자와 소멸자가 있었다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-036	원문은 생성자·소멸자 도입을 특히 중요하게 평가한다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-037	객체의 실행 환경을 만들고 되돌리는 기능이 자원 관리 전략과 간결한 표현 기법의 출발점이 됐기 때문이다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-038	가상 함수는 실행 시간의 선택을 표현하는 다형성을 제공했지만, 시스템 코드에 충분히 빠른지에 대한 논쟁이 있었다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-039	원문은 근본적으로 복잡한 시스템을 만들 때 언어와 도구의 도움을 활용해야 한다고 설명한다.	(16.1.2, p.350-354 / PDF p.343-347)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-040	C++를 범용 언어, 더 나은 C, 데이터 추상화를 제공하며 객체지향 프로그래밍을 지원하는 언어로 서술한다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-041	데이터 추상화에는 정보 은닉뿐 아니라 클래스 계층에 속하지 않는 클래스와 제네릭 프로그래밍도 포함한다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-042	템플릿을 설계할 때 유연성·효율성·이른 타입 검사를 동시에 얻기 어려웠다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-043	C 스타일 시스템 코드와 경쟁하기 위해 유연성과 효율성을 선택했고, 타입 확인 개선은 이어졌다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-044	예외 설계는 여러 호출 수준으로의 전파, 핸들러에 임의 정보 전달, 로컬 객체의 소멸자에 기반한 자원 관리와의 통합에 초점을 뒀다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-045	이 자원 관리 기법을 Resource Acquisition Is Initialization, RAII라 부른다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-046	추상 클래스의 다중 상속을 포함한 상속 일반화도 이루어졌지만, 원문은 템플릿과 예외보다 중요도가 낮다고 평가한다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-047	복소수·벡터·스택·입출력 스트림을 설계하면서 연산자 오버로딩도 설계했고, 문자열·리스트와 템플릿 라이브러리 경험이 언어에 영향을 주었다. 1998년 표준 라이브러리의 주요 혁신은 제네릭 컨테이너와 알고리즘의 프레임워크인 STL이었다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-048	초기의 태스크 라이브러리와 달리 표준 동시성 기능이 널리 사용되기까지는 2011년까지 기다려야 했다고 서술한다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-049	코루틴의 C++20 포함은 원문에서 예정으로 표현된다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-050	여러 언어의 영향보다 당면한 응용 분야의 요구를 결정적인 기준으로 삼은 문제 지향적 개발이 핵심이다.	(16.1.2, p.350-354 / PDF p.343-347)
U16-051	사용이 늘어나면서 구현자와 사용자 사이의 일관된 약속이 필요해졌다. 1987년 무렵 표준화의 필요성이 분명해졌고, 공유·검토된 레퍼런스 매뉴얼이 ANSI 표준의 기초가 됐다.	(16.1.3, p.354-357 / PDF p.347-350)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-052	ANSI X3J16 위원회는 1989년 12월 소집됐고, 1991년에는 ISO 작업에 포함됐다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-053	ISO C++ 위원회는 WG21로 불린다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-054	첫 ISO C++ 표준은 1998년 22:0으로 승인됐다. 2003년 버그 수정판은 원문에서 기본적으로 C++98과 같은 언어라고 설명한다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-055	두 번째 표준 C++11은 2011년 8월 21:0으로 승인됐다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-056	까다로워진 작업 절차는 명세의 엄격성을 높이는 동시에 혁신을 제한하기도 했다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-057	표준 문서가 커진 것은 새 기능뿐 아니라 더 자세한 명세 때문이었다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-058	기존 수십억 줄의 코드를 망가뜨리지 않도록 장기간 안정성을 유지하는 것도 필수 요구였다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-059	C++11의 전반적인 목표는 시스템 프로그래밍과 라이브러리 제작에 더 적합한 언어를 만들고, 가르치고 배우기 쉽게 하는 것이었다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-060	타입 안전성과 이식성을 갖춘 동시성, 적합한 메모리 모델과 스레드 라이브러리는 이 목표의 일부다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-061	C++11 이후에는 13년 간격이 너무 길다는 공감 아래 정해진 시점에 표준을 내는 기차 모델과 3년 주기를 채택했다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-062	기능을 병렬로 개발하기 위해 기술 명세 TS를 사용했지만, 추가 단계와 지연이 생기기도 했다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-063	원문은 C++17이 계획된 대규모 변화에 이르지 못했고 컨셉·모듈·레인지가 준비되지 않았다고 평가한다.	(16.1.3, p.354-357 / PDF p.347-350)
U16-064	이 평가는 원문 시점의 평가로 보존한다.	(16.1.3, p.354-357 / PDF p.347-350)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-065	표준은 기능의 동작을 규정하지만 효과적인 사용법을 모두 알려주지는 않는다.	(16.1.4, p.357-358 / PDF p.350-351)
U16-066	기능 자체의 기술적 이해와, 기능·라이브러리·도구를 함께 써서 관리하기 쉽고 에러가 적으며 빠른 소프트웨어를 만드는 일은 구별된다.	(16.1.4, p.357-358 / PDF p.350-351)
U16-067	일관된 프로그래밍 스타일을 개발·지원하고 기존 코드도 그 스타일로 발전시키는 일이 필요하다.	(16.1.4, p.357-358 / PDF p.350-351)
U16-068	2015년에 시작한 C++ Core Guidelines는 원문에서 타입 안전성과 자원 안전성에 초점을 둔 노력으로 소개된다.	(16.1.4, p.357-358 / PDF p.350-351)
U16-069	정적 분석 도구와 이를 지원하는 라이브러리의 도움으로 효과적인 스타일을 지원하려는 것이다.	(16.1.4, p.357-358 / PDF p.350-351)
U16-070	이 교재는 원문이 언급한 목적만 정리하며 외부 지침의 세부 내용을 덧붙이지 않는다.	(16.1.4, p.357-358 / PDF p.350-351)
U16-071	원문은 C++를 산업계의 요구 속에서 발전한 언어로 설명한다.	(16.1.5, p.358-359 / PDF p.1,351)
U16-072	장치 드라이버·네트워킹·임베디드 시스템을 비롯해 운영체제, 웹 기반 시설, 금융, 의료, 자동차, 항공우주, 과학, 그래픽스·게임 등 여러 응용 분야를 든다.	(16.1.5, p.358-359 / PDF p.1,351)
U16-073	공통점은 하드웨어를 효과적으로 사용하면서 복잡도를 낮추는 능력이 필요하다는 것이다.	(16.1.5, p.358-359 / PDF p.1,351)
U16-074	사용 분야와 사용자 수에 관한 서술은 원문의 역사적 관점에 속한다.	(16.1.5, p.358-359 / PDF p.1,351)
U16-075	이 절은 C++11·C++14·C++17에 추가된 언어 기능과 표준 라이브러리를 구별해 나열한다.	(16.2, p.359 / PDF p.1)
U16-076	기능은 따로 고립돼 쓰이지 않으며, 새 기능 상당수는 기존 기능이 제공하는 틀 위에서 의미를 갖는다.	(16.2, p.359 / PDF p.1)
U16-077	아래 목록은 원문의 기능 이름을 학습 항목별로 묶은 것이며, 이 장에서 설명하지 않은 세부 규칙은 새로 보충하지 않는다.	(16.2, p.359 / PDF p.1)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-078	초기화·상수·타입	(16.2.1, p.359-361 / PDF p.1-3)
U16-079	{} 범용 초기화, auto 타입 추론, 타입 축소 방지, constexpr, enum class, static_assert, initializer_list 매핑	(16.2.1, p.359-361 / PDF p.1-3)
U16-080	순회·값·함수	(16.2.1, p.359-361 / PDF p.1-3)
U16-081	구간 for, nullptr, rvalue 참조와 이동, 람다, 가변 템플릿 인자, 타입·템플릿 별칭	(16.2.1, p.359-361 / PDF p.1-3)
U16-082	문자·표현·선언	(16.2.1, p.359-361 / PDF p.1-3)
U16-083	유니코드 문자, long long, alignas/alignof, decltype, 로우 문자열, 일반화된 POD·union, 로컬 클래스의 템플릿 인자 사용, 접미사 반환 타입 구문	(16.2.1, p.359-361 / PDF p.1-3)
U16-084	예외·속성	(16.2.1, p.359-361 / PDF p.1-3)
U16-085	[[carries_dependency]], [[noreturn]], noexcept 지정자와 noexcept 연산자	(16.2.1, p.359-361 / PDF p.1-3)
U16-086	매크로·이름 공간	(16.2.1, p.359-361 / PDF p.1-3)
U16-087	C99 관련 확장 정수·문자열·매크로 기능, __func__, inline 네임스페이스	(16.2.1, p.359-361 / PDF p.1-3)
U16-088	생성·변환·템플릿 제어	(16.2.1, p.359-361 / PDF p.1-3)
U16-089	생성자 위임, 클래스 내 멤버 초기화, default/delete, 명시적 변환 연산자, 사용자 정의 리터럴, extern template, 함수 템플릿 기본 인자, 생성자 상속	(16.2.1, p.359-361 / PDF p.1-3)
U16-090	언어 규칙·동시성	(16.2.1, p.359-361 / PDF p.1-3)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-091	중첩 템플릿 인자의 >> 표기, override/final, SFINAE 규칙, 메모리 모델, thread_local	(16.2.1, p.359-361 / PDF p.1-3)
U16-092	함수 반환 타입 추론과 개선된 constexpr 함수(예: for 루프 허용).	(16.2.2, p.361 / PDF p.3)
U16-093	가변 템플릿, 바이너리 리터럴과 자릿수 구분자.	(16.2.2, p.361 / PDF p.3)
U16-094	제네릭 람다, 더 일반적인 람다 캡처, [[deprecated]] 속성과 작은 확장들.	(16.2.2, p.361 / PDF p.3)
U16-095	안전성이 보장된 복사 생략, 과정렬된 타입의 동적 할당, 더 엄격한 평가 순서.	(16.2.3, p.361-362 / PDF p.3-4)
U16-096	UTF-8 리터럴(u8), 16진 부동소수점 리터럴, 접힘 표현식.	(16.2.3, p.361-362 / PDF p.3-4)
U16-097	auto 템플릿 파라미터, 클래스 템플릿 인자 타입 추론, 컴파일 시간 if.	(16.2.3, p.361-362 / PDF p.3-4)
U16-098	초깃값을 포함한 선택 구문, constexpr 람다, inline 변수, 구조화된 바인딩.	(16.2.3, p.361-362 / PDF p.3-4)
U16-099	[[fallthrough]], [[nodiscard]], [[maybe_unused]], std::byte, 기저 타입 값으로 enum 초기화와 작은 확장들.	(16.2.3, p.361-362 / PDF p.3-4)
U16-100	C++11 라이브러리의 확장은 새 구성 요소의 추가와 기존 구성 요소의 개선을 모두 포함한다.	(16.2.4, p.362-363 / PDF p.4-5)
U16-101	예를 들어 정규 표현식은 새 구성 요소이고, 컨테이너의 이동 생성자 지원은 기존 구성 요소의 확장이다.	(16.2.4, p.362-363 / PDF p.4-5)
U16-102	컨테이너·자원	(16.2.4, p.362-363 / PDF p.4-5)
U16-103	initializer_list 생성자, 컨테이너 이동, forward_list, unordered_map/unordered_multimap/unordered_set/unordered_multiset, unique_ptr/shared_ptr/weak_ptr, array, emplace(), 스코프 할당자	(16.2.4, p.362-363 / PDF p.4-5)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-104	동시성·예외	(16.2.4, p.362-363 / PDF p.4-5)
U16-105	thread, 뮤텍스·락·조건 변수, future·promise·async(), 예외 복사와 다시 던짐, system_error, atomic	(16.2.4, p.362-363 / PDF p.4-5)
U16-106	범용 라이브러리	(16.2.4, p.362-363 / PDF p.4-5)
U16-107	tuple, regex, 난수 분포·엔진, 정수 타입 이름, constexpr/noexcept 사용, function·bind(), 문자열 수치 변환	(16.2.4, p.362-363 / PDF p.4-5)
U16-108	특성·시간·알고리즘	(16.2.4, p.362-363 / PDF p.4-5)
U16-109	타입 특성, duration/time_point, ratio, quick_exit, move()/copy_if()/is_sorted(), 가비지 컬렉션 ABI	(16.2.4, p.362-363 / PDF p.4-5)
U16-110	[원문 확인 필요] 원문 363쪽의 높은 수준 동시성 목록에는 packaged_thread라고 인쇄돼 있다.	(16.2.4, p.362-363 / PDF p.4-5)
U16-111	이 장에서는 원문 표기를 보존하며, 다른 식별자로 조용히 고치지 않는다.	(16.2.4, p.362-363 / PDF p.4-5)
U16-112	해당 기능의 구체적 코드는 15장 원문과 구별해 확인한다.	(16.2.4, p.362-363 / PDF p.4-5)
U16-113	shared_mutex와 사용자 정의 리터럴.	(16.2.5, p.363-364 / PDF p.5-6)
U16-114	타입을 이용한 튜플 접근, 연관 컨테이너의 이형 조회, 작은 기능 추가.	(16.2.5, p.363-364 / PDF p.5-6)
U16-115	파일 시스템, 병렬 알고리즘, 특수 수학 함수.	(16.2.6, p.364 / PDF p.6)
U16-116	string_view, any, variant, optional, invoke().	(16.2.6, p.364 / PDF p.6)
U16-117	기초 문자열 변환 to_chars/from_chars, 다형적 할당과 작은 확장.	(16.2.6, p.364 / PDF p.6)
U16-118	광범위한 기존 코드 때문에 기능 제거는 오랜 경고 뒤에 신중히 이뤄진다.	(16.2.7, p.364-365 / PDF p.6-7)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-119	원문은 C++17에서 예외 명세와 그 지원 기능인 unexcepted_handler·set_unexpected()·get_unexpected()·unexpected()가 제거됐다고 설명하고 noexcept 사용을 권한다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-120	[원문 확인 필요] unexcepted_handler는 원문 365쪽의 표기를 그대로 옮겼다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-121	원문에서 제거 사례로 든 예외 명세 (원문 코드 발췌) void f() throw(X,Y); // C++98 스타일, 이제부턴 에러	(16.2.7, p.364 / PDF p.6)
U16-122	문자 세 개의 예약어(trigraphs), 저장 영역 명시자 register, bool에 대한 ++가 제거되거나 허용되지 않는다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-123	auto_ptr 대신 unique_ptr을 사용한다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-124	C++98의 export 기능은 제거됐고, 원문은 모듈 키워드의 export를 따로 언급한다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-125	소멸자가 있는 클래스에서 복사 연산을 자동 생성하는 기능은 지원 중단으로 설명된다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-126	문자열을 char*에 대입하는 기능은 제거됐으며 const char* 또는 auto를 사용하도록 한다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-127	기존 일부 함수 객체와 관련 함수는 람다와 function으로 대체하도록 권한다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-128	지원 중단은 해당 기능을 피하라는 강한 신호이지만 즉시 제거와 동일하지 않다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-129	컴파일러는 경고를 낼 수 있고, 호환성을 위해 지원 중단 기능이 오래 유지될 수도 있다.	(16.2.7, p.364-365 / PDF p.6-7)
U16-130	원문은 사소한 예외를 제외하면 C++가 C11의 대부분을 포함하며, 차이의 상당 부분이 C++의 엄격한 타입 검사에서 생긴다고 설명한다.	(16.3, p.365 / PDF p.7)
U16-131	이 설명은 두 언어의 모든 프로그램과 기능이 완전히 호환된다는 뜻으로 확대하지 않는다.	(16.3, p.365 / PDF p.7)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-132	차이는 컴파일러 진단으로 확인할 수 있다고 서술한다.	(16.3, p.365 / PDF p.7)
U16-133	ISO C와 ISO C++는 Classic C에서 갈라져 서로 다른 속도와 방향으로 발전한 형제 언어로 설명된다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-134	둘 다 전통적인 C 스타일을 제공하지만 방식이 약간 다르고, 둘 모두 Classic C와도 100% 호환되지는 않는다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-135	가계도는 기능을 직접 계승한 관계와 다른 언어에서 차용한 관계를 구별한다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-136	호환성 차이의 원인이 언제나 C++의 변경인 것은 아니다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-137	분리 후 C에 추가된 기능에서 생긴 차이도 있다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-138	원문은 T*와 void* 대입, 전역 const의 링크, inline의 세부 의미를 예로 든다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-139	한 언어의 라이브러리를 다른 언어에서 쓰거나 공통 도구를 만드는 경우 이 차이를 고려해야 한다.	(16.3.1, p.366-367 / PDF p.8-9)
U16-140	설계와 프로그래밍 스타일이 C++의 기능을 활용하지 못할 수 있다.	(16.3.2, p.367 / PDF p.9)
U16-141	void*를 캐스트 없이 T*로 바꾸는 코드는 문제가 될 수 있다.	(16.3.2, p.367 / PDF p.9)
U16-142	class·private 등 C++ 키워드를 C 코드의 식별자로 쓸 수 있다.	(16.3.2, p.367 / PDF p.9)
U16-143	C와 C++로 각각 컴파일한 코드 사이의 링크 호환성을 고려해야 한다.	(16.3.2, p.367 / PDF p.9)
U16-144	다른 문제가 없다면 C 코드를 C로 컴파일한 뒤 extern "C"를 이용해 C++에 링크할 수 있다.	(16.3.2, p.367 / PDF p.9)
U16-145	따라서 문법 변환, 설계 개선, 링크 문제는 각각 구별해서 다뤄야 한다.	(16.3.2, p.367 / PDF p.9)
U16-146	C 프로그램을 한 줄씩 C++로 바꾸는 것만으로는 기본 구조와 에러의 근본 원인이 사라지지 않는다.	(16.3.2.1, p.367-369 / PDF p.9-11)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-147	원문은 불완전한 에러 처리, 자원 누수, 버퍼 오버플로우가 번역 뒤에도 남을 수 있다고 설명한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-148	혜택을 얻으려면 설계와 구현 스타일도 바꿔야 한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-149	표준 라이브러리를 새로운 스타일의 학습 자료로 사용하고 C 라이브러리와의 차이를 확인한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-150	매크로 대신 원문에 제시된 const·constexpr·enum/enum class·inline·template·namespace를 목적에 맞게 사용한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-151	변수를 필요 지점에서 선언하고 초기화한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-152	원문은 구문 위치, for 초기화 절, 조건 절에서의 선언을 언급한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-153	malloc 대신 new, realloc 대신 vector를 권하지만 malloc/free를 무조건 new/delete로 맞바꾸는 방식은 피하도록 한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-154	매우 낮은 수준의 구현 외에는 void*·공용체·캐스트를 줄인다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-155	명시 변환이 불가피하면 의도가 드러나는 명명된 캐스트를 사용한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-156	배열·C 스타일 문자열 대신 string·array·vector를 활용한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-157	특화 코드나 단순 배열 순회 외에는 포인터 산술을 사용하지 않도록 한다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-158	장황한 C 스타일 코드가 더 짧은 표준 라이브러리 사용 코드보다 반드시 효율적이라고 가정하지 않는다.	(16.3.2.1, p.367-369 / PDF p.9-11)
U16-159	C에서 void*를 다른 포인터 타입의 대입 우측이나 초깃값으로 사용할 수 있는 경우에도 C++에서는 항상 허용되지 않는다.	(16.3.2.2, p.369-370 / PDF p.11-12)
U16-160	원문은 malloc의 결과를 int*로 바로 초기화하는 코드를 이 차이의 예로 제시한다.	(16.3.2.2, p.369-370 / PDF p.11-12)

번호	문장·표 항목·원문 코드·수식	원문 위치

U16-161	malloc 결과의 암묵적 포인터 변환 (원문 코드 발췌)
	void f(int n)
	{
	int* p = malloc(n*sizeof(int)); /* C++에서는 불허. C++의 "new"로 할당하자. */
	// ...
	}
		(16.3.2.2, p.369 / PDF p.11)

U16-162	이 제한은 단순한 불편만을 의미하지 않는다.	(16.3.2.2, p.369-370 / PDF p.11-12)
U16-163	void*를 임의의 포인터 타입으로 암묵적으로 바꿀 때 잘못된 메모리 접근이 생길 수 있음을 다음 예가 보여 준다.	(16.3.2.2, p.369-370 / PDF p.11-12)
U16-164	결과를 수치나 출력으로 새로 예측하지 않는다.	(16.3.2.2, p.369-370 / PDF p.11-12)

U16-165	void* 변환이 초래할 수 있는 문제 (원문 코드 발췌)
	char ch;
	void* pv = &ch;
	int* pi = pv; // C++에서는 허용하지 않음
	*pi = 666; // ch와 근처의 바이트를 덮어씀
		(16.3.2.2, p.369 / PDF p.11)

U16-166	원문은 두 언어에서 모두 사용할 malloc 코드라면 결과를 적절한 타입으로 캐스트해야 한다고 설명한다.	(16.3.2.2, p.369-370 / PDF p.11-12)
U16-167	C++만 사용할 때는 malloc을 사용하지 않도록 권한다.	(16.3.2.2, p.369-370 / PDF p.11-12)
U16-168	C와 C++는 서로 다른 링크 방식으로 구현될 수 있다.	(16.3.2.3, p.370 / PDF p.12)
U16-169	원문은 타입 검사와 함수 오버로딩을 그 이유로 든다.	(16.3.2.3, p.370 / PDF p.12)
U16-170	C 함수와 C++ 코드 사이에서 호출할 수 있게 하려면 C 링크를 명시한다.	(16.3.2.3, p.370 / PDF p.12)
U16-171	C 링크를 지정하는 선언 (원문 코드 발췌) extern "C" double sqrt(double);	(16.3.2.3, p.370 / PDF p.12)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-172	이 선언을 통해 C 또는 C++ 코드 조각에서 sqrt(double)을 호출할 수 있으며, 정의는 C나 C++ 함수로 컴파일될 수 있다.	(16.3.2.3, p.370 / PDF p.12)
U16-173	C는 오버로딩을 지원하지 않으므로 한 스코프에서 특정 이름으로 C에 링크되는 함수는 하나뿐이다.	(16.3.2.3, p.370 / PDF p.12)
U16-174	링크 명세는 타입 검사를 바꾸지 않으며, C++ 함수 호출·인자 검사 규칙이 그대로 적용된다.	(16.3.2.3, p.370 / PDF p.12)
U16-175	제공 파일에서 확인되는 참고 자료는 Boost 라이브러리 항목과 1990년 C 표준 항목까지다.	(16.4, p.370 / PDF p.12)
U16-176	여기서 인용된 자료는 추가 지식의 출처로 사용하지 않았다.	(16.4, p.370 / PDF p.12)
U16-177	이어지는 인쇄 371쪽 이후의 목록은 원문에서 확인 불가.	(16.4, p.370 / PDF p.12)
U16-178	원문에서 확인 불가 목차에는 16.5 조언이 인쇄 376쪽부터 시작한다고 나오지만 해당 본문은 제공되지 않았다.	(16.5, PDF p.17)
U16-179	목차의 위치만 확인했으며 조언의 내용을 만들어 넣지 않았다.	(16.5, PDF p.17)
U16-180	주요 역할	(표준과 스타일, p.357-358 / PDF p.350-351)
U16-181	무엇이 어떻게 동작하는지 규정	(표준과 스타일, p.357-358 / PDF p.350-351)
U16-182	기능·라이브러리·도구를 함께 효과적으로 사용	(표준과 스타일, p.357-358 / PDF p.350-351)
U16-183	원문의 관심	(표준과 스타일, p.357-358 / PDF p.350-351)
U16-184	기능의 정확한 명세	(표준과 스타일, p.357-358 / PDF p.350-351)
U16-185	관리하기 쉽고 에러가 적으며 빠른 소프트웨어	(표준과 스타일, p.357-358 / PDF p.350-351)
U16-186	지원 중단	(지원 중단과 제거, p.364-365 / PDF p.6-7)

번호	문장·표 항목·원문 코드·수식	원문 위치
U16-187	피해야 한다는 강한 신호이며 경고를 낼 수 있다. 호환성 때문에 유지될 수도 있다.	(지원 중단과 제거, p.364-365 / PDF p.6-7)
U16-188	제거	(지원 중단과 제거, p.364-365 / PDF p.6-7)
U16-189	표준에서 기능을 없앤다. 원문은 대규모 기존 코드 때문에 오랜 경고와 신중함이 필요하다고 설명한다.	(지원 중단과 제거, p.364-365 / PDF p.6-7)
U16-190	설계와 스타일	(C/C++ 호환성의 서로 다른 층위, p.367-370 / PDF p.9-12)
U16-191	문법만 옮겨도 자원 누수·불완전한 에러 처리의 근본 원인은 남을 수 있다.	(C/C++ 호환성의 서로 다른 층위, p.367-370 / PDF p.9-12)
U16-192	타입 검사	(C/C++ 호환성의 서로 다른 층위, p.367-370 / PDF p.9-12)
U16-193	void*의 암묵적 변환처럼 C++에서 더 엄격한 제한이 있다.	(C/C++ 호환성의 서로 다른 층위, p.367-370 / PDF p.9-12)
U16-194	링크	(C/C++ 호환성의 서로 다른 층위, p.367-370 / PDF p.9-12)
U16-195	각 언어로 컴파일한 코드를 연결할 때 extern "C"로 C 링크를 명시한다.	(C/C++ 호환성의 서로 다른 층위, p.367-370 / PDF p.9-12)
U16-196	표준과 스타일	(16.1.4, p.357-358 / PDF p.350-351)
U16-197	지원 중단	(16.2.7, p.364-365 / PDF p.6-7)